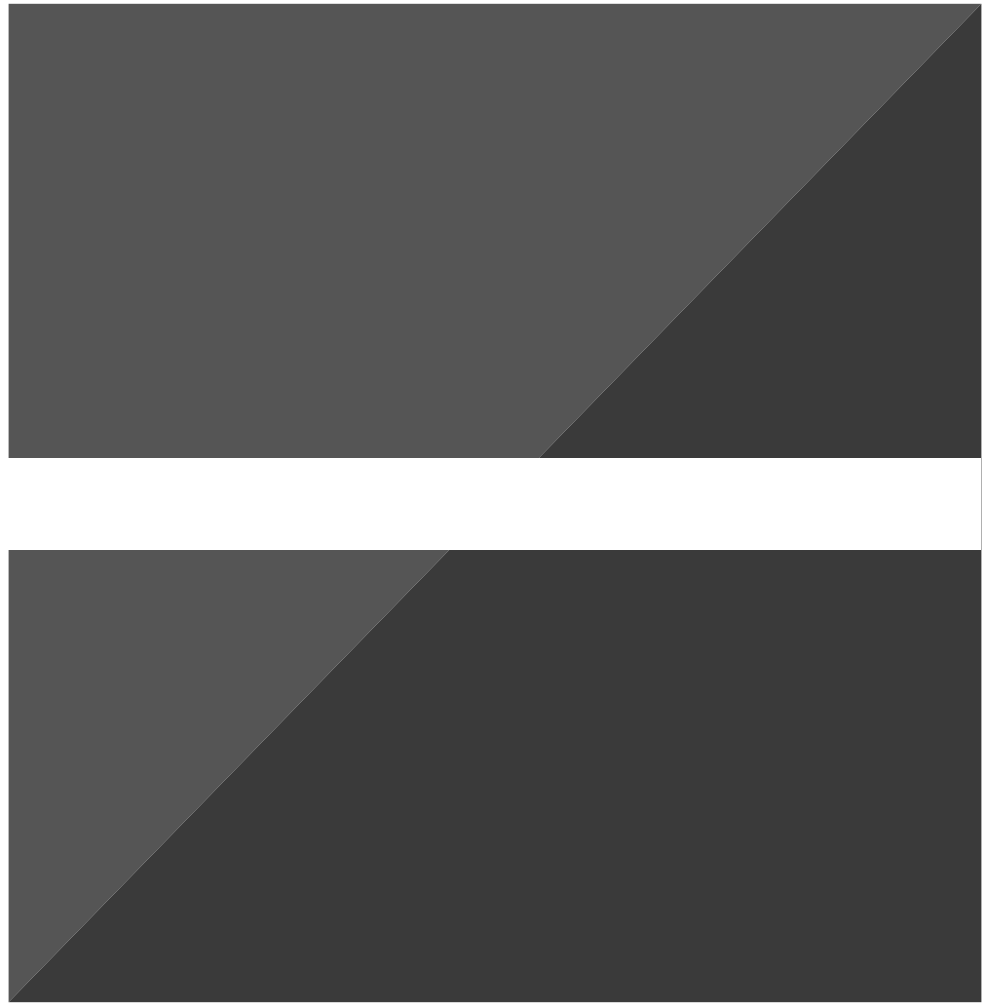
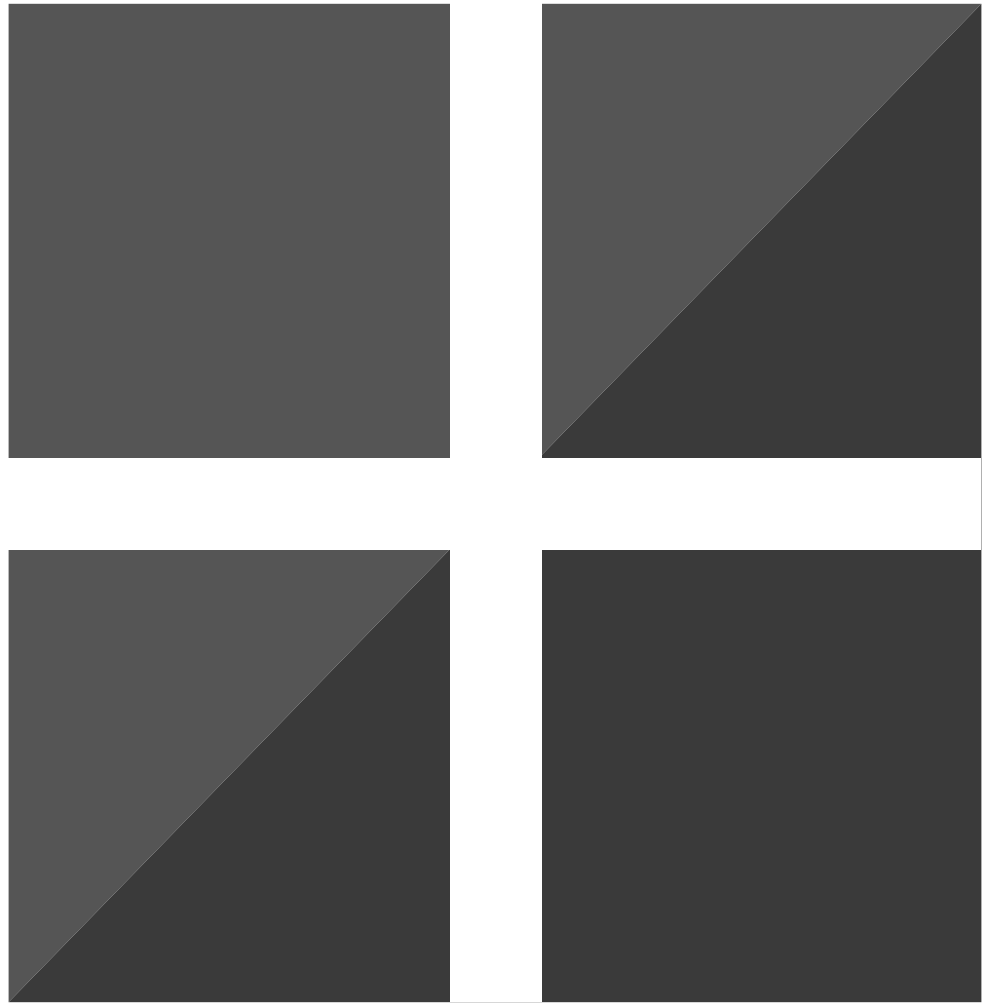
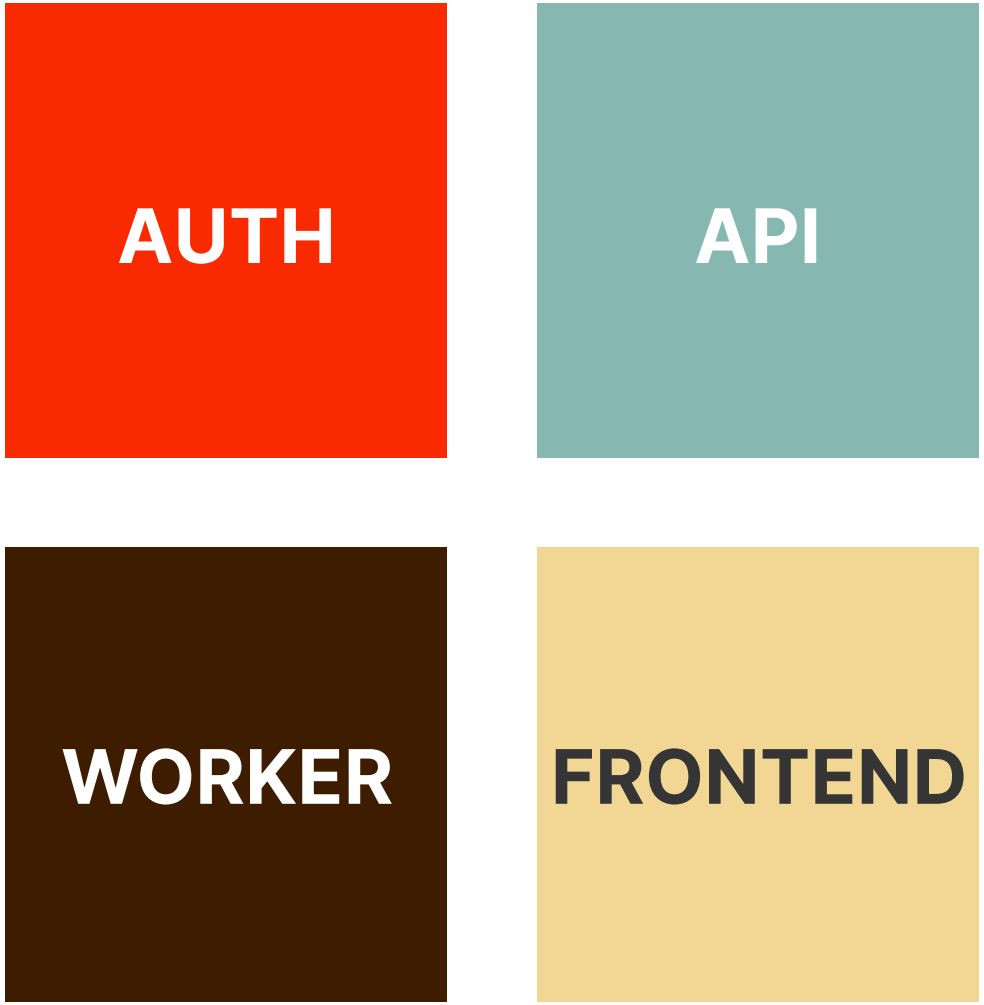
The case for containers

01


MONOLITH






AUTH
API
WORKER
FRONTEND

Benefits of using containers

Quicker deployments

Easier to iterate on

Resilient

Benefits of using containers

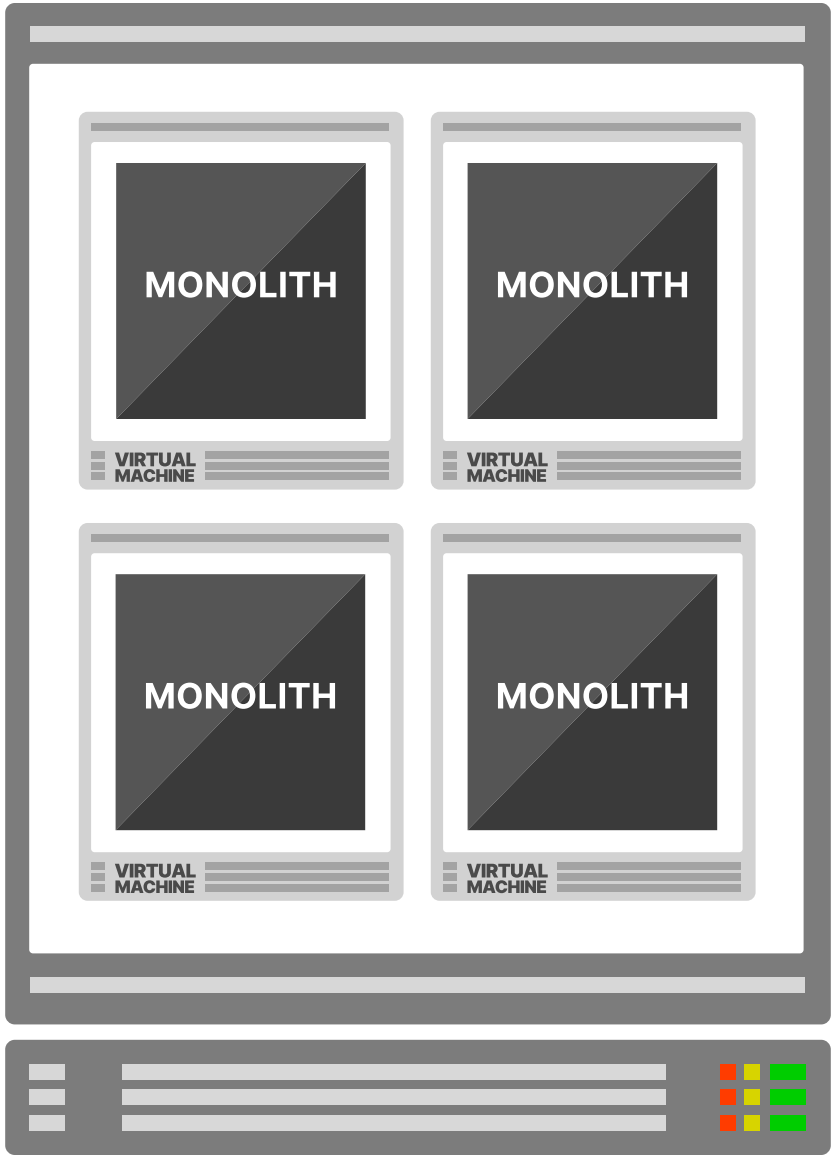
Quicker deployments

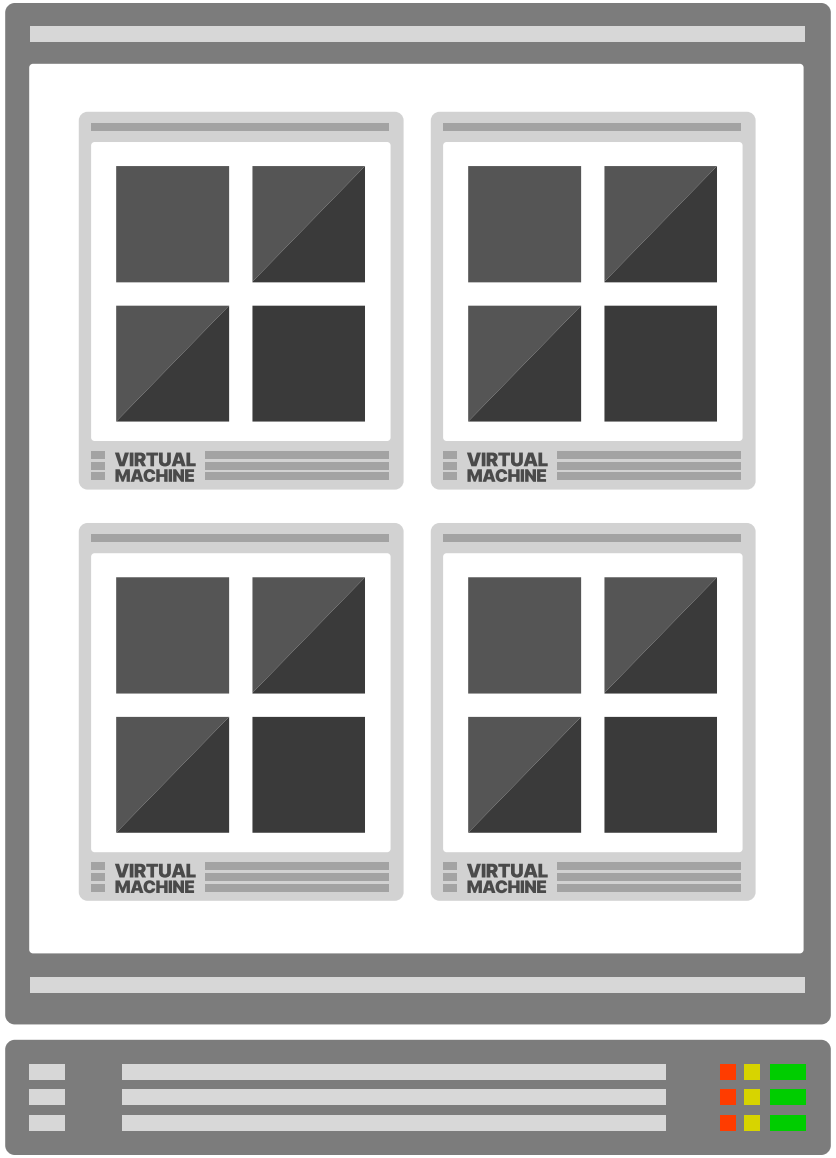
Easier to iterate on

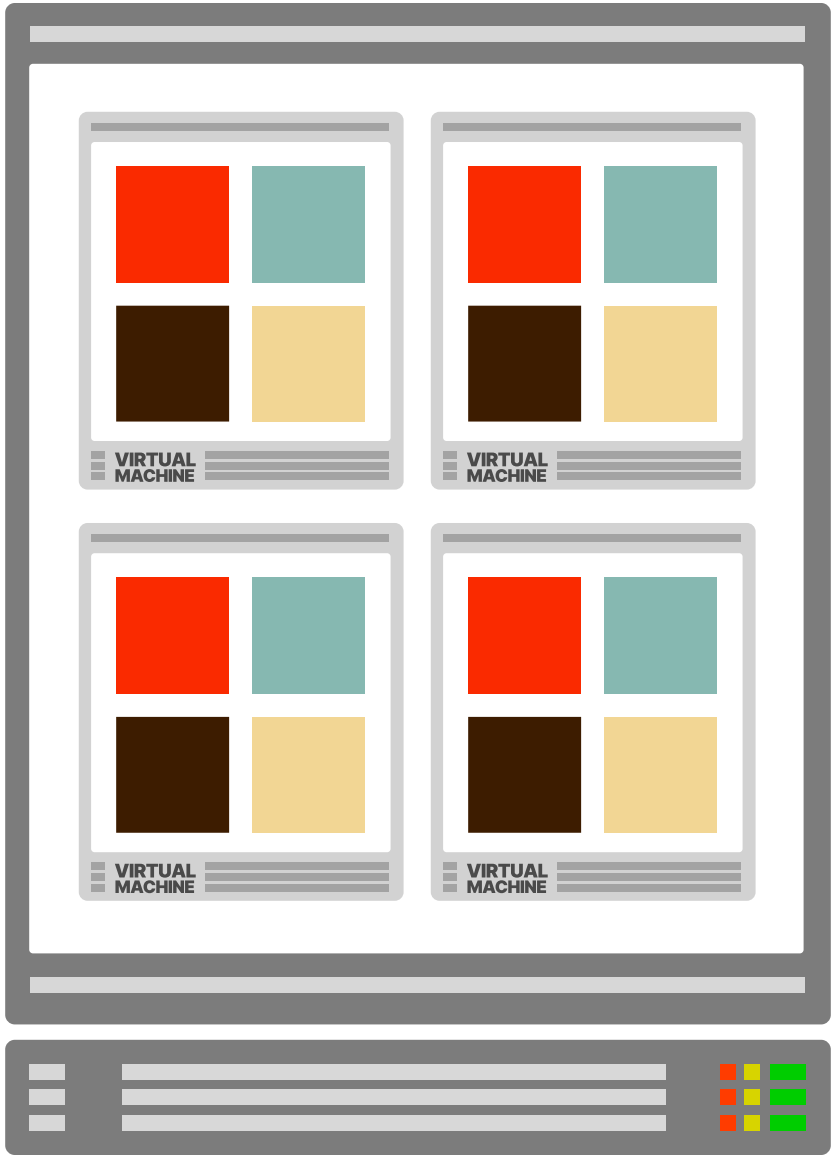
Resilient

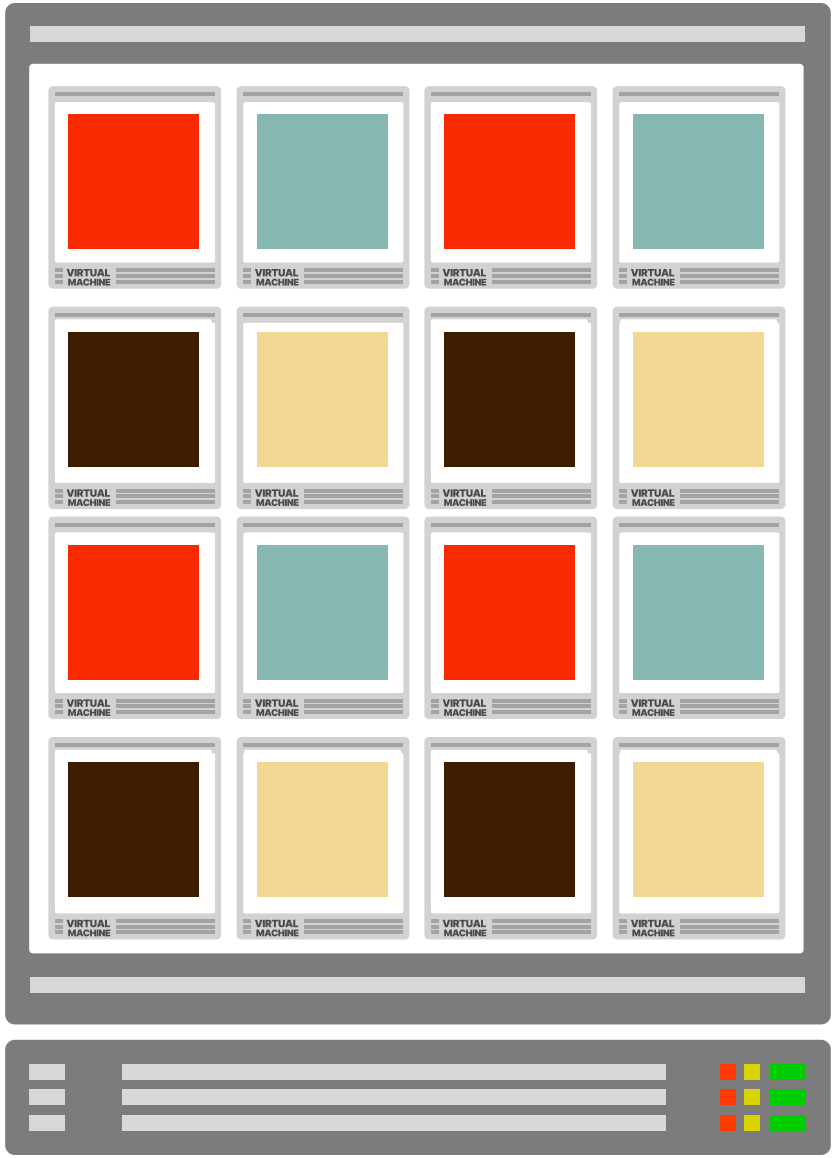
Benefits of using containers

Quicker deployments
Easier to iterate on
Resilient


MONOLITH
VIRTUAL MACHINE
MONOLITH
VIRTUAL MACHINE
MONOLITH
VIRTUAL MACHINE
MONOLITH
VIRTUAL MACHINE


VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE

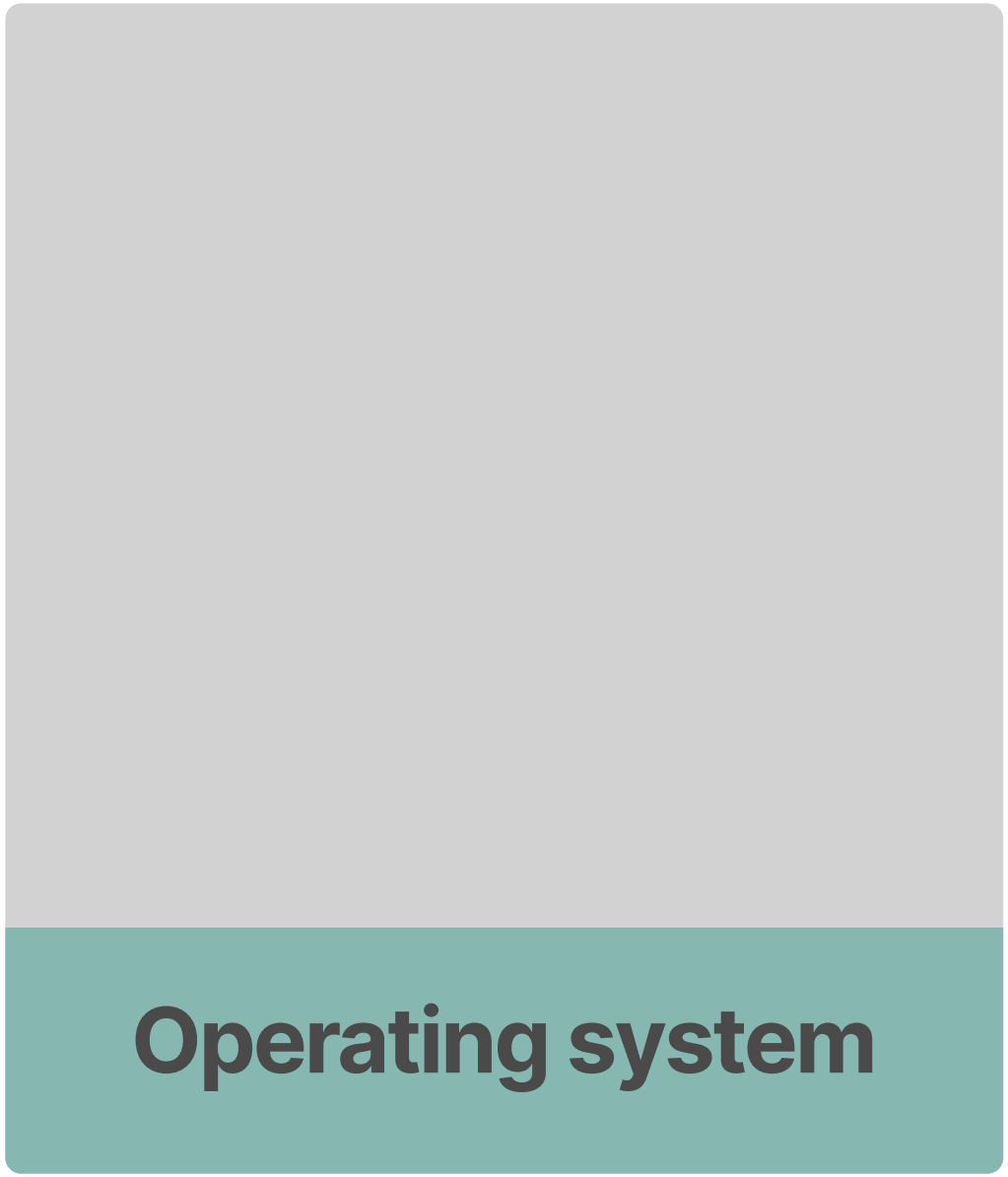

VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE

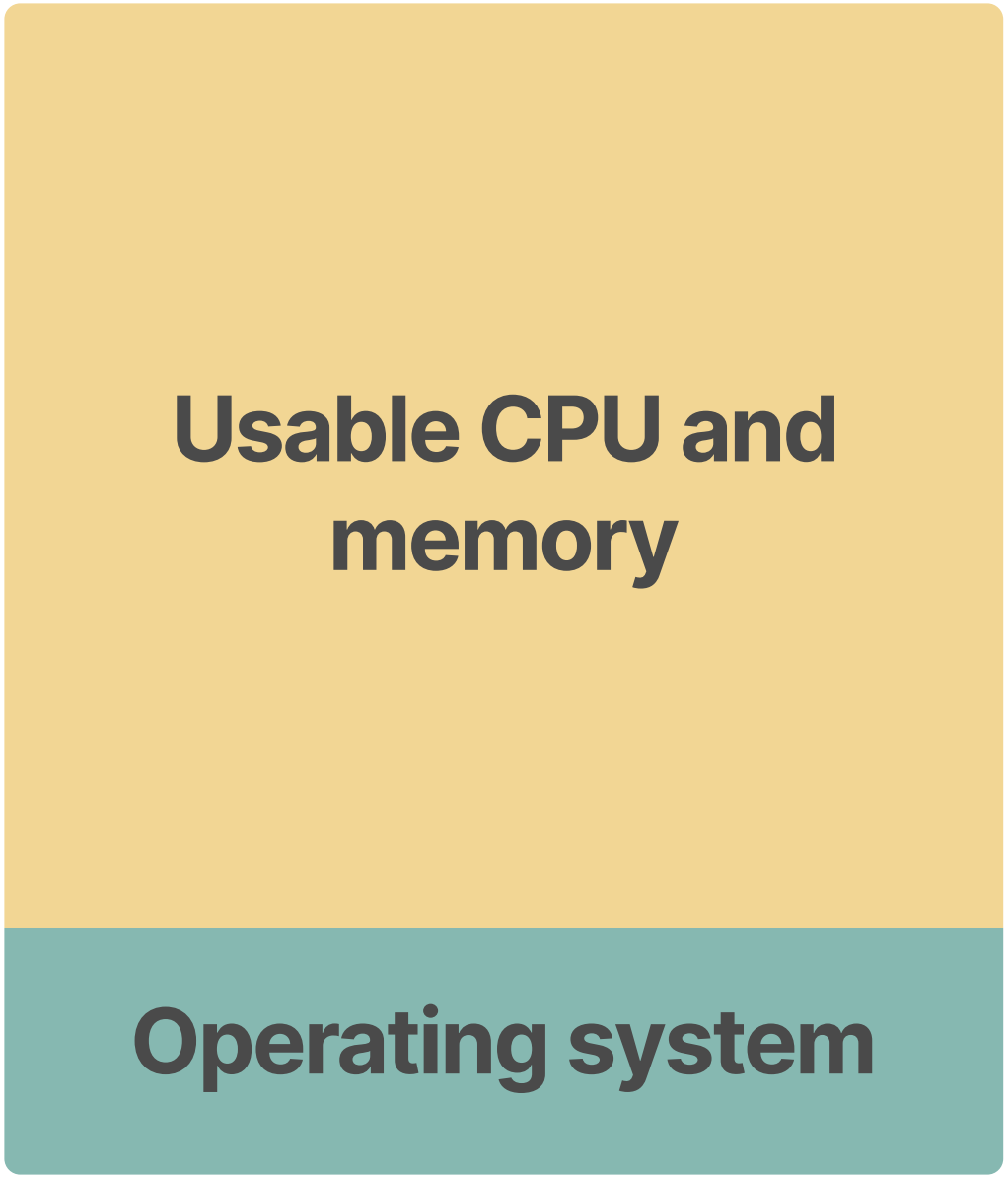

VIRTUAL
MACHINE

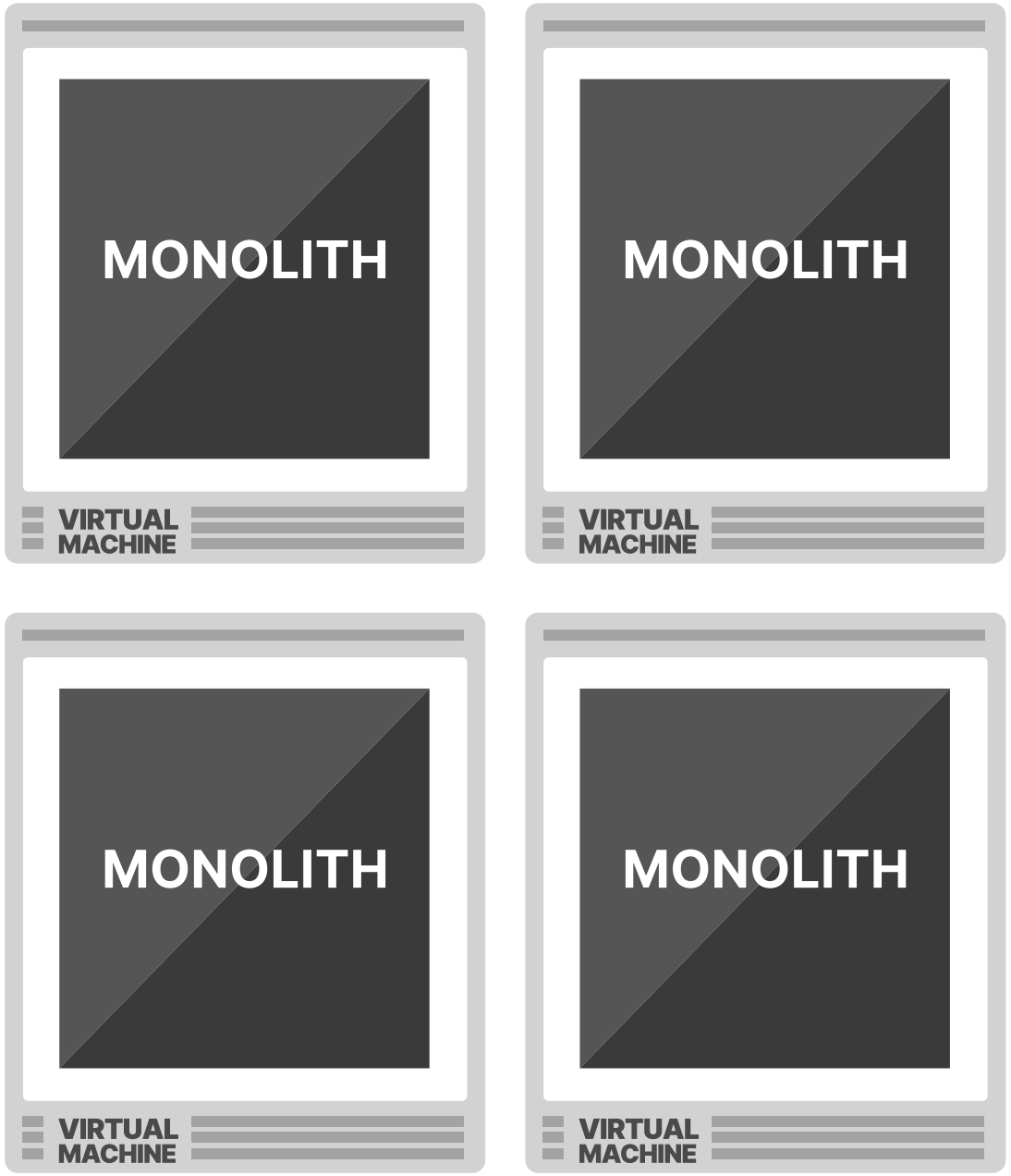
Operating system and virtual machine overhead

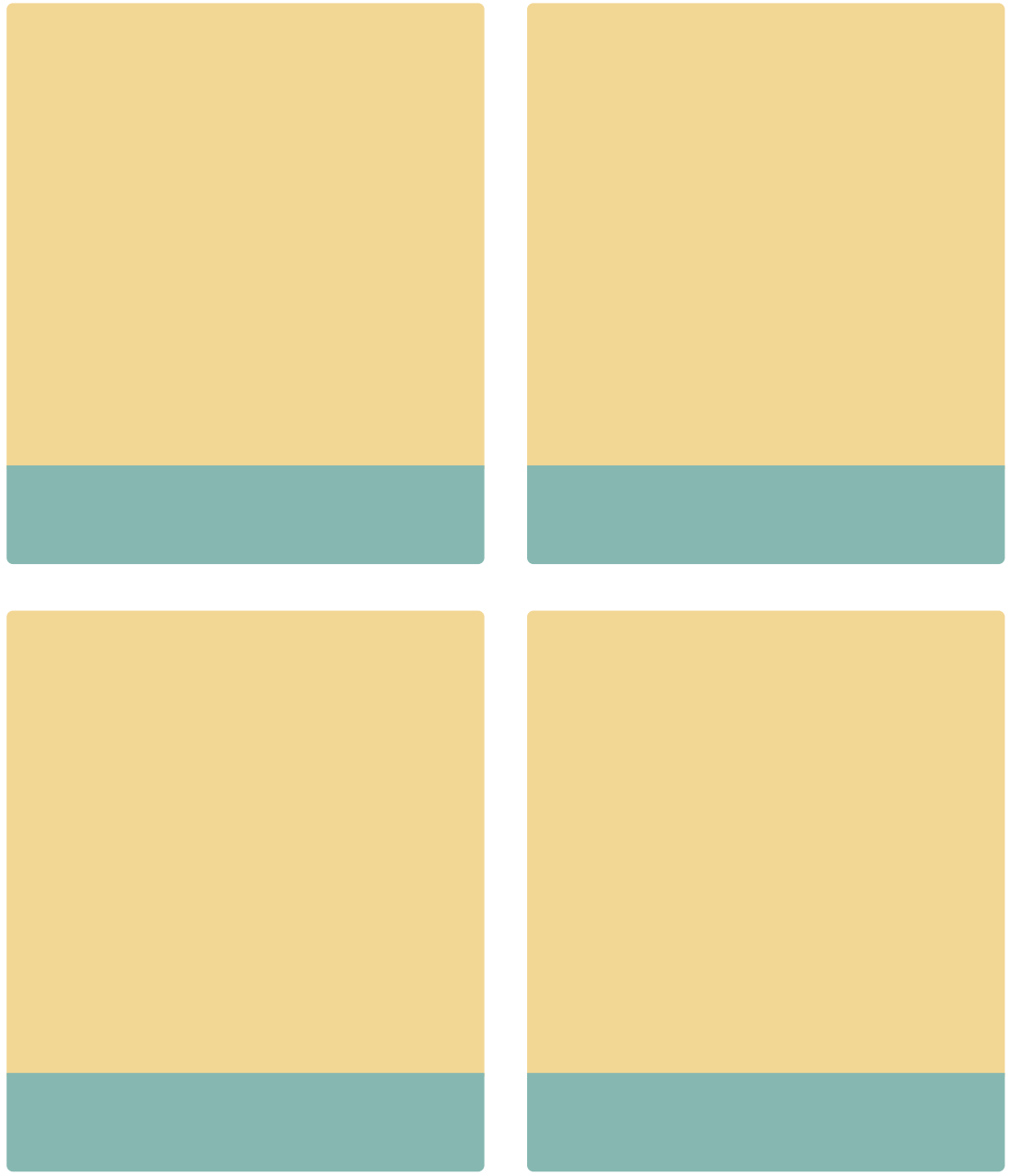
Challenge 1

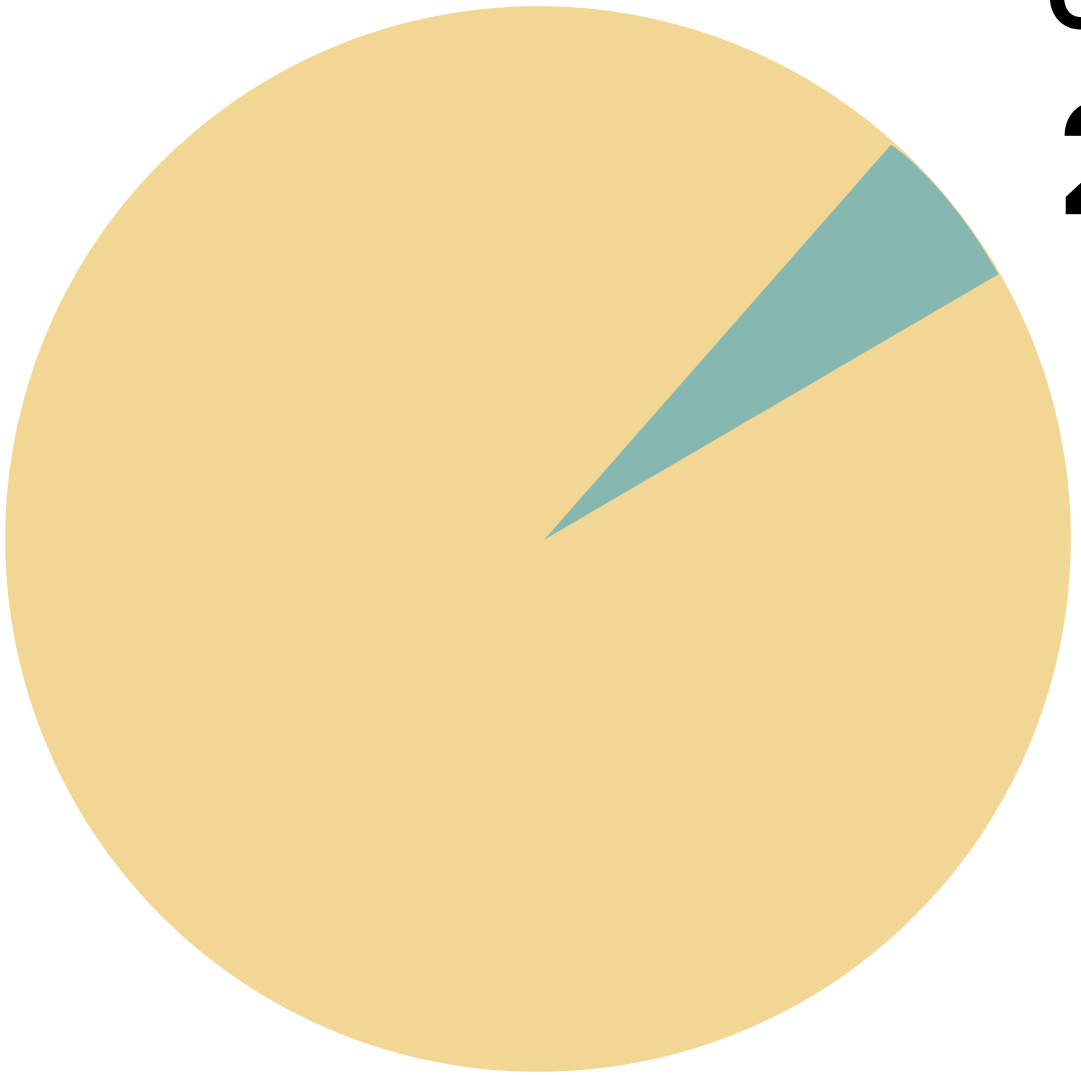

MONOLITH
VIRTUAL
MACHINE


Operating system


Usable CPU and memory
Operating system

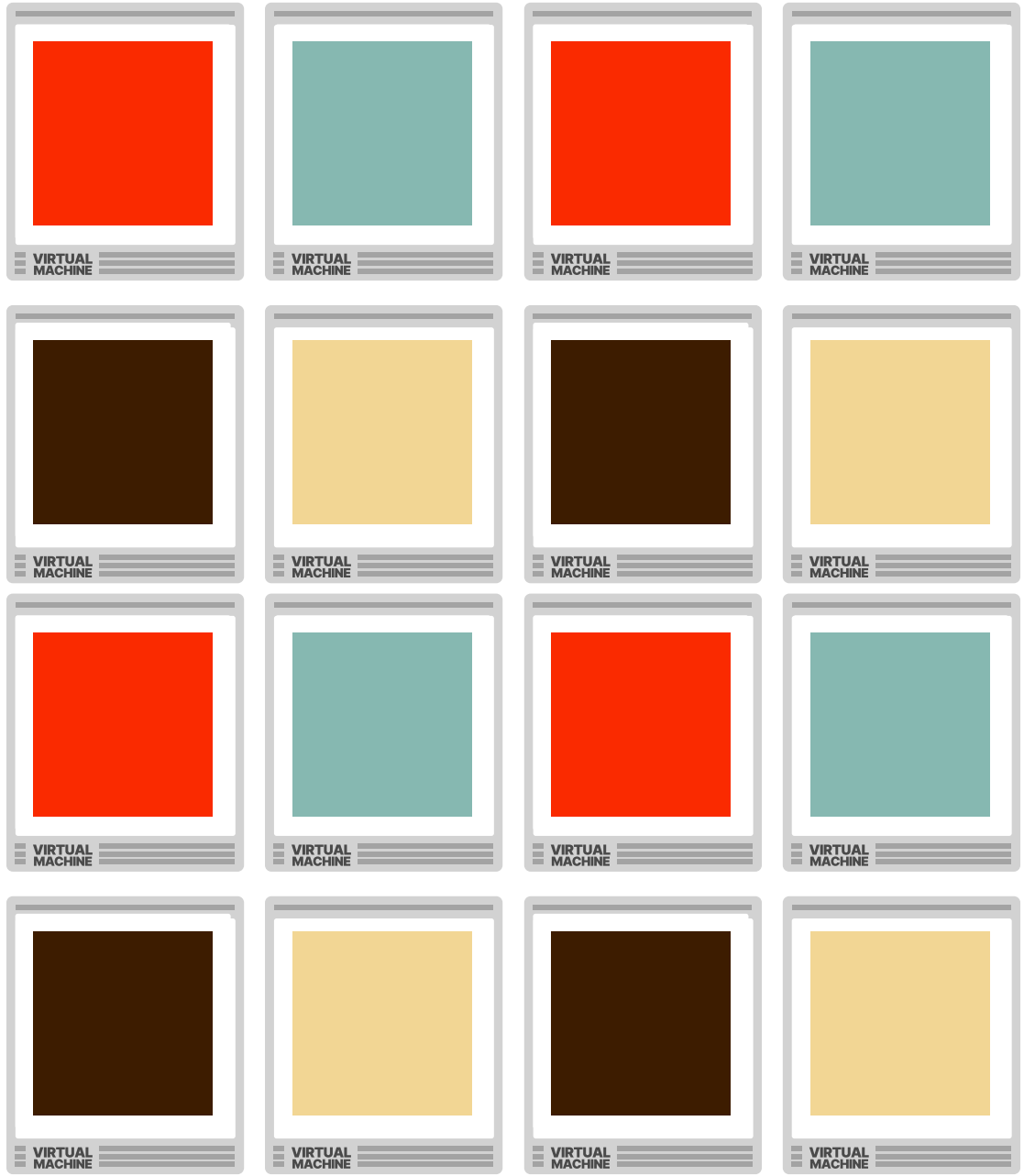

MONOLITH
VIRTUAL MACHINE
MONOLITH
VIRTUAL MACHINE
MONOLITH
VIRTUAL MACHINE
MONOLITH
VIRTUAL MACHINE

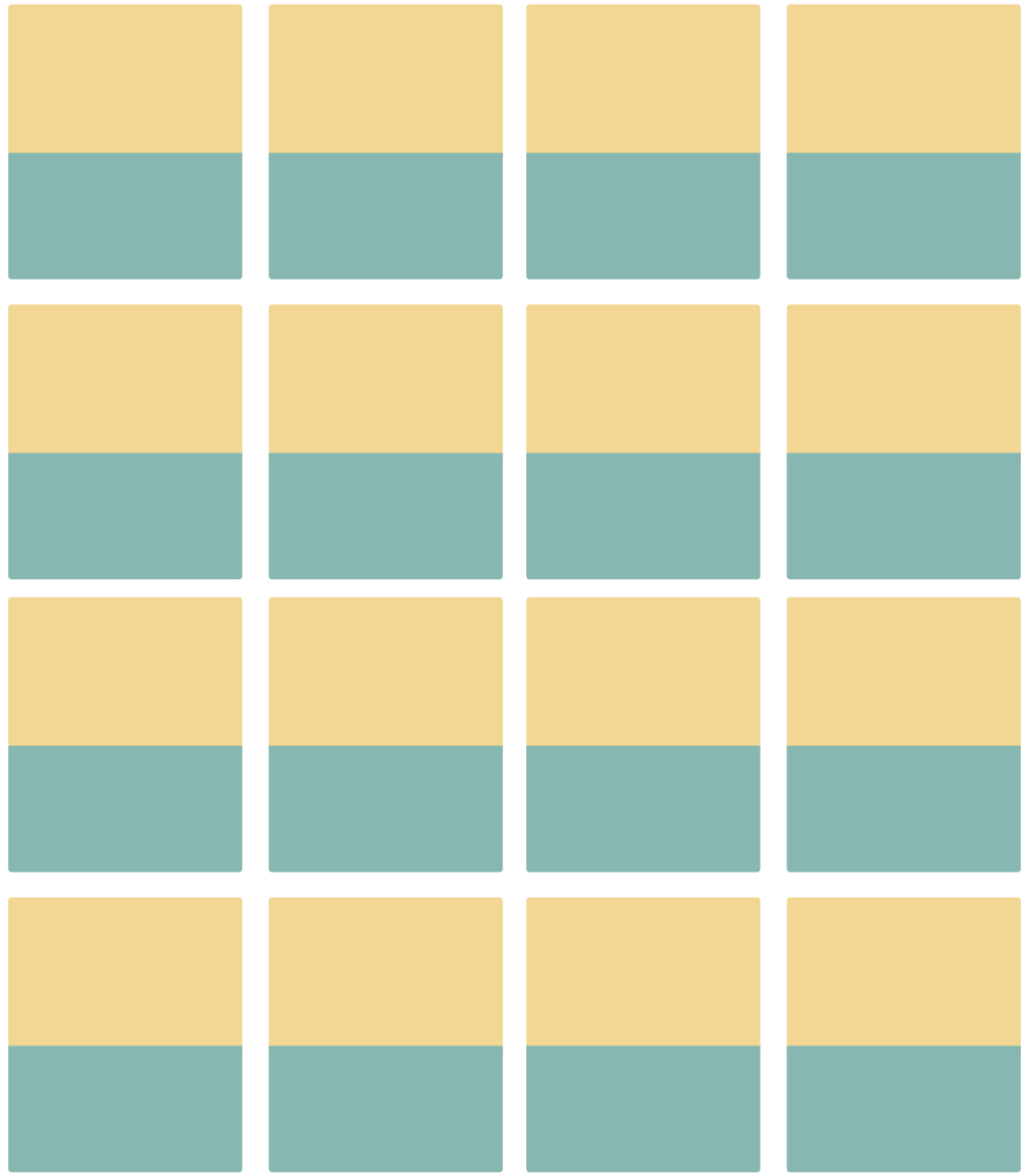


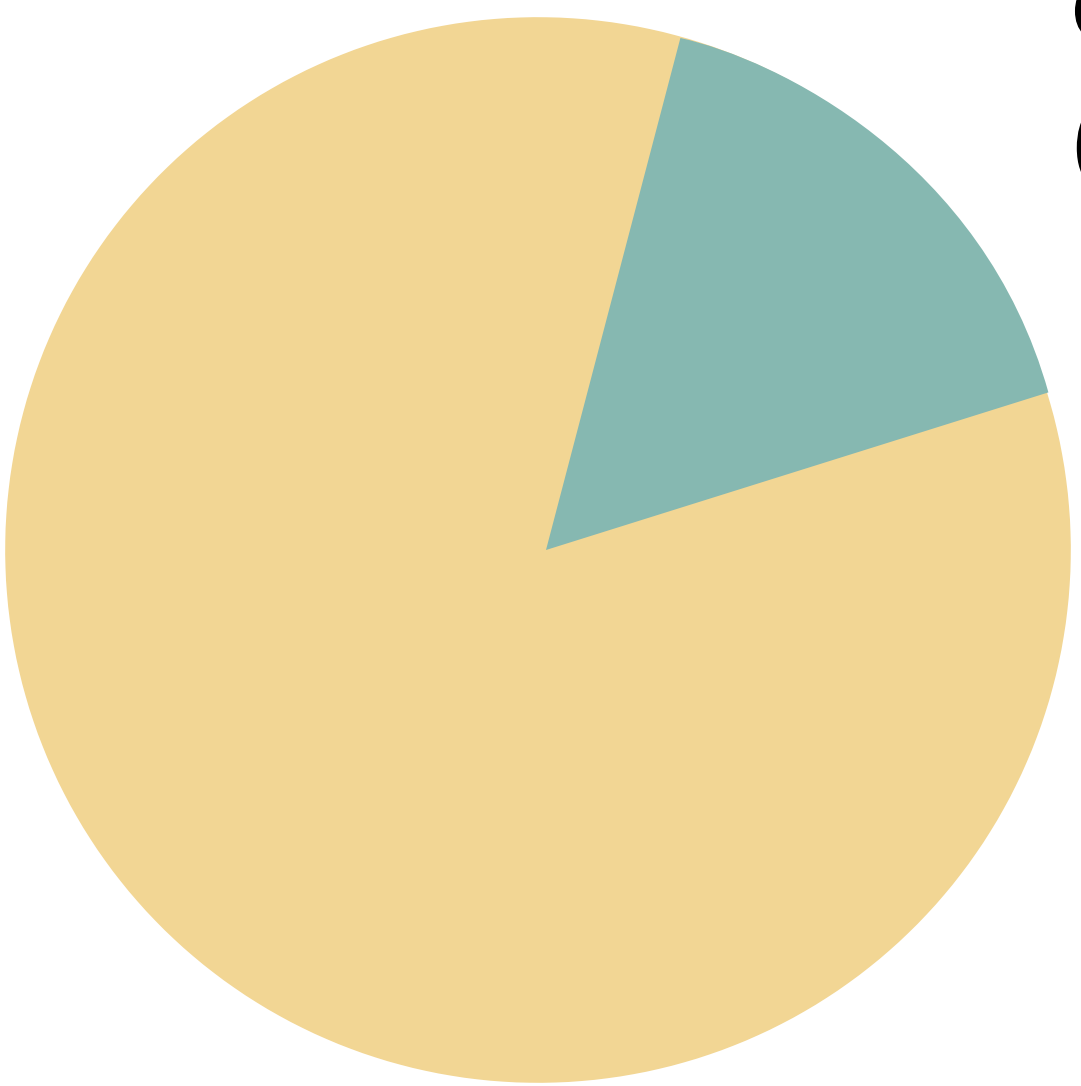


Overhead

2-3%


VIRTUAL
MACHINE



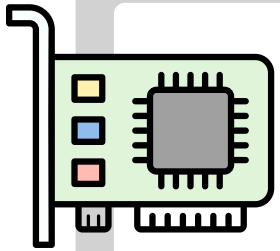


Overhead

6-10%

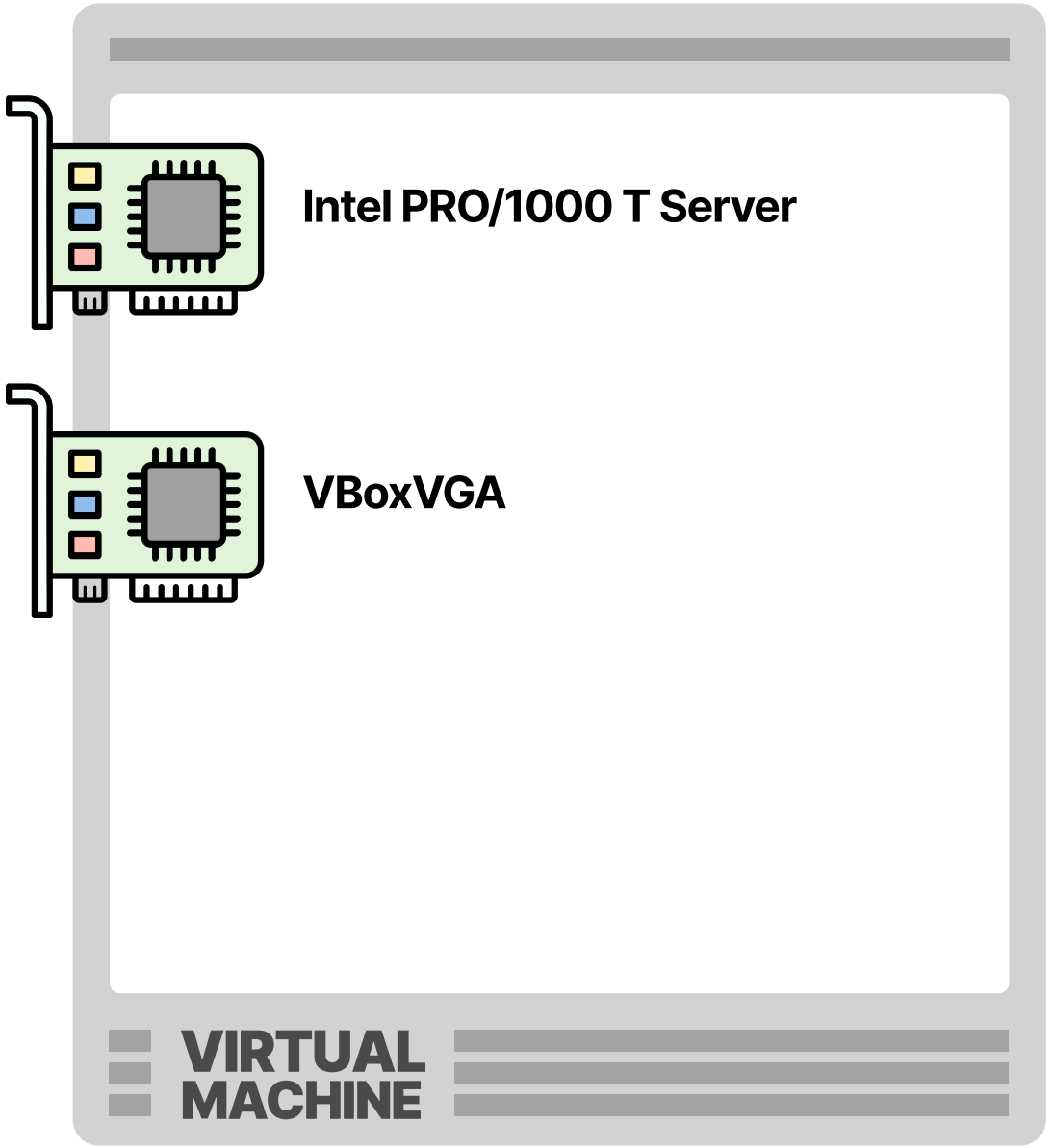
Fixed allocation and emulation

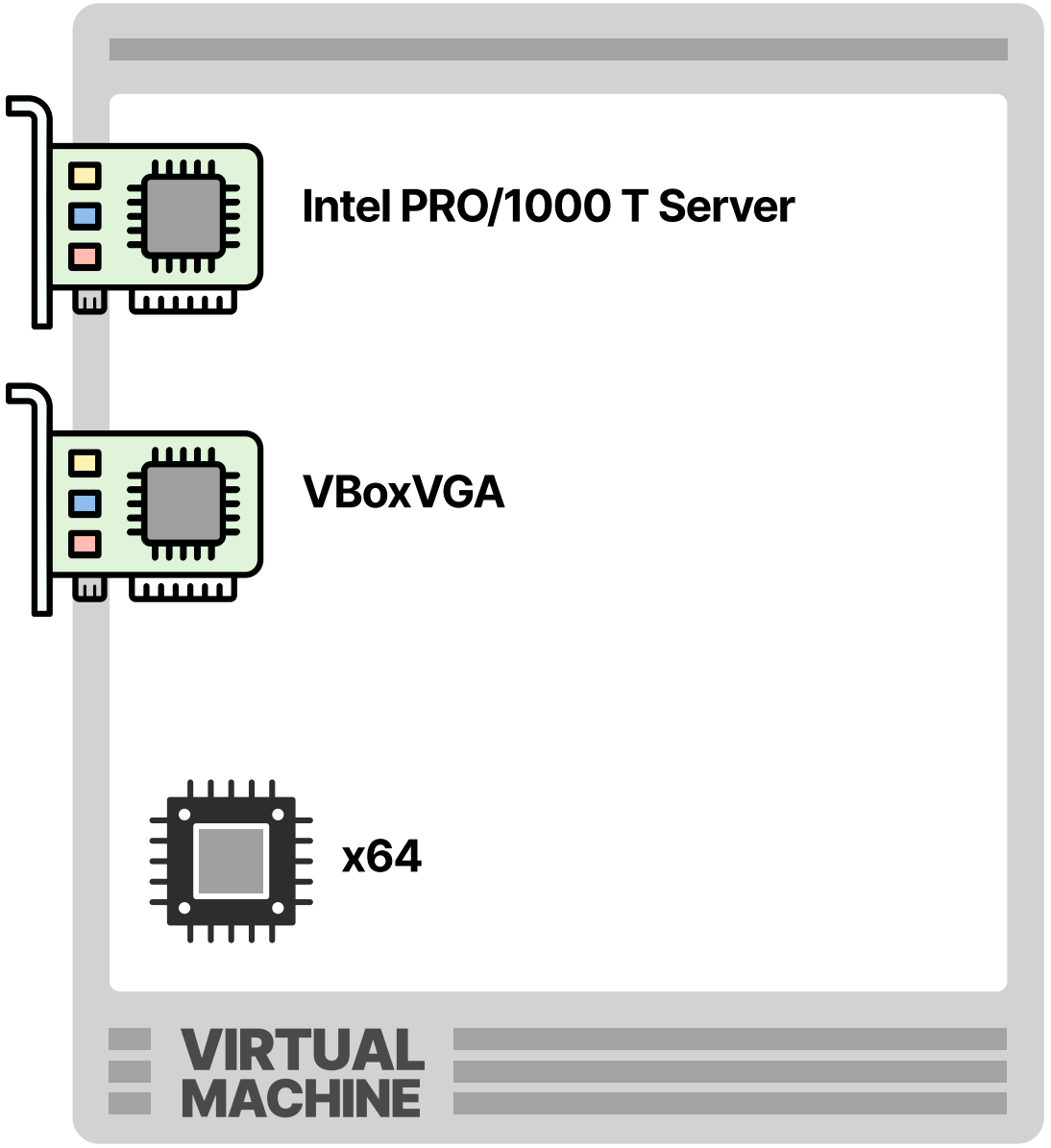
Resources allocation and utilisation



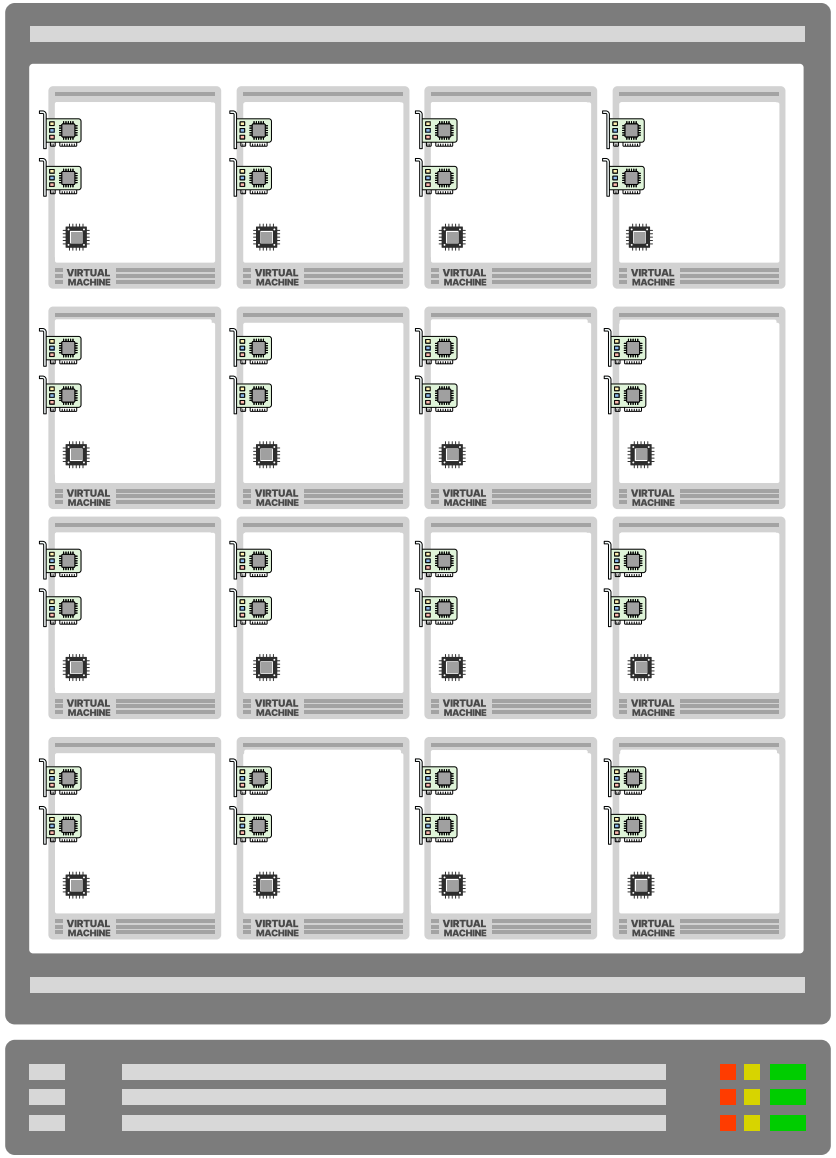
Intel PRO/1000 T Server

VIRTUAL MACHINE


Intel PRO/1000 T Server
VBoxVGA
VIRTUAL MACHINE

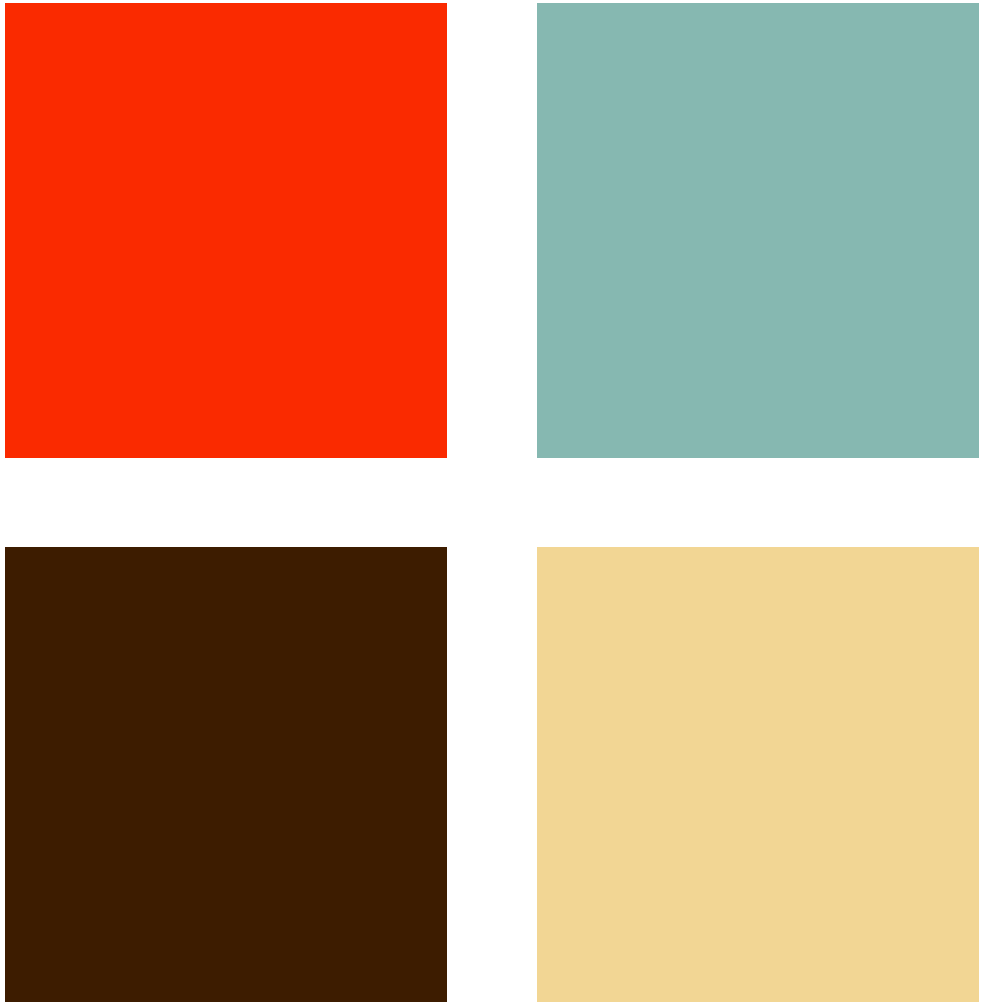

Intel PRO/1000 T Server
VBoxVGA
x64
VIRTUAL
MACHINE

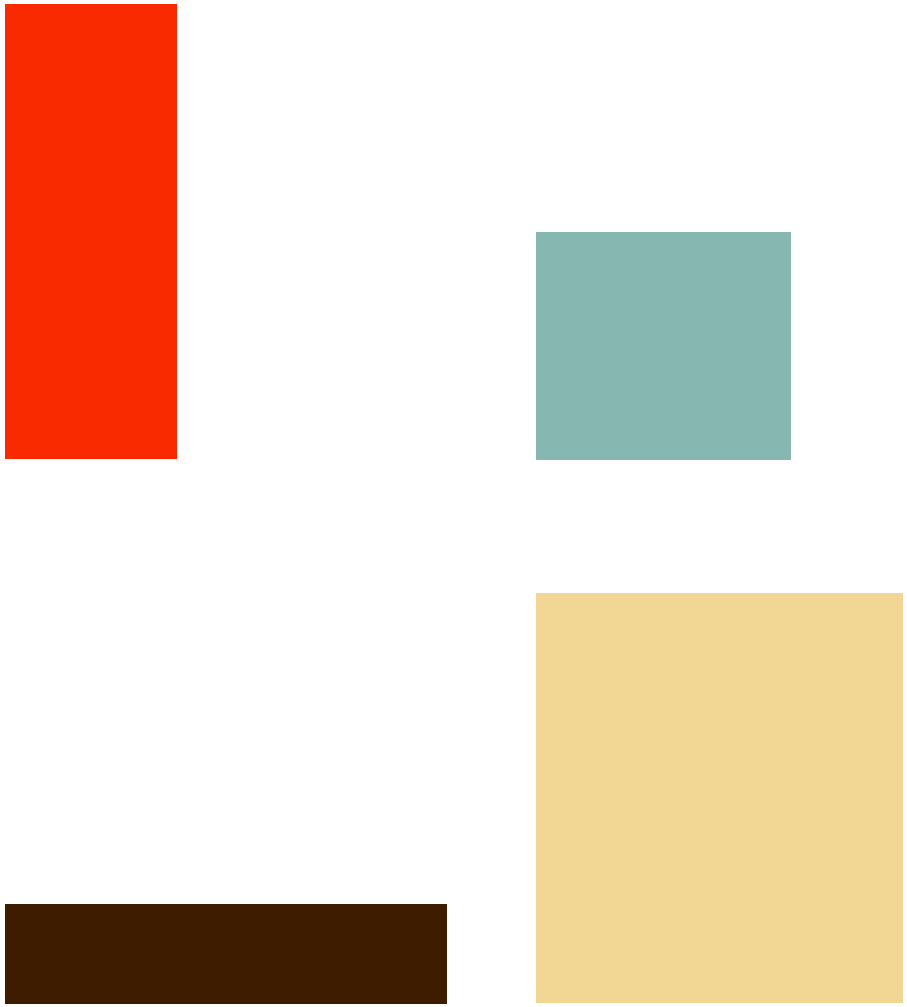

Intel PRO/1000 T Server
VBoxVGA
x64
VIRTUAL MACHINE
Intel PRO/1000 T Server
VBoxVGA
x64
VIRTUAL MACHINE
Intel PRO/1000 T Server
VBoxVGA
x64
VIRTUAL MACHINE
Intel PRO/1000 T Server
VBoxVGA
x64
VIRTUAL MACHINE

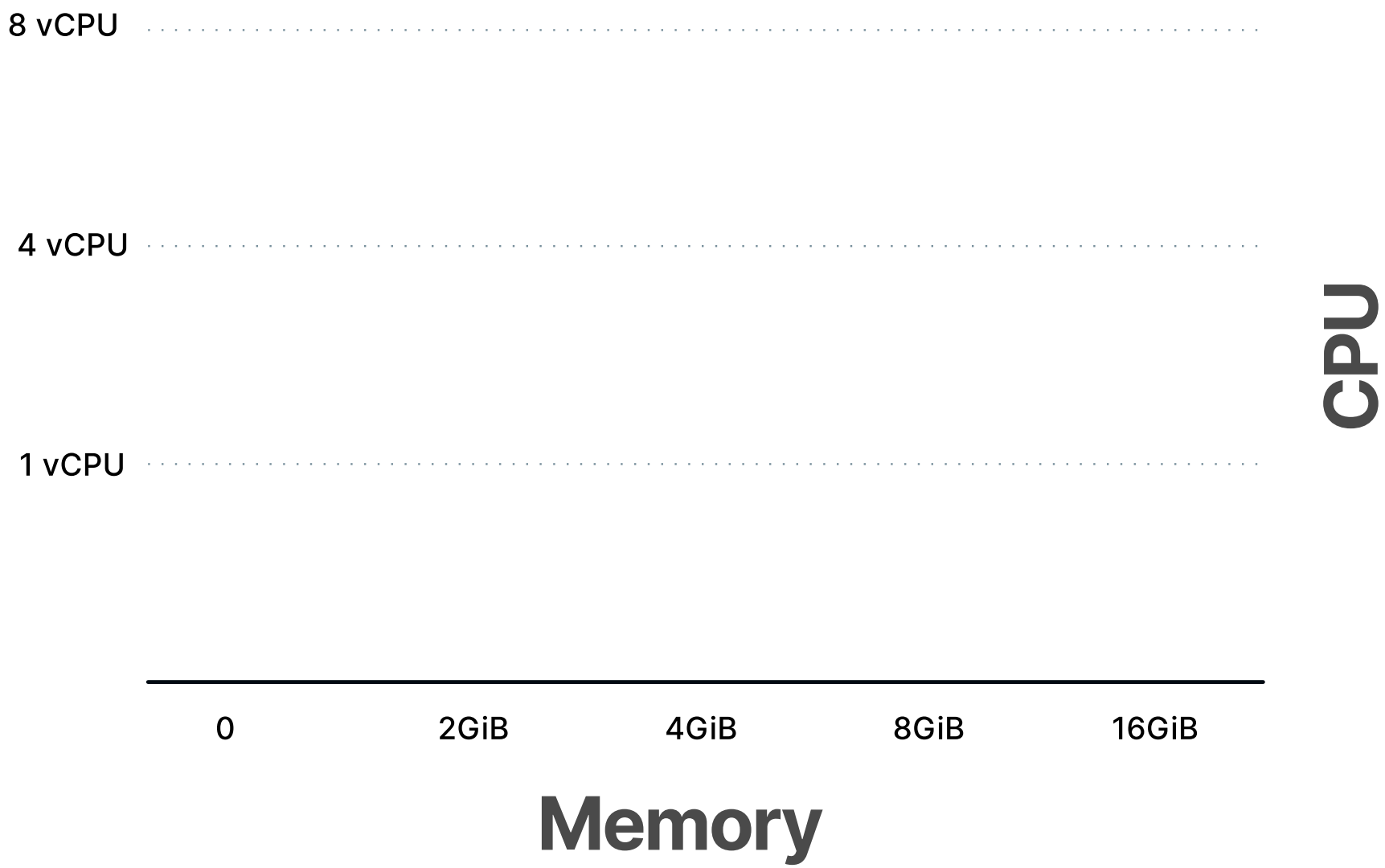

VIRTUAL MACHINE

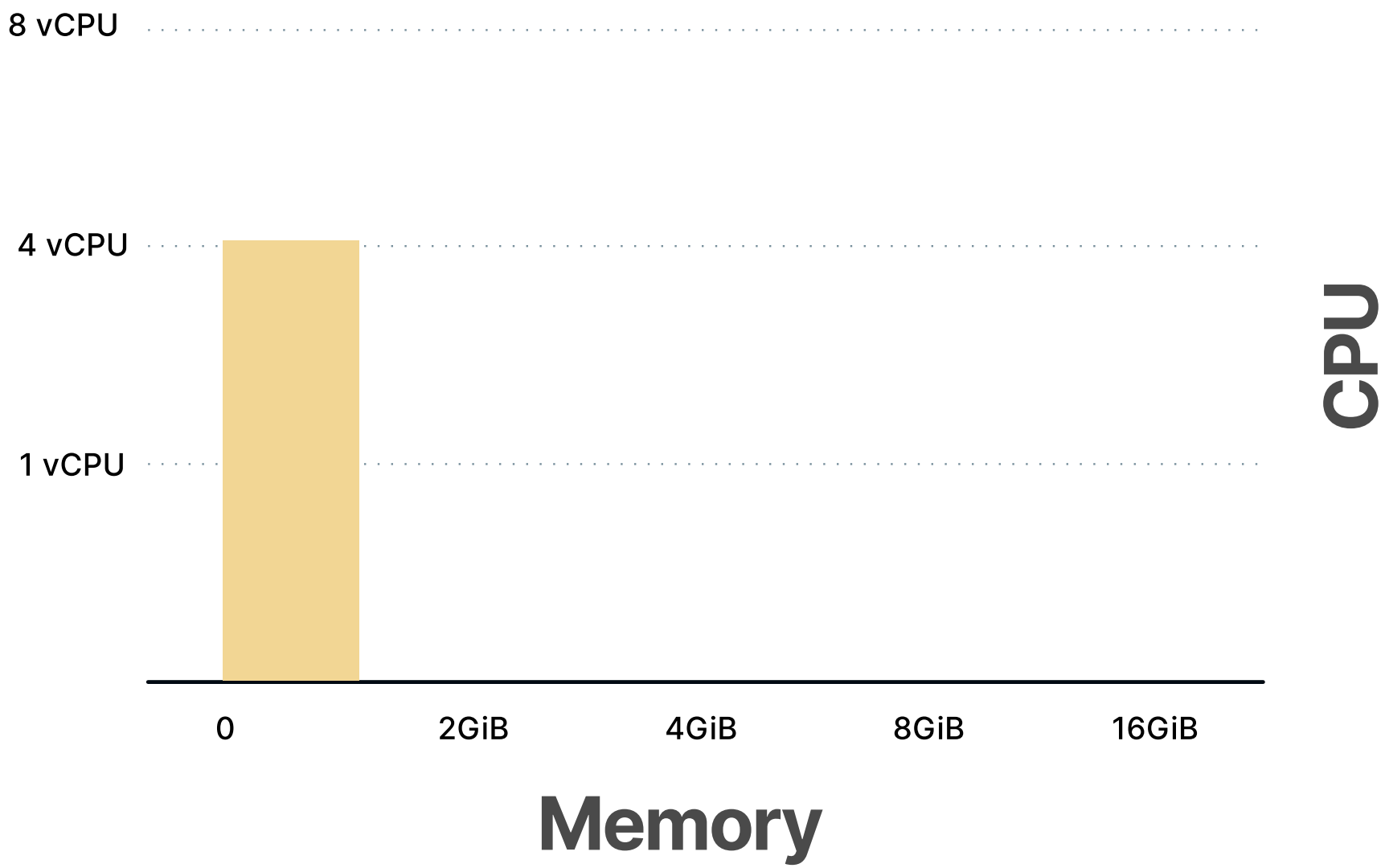
Resources allocation and utilisation

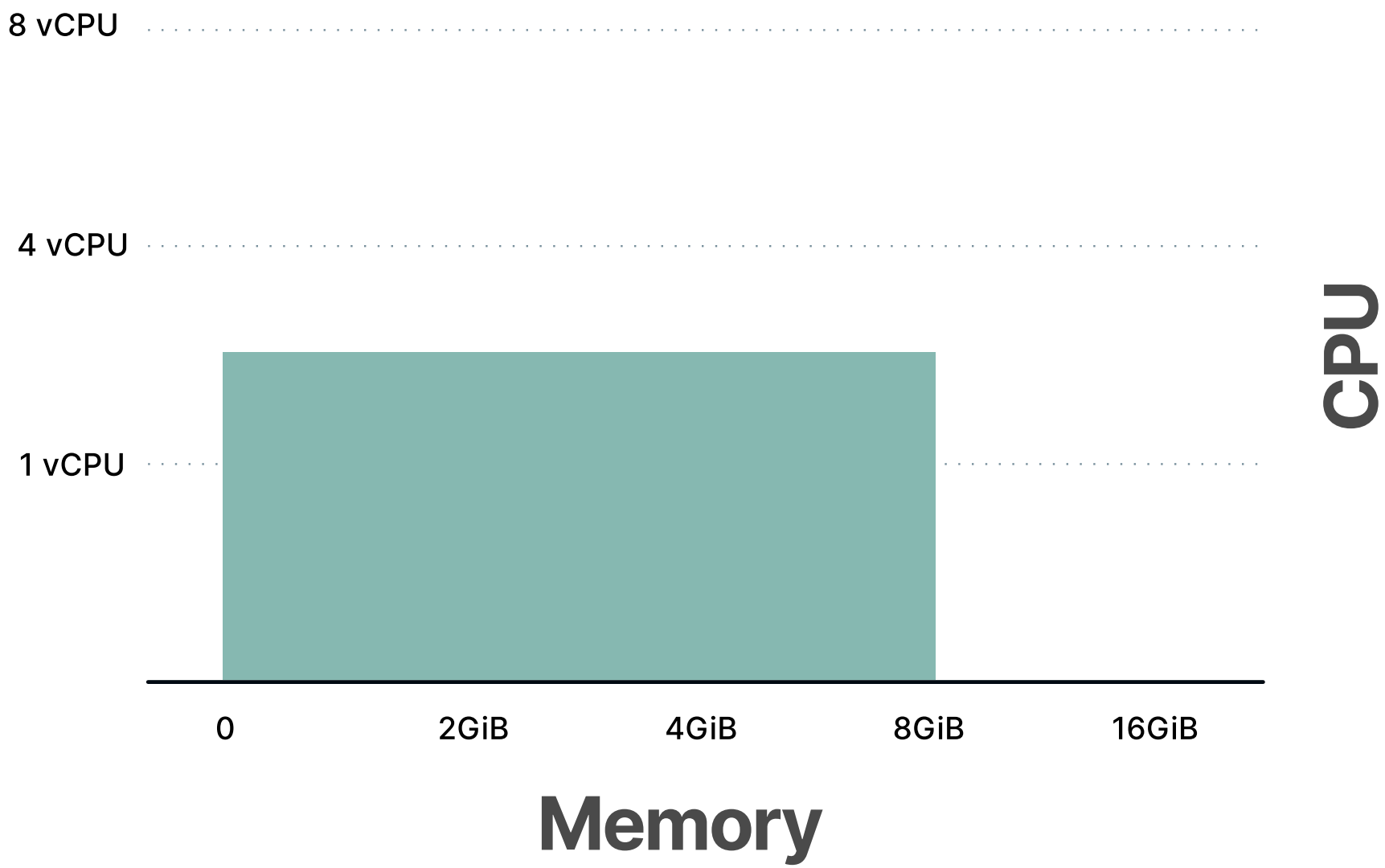
Challenge 2

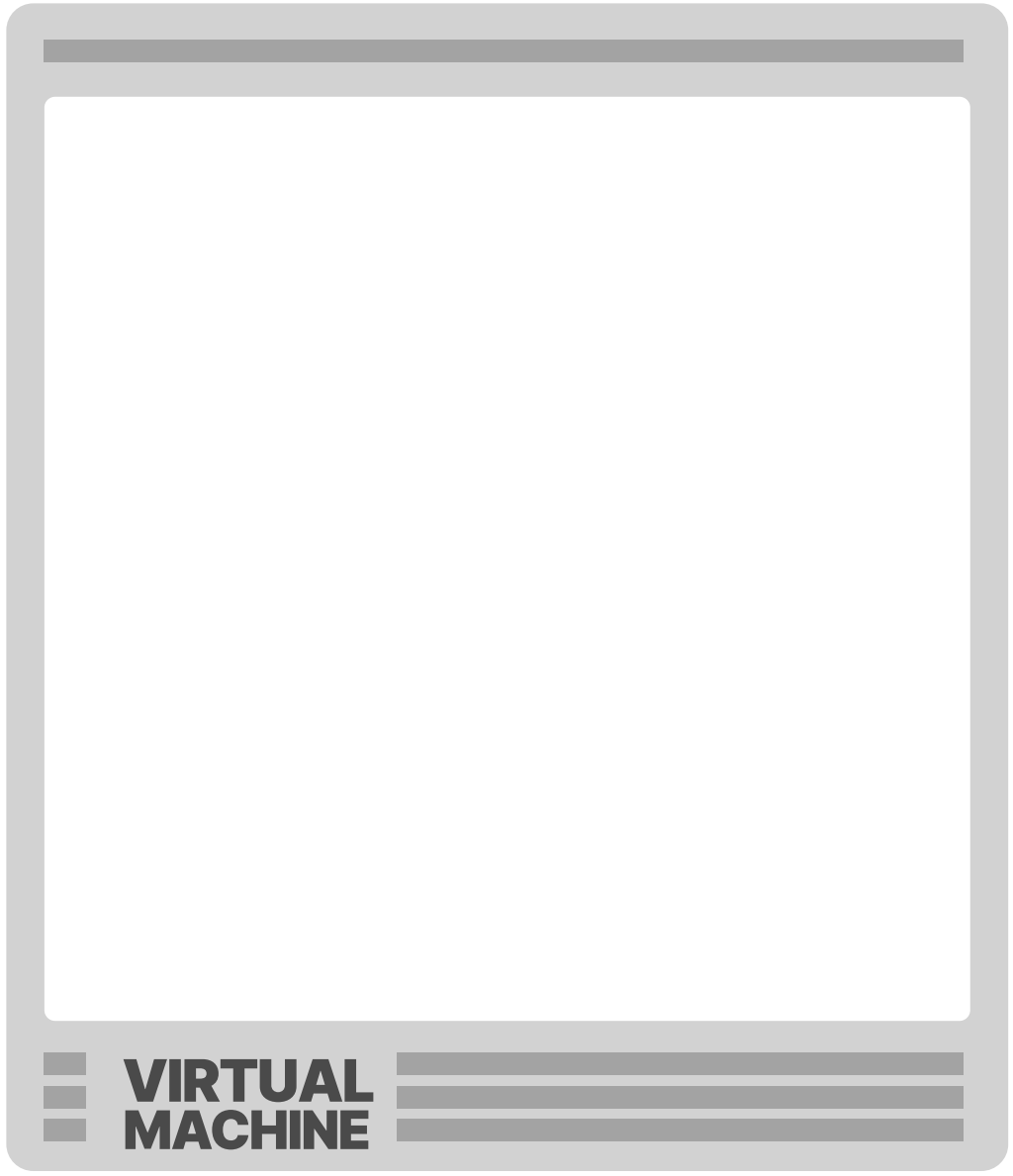





8 vCPU
4 vCPU
1 vCPU
CPU
0
2GiB
4GiB
8GiB
16GiB
Memory

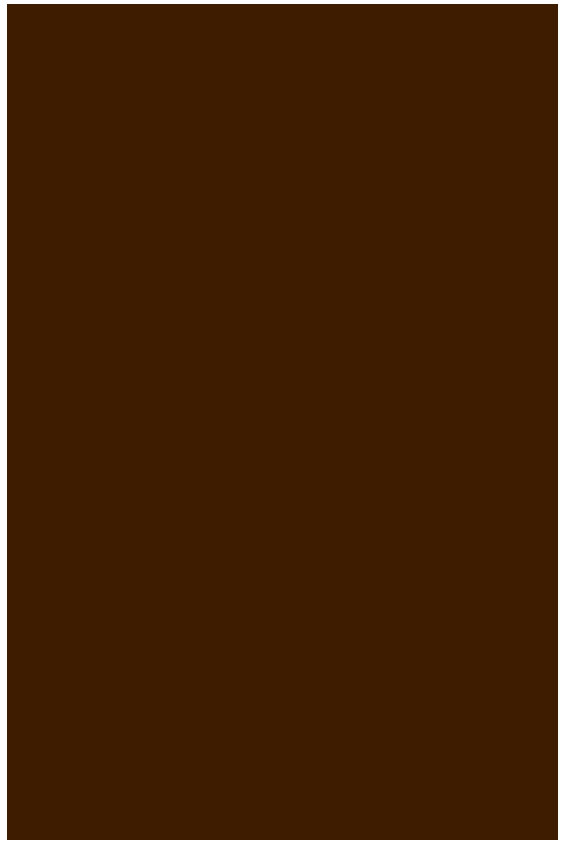

8 vCPU
4 vCPU
1 vCPU
CPU
0
2GiB
4GiB
8GiB
16GiB
Memory


8 vCPU
4 vCPU
1 vCPU
CPU
0
2GiB
4GiB
8GiB
16GiB
Memory


VIRTUAL
MACHINE

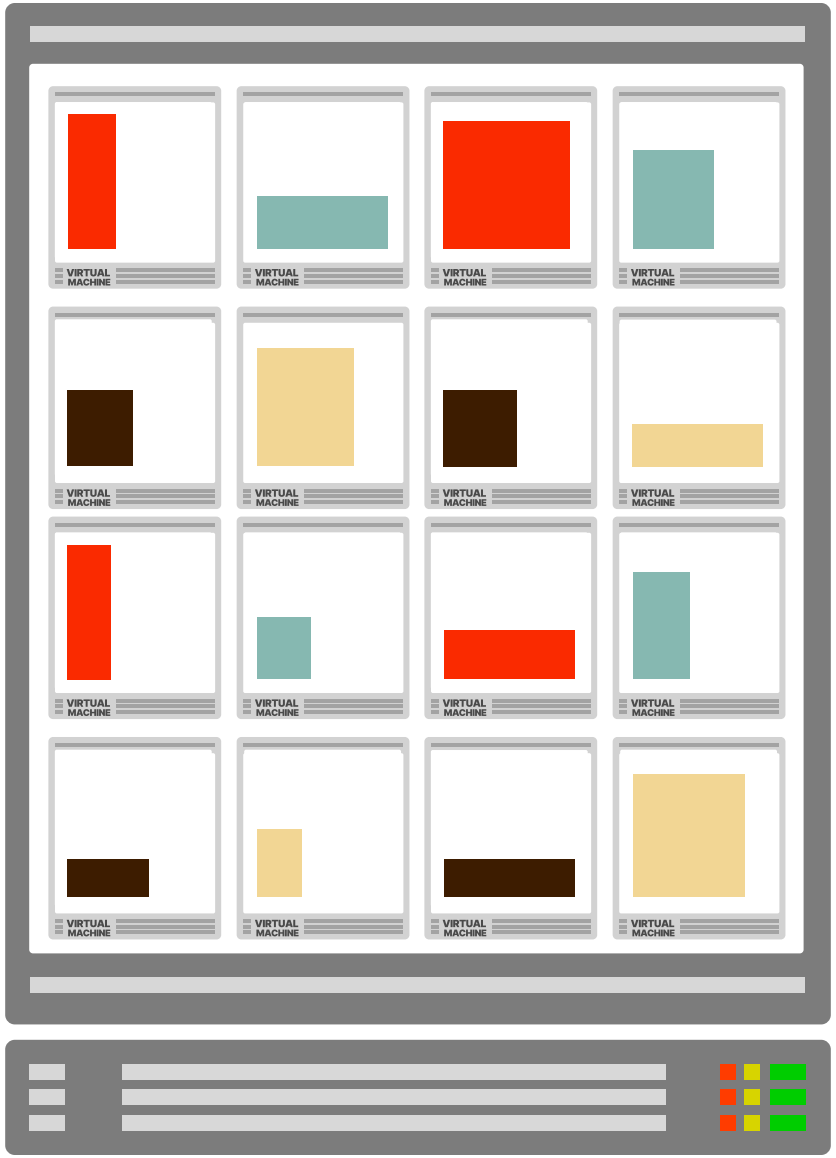

VIRTUAL
MACHINE

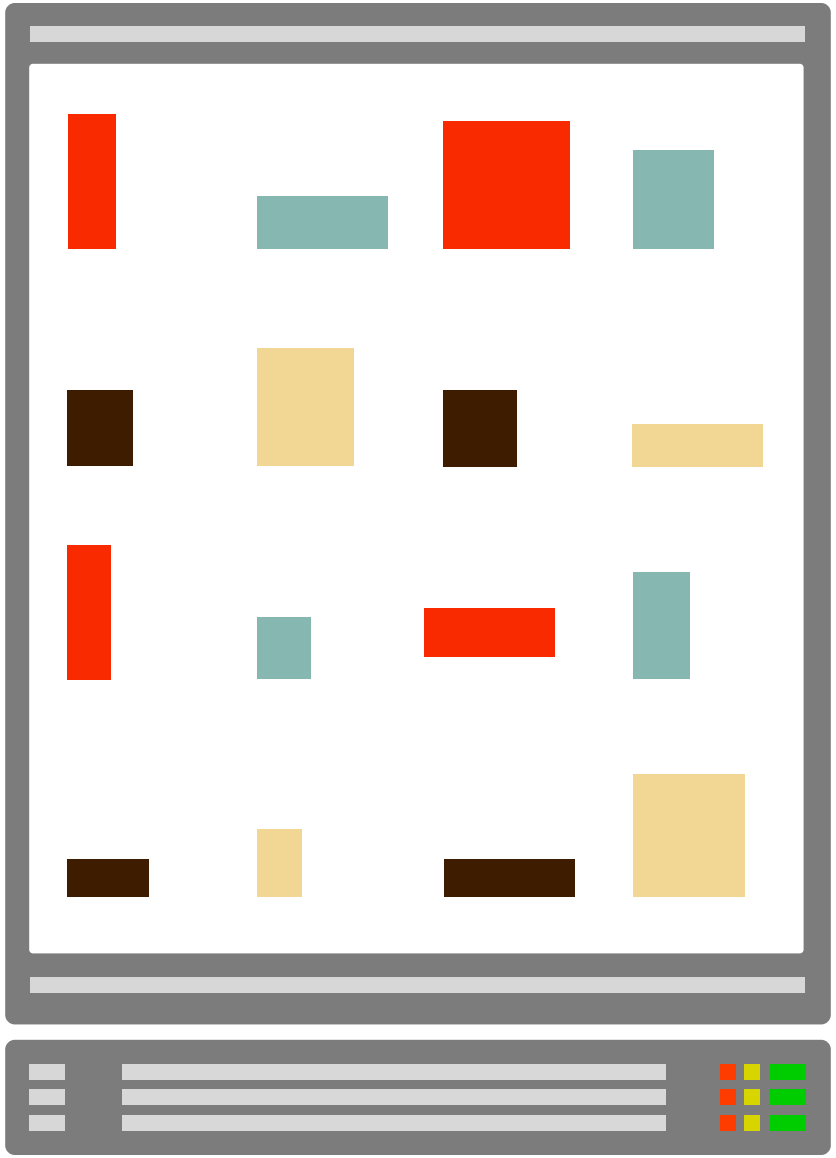

Underutilized
VIRTUAL
MACHINE

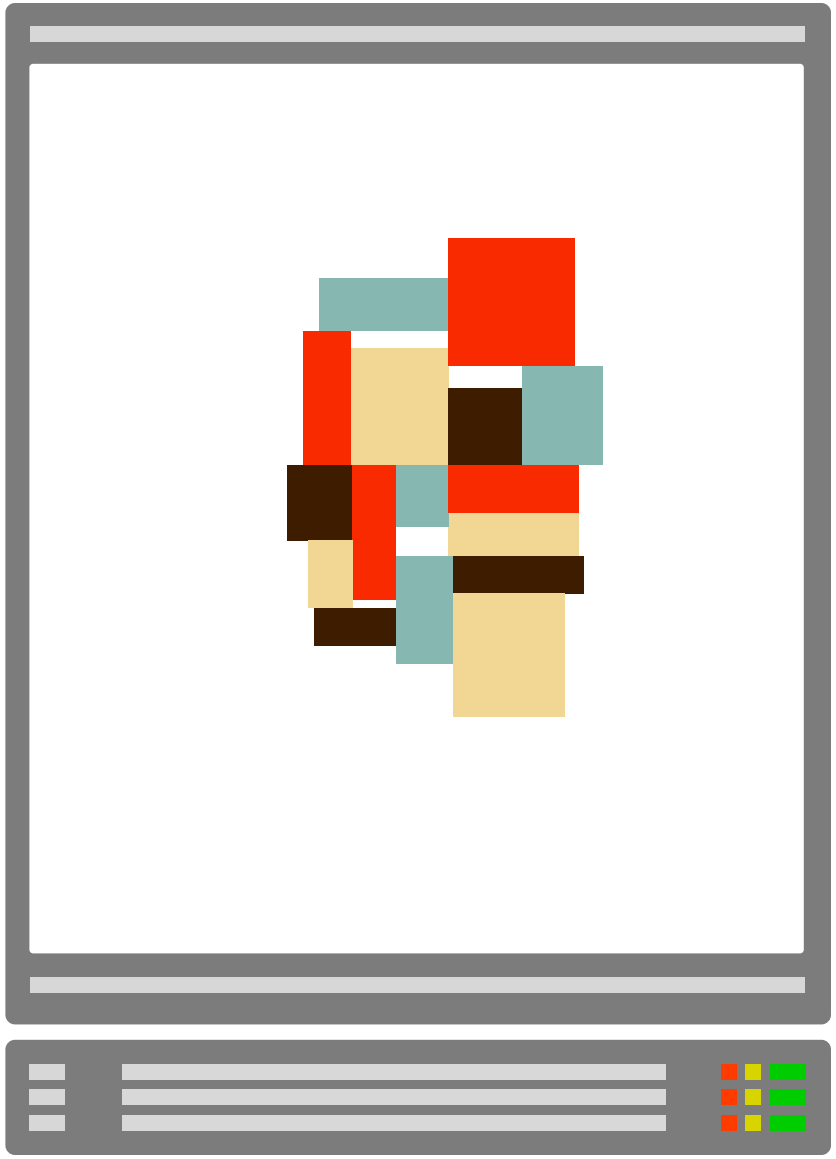


VIRTUAL
MACHINE


Underutilized
VIRTUAL
MACHINE

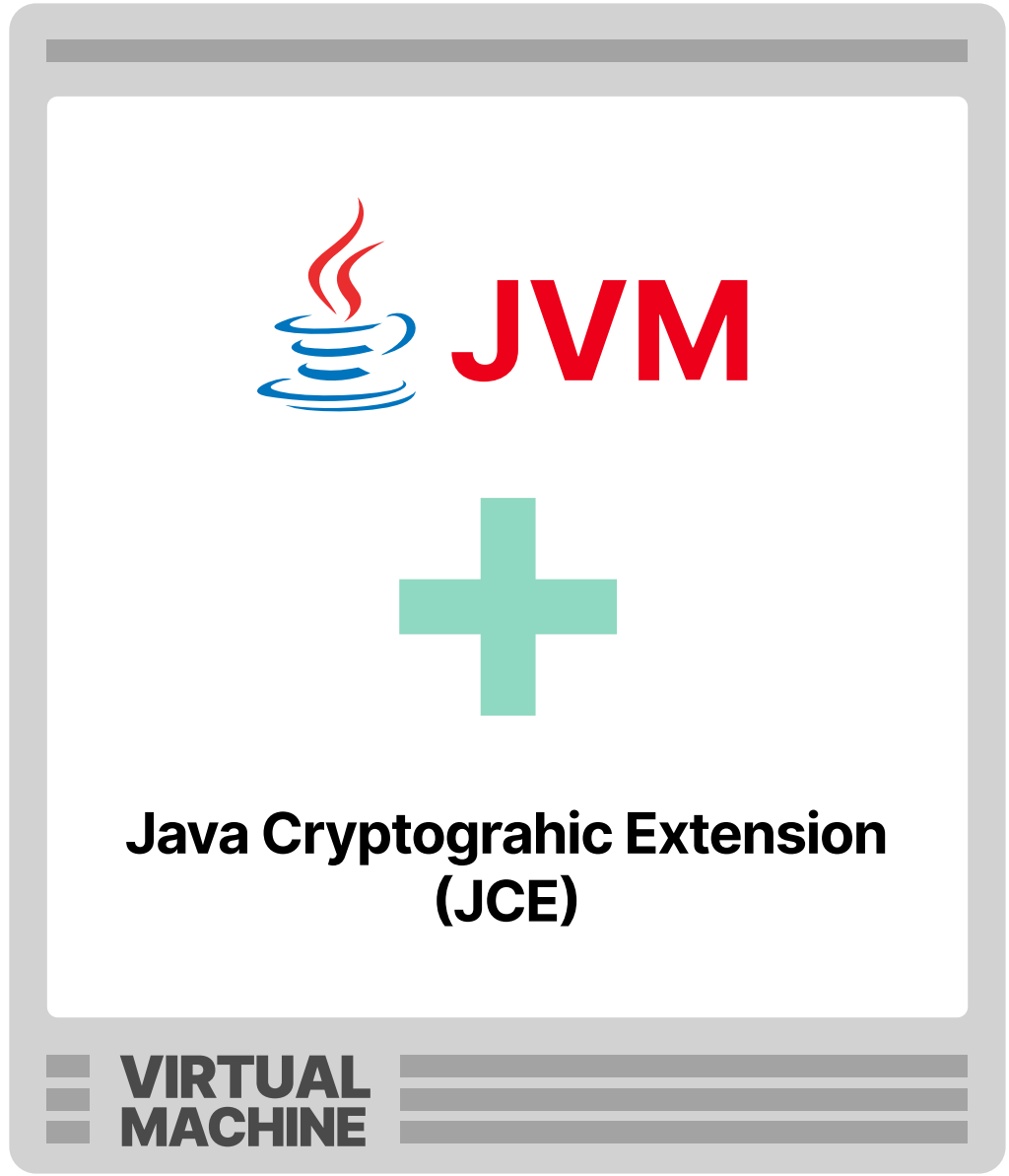

VIRTUAL MACHINE



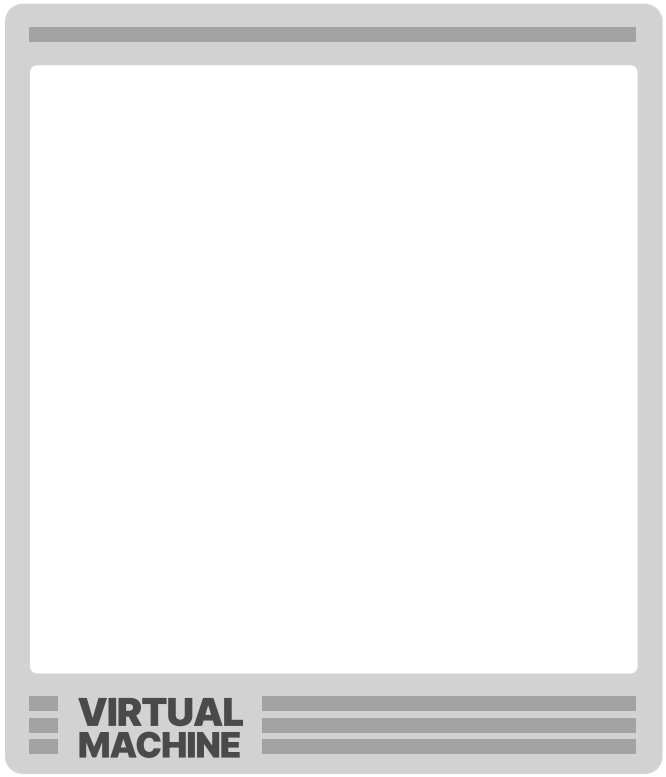


Packaging and distribution

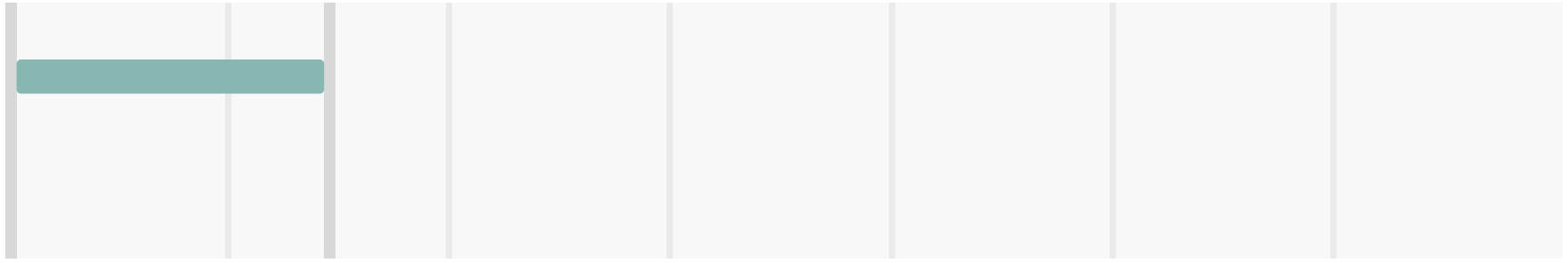
Challenge 3

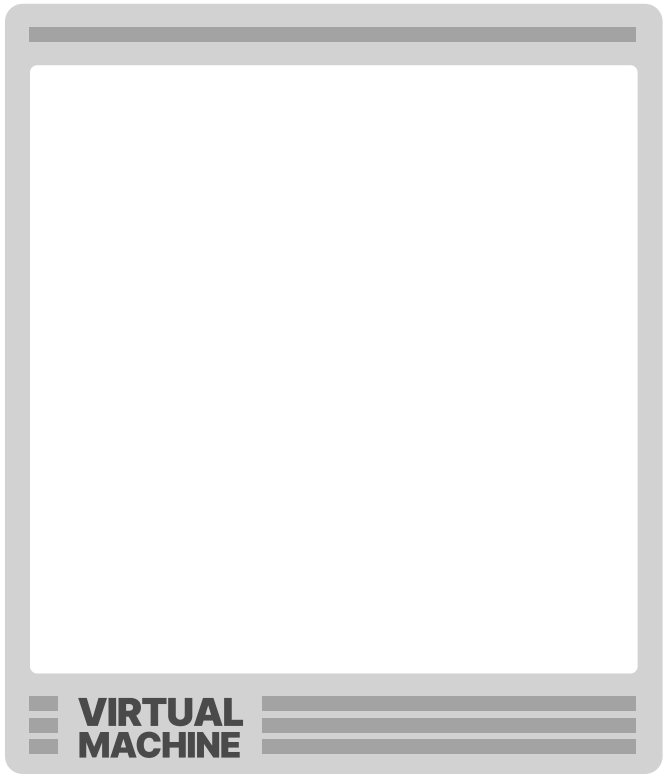

JVM
Java Cryptograhic Extension
(JCE)
VIRTUAL
MACHINE


node
JS
+
C/C++
VIRTUAL
MACHINE

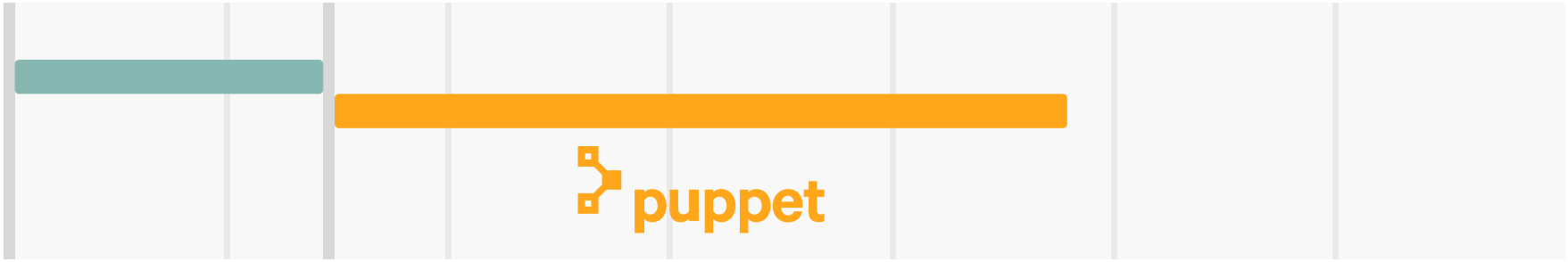


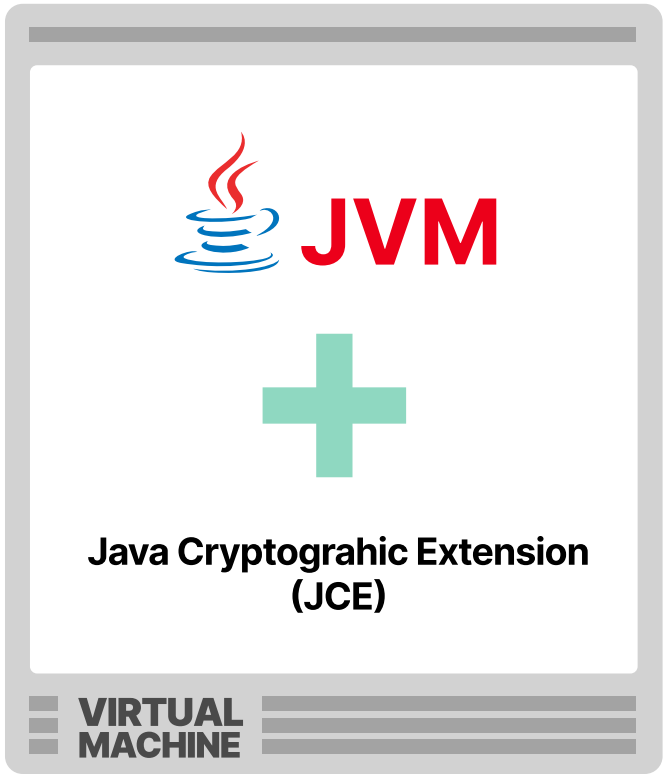
Start **Ready**

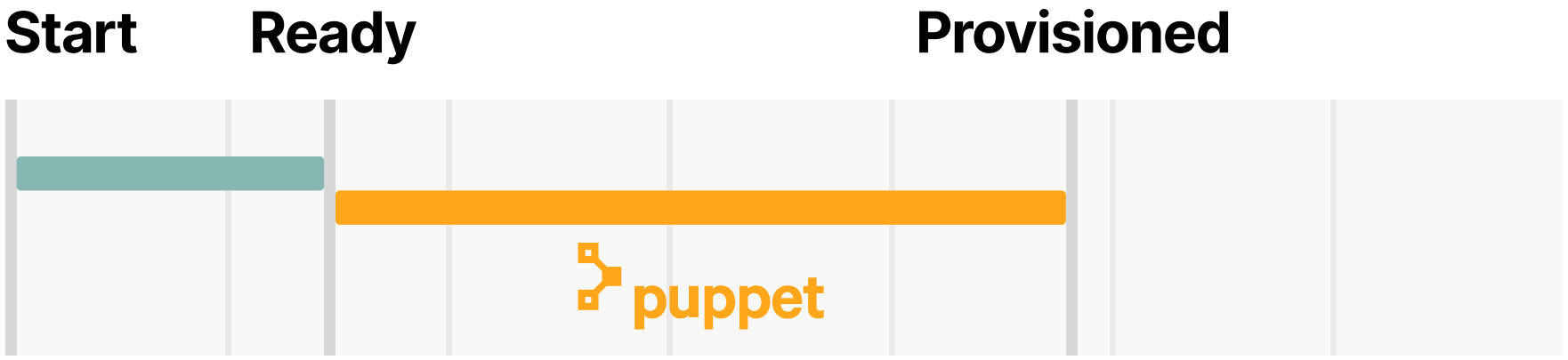


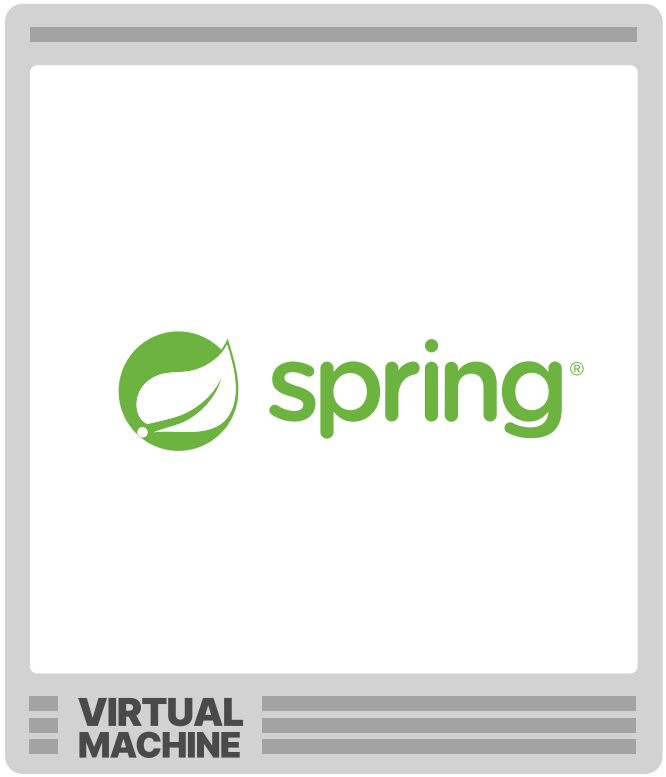


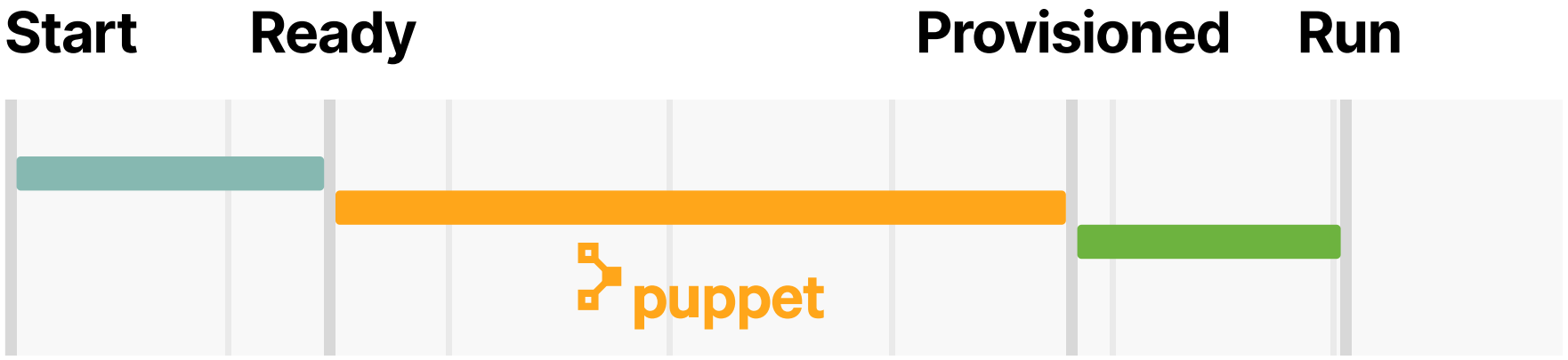
Start **Ready**




JVM
Java Cryptograhic Extension
(JCE)
VIRTUAL
MACHINE

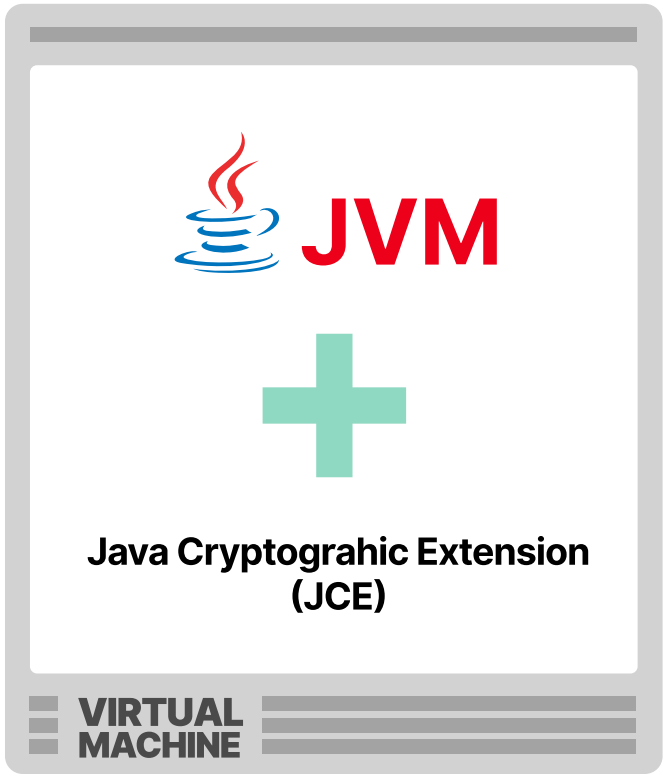

Start
Ready
Provisioned
puppet

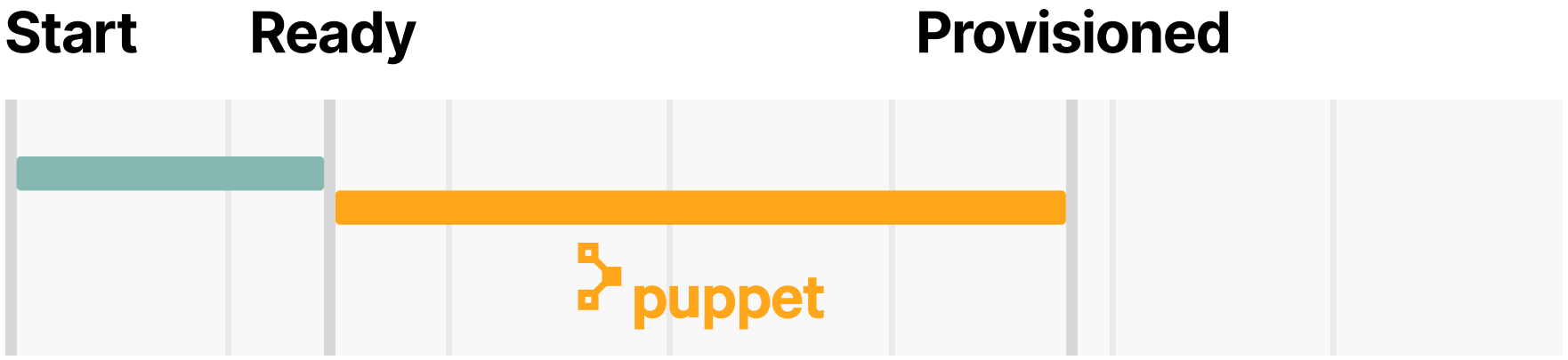

spring
VIRTUAL
MACHINE


Start
Ready
Provisioned
Run
puppet

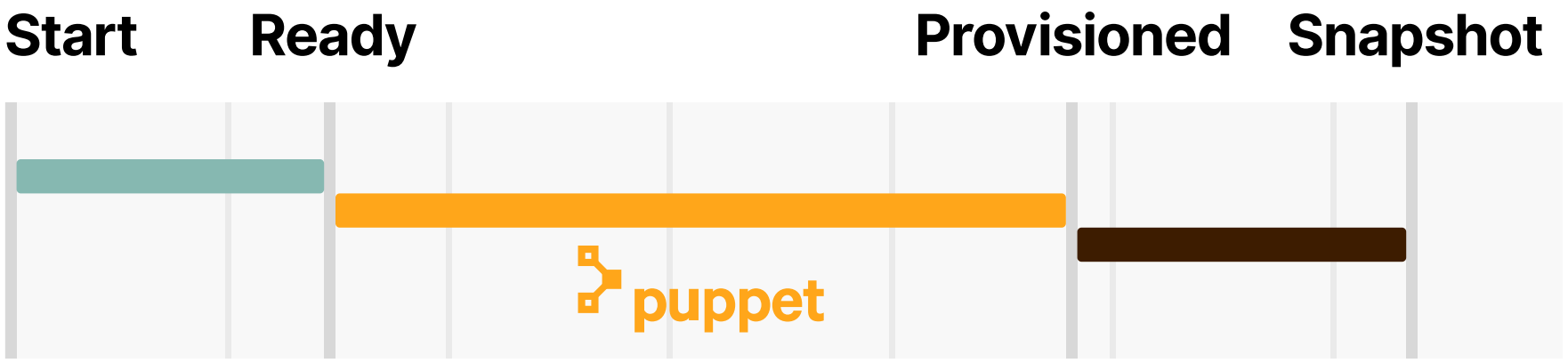
Snapshotting

Packaging and distribution


JVM
Java Cryptograhic Extension
(JCE)
VIRTUAL
MACHINE


Start
Ready
Provisioned
puppet

vm_v1.ISO

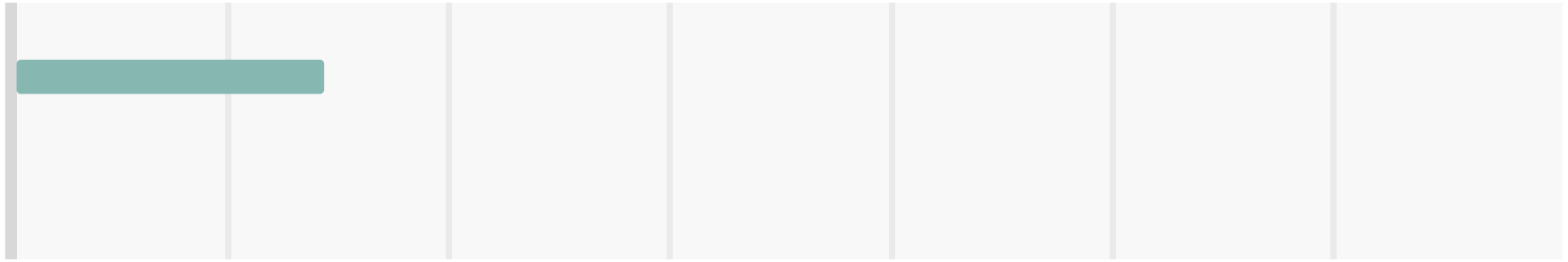


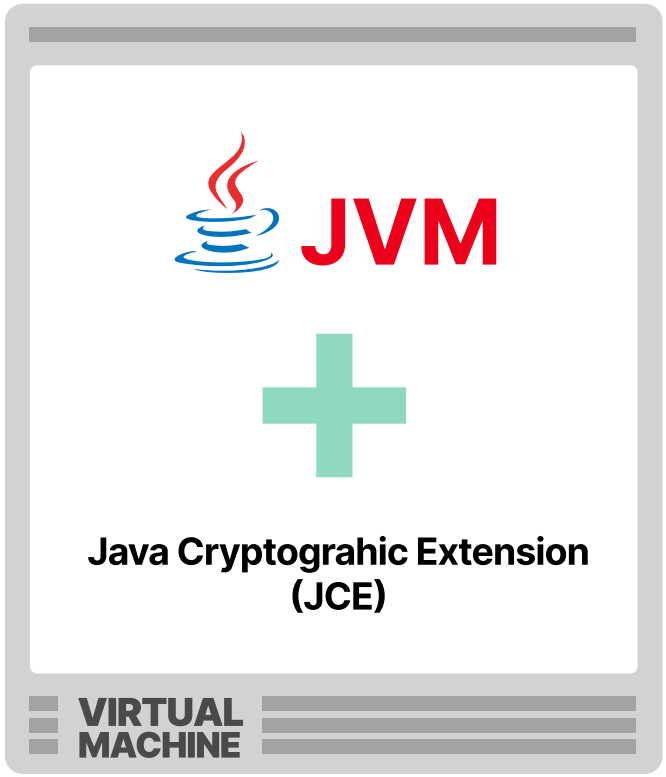
Running a snaphot

Packaging and distribution

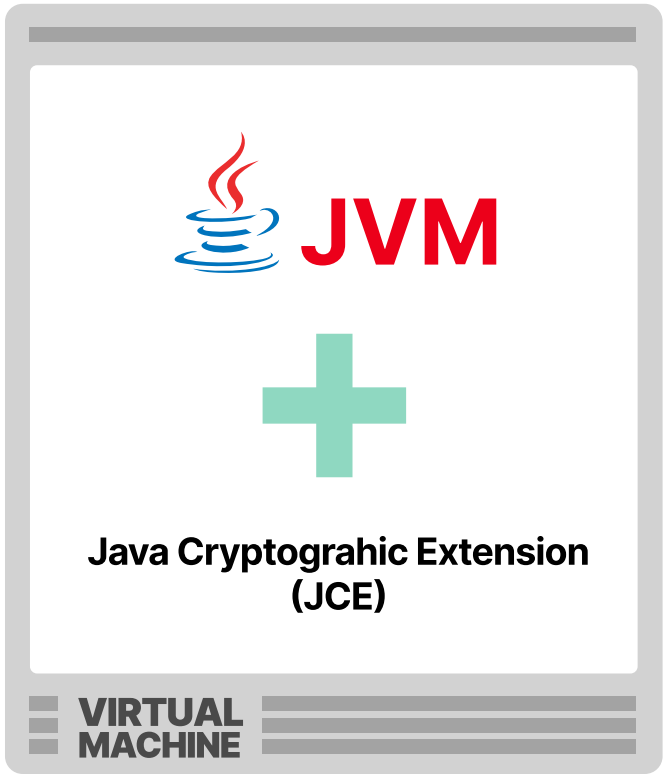
vm_v1.ISO

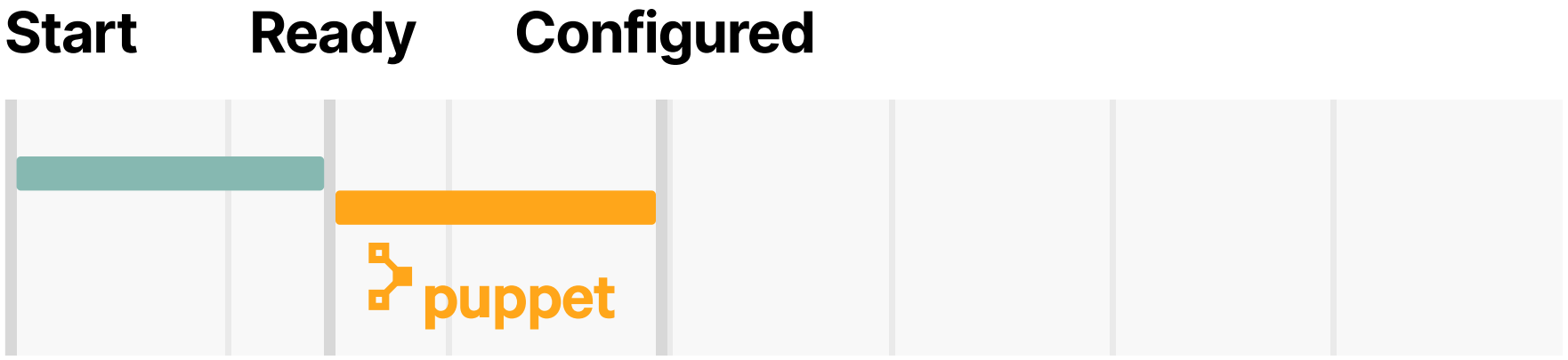
Start

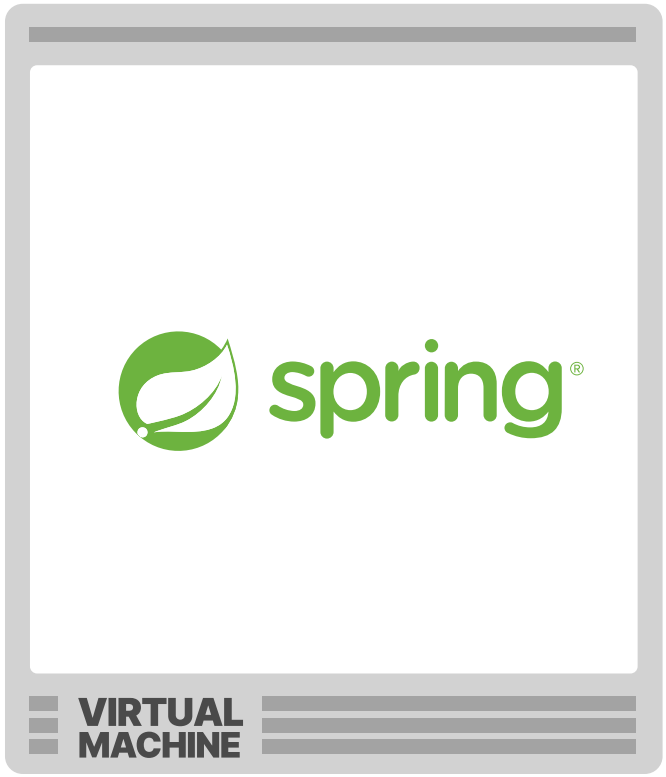


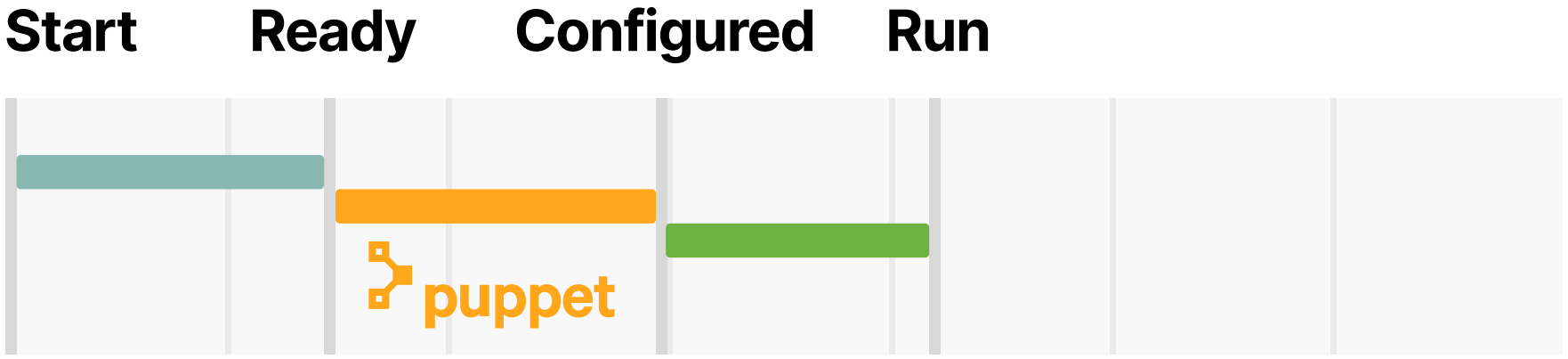


Start **Ready**


JVM
Java Cryptograhic Extension
(JCE)
VIRTUAL
MACHINE


Start
Ready
Configured
puppet


spring
VIRTUAL
MACHINE


Start
Ready
Configured
Run
puppet

Benefits of using virtual machines

Benefits of using virtual machines

Process isolation

Limit resources utilization

Limit visibility

Benefits of using virtual machines

Process isolation

Limit resources utilization

Limit visibility

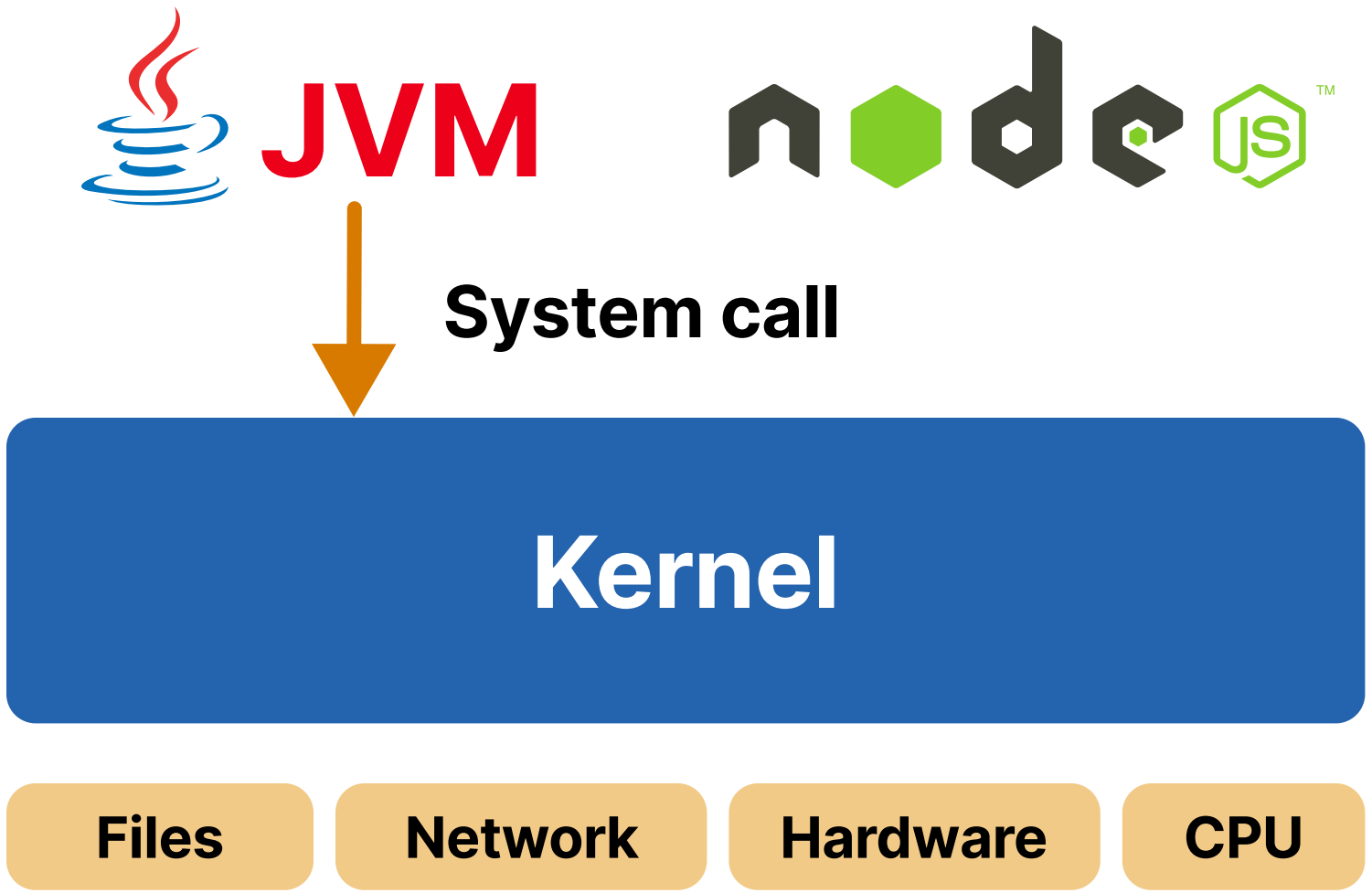
Benefits of using virtual machines

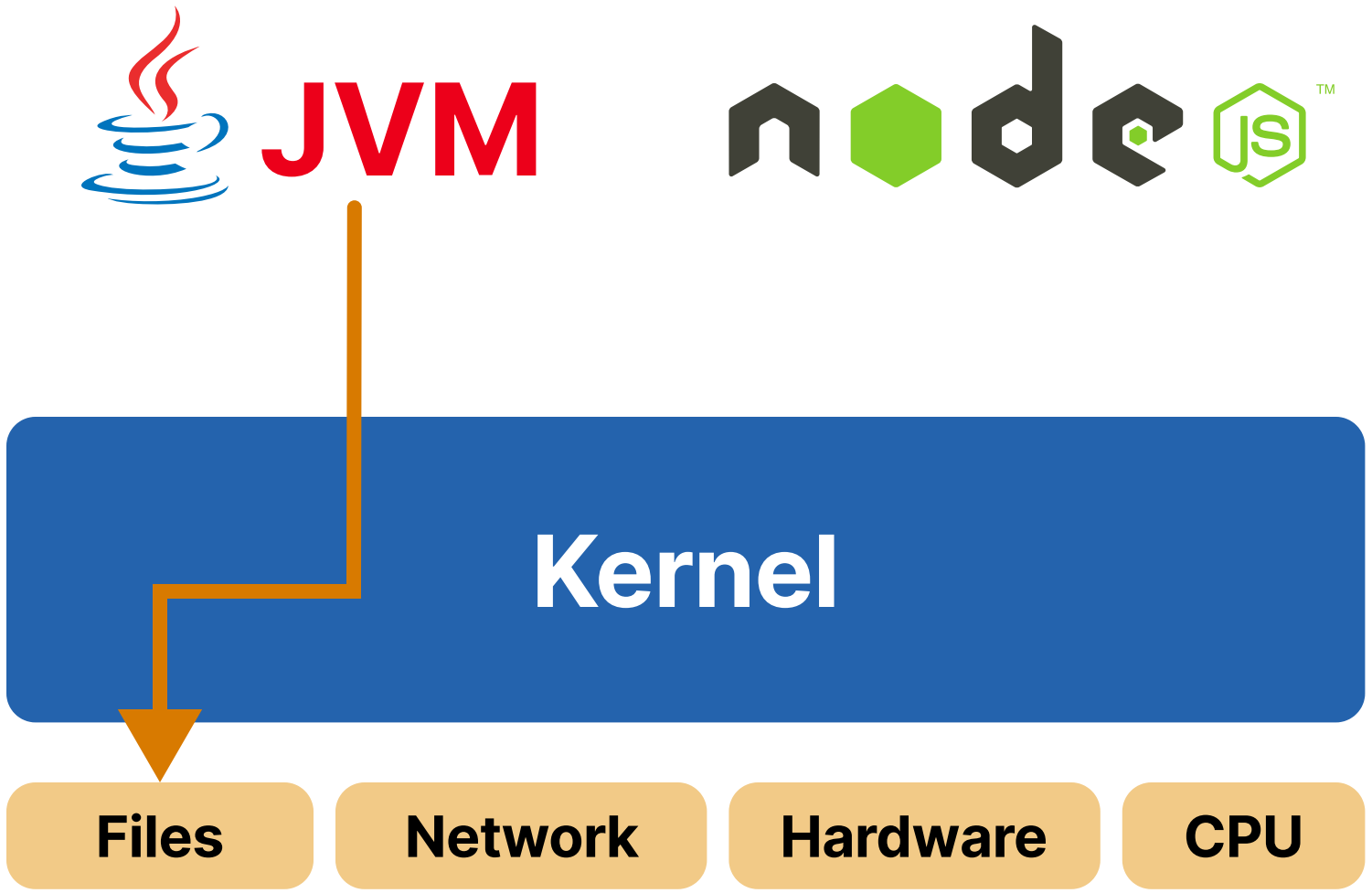
Process isolation

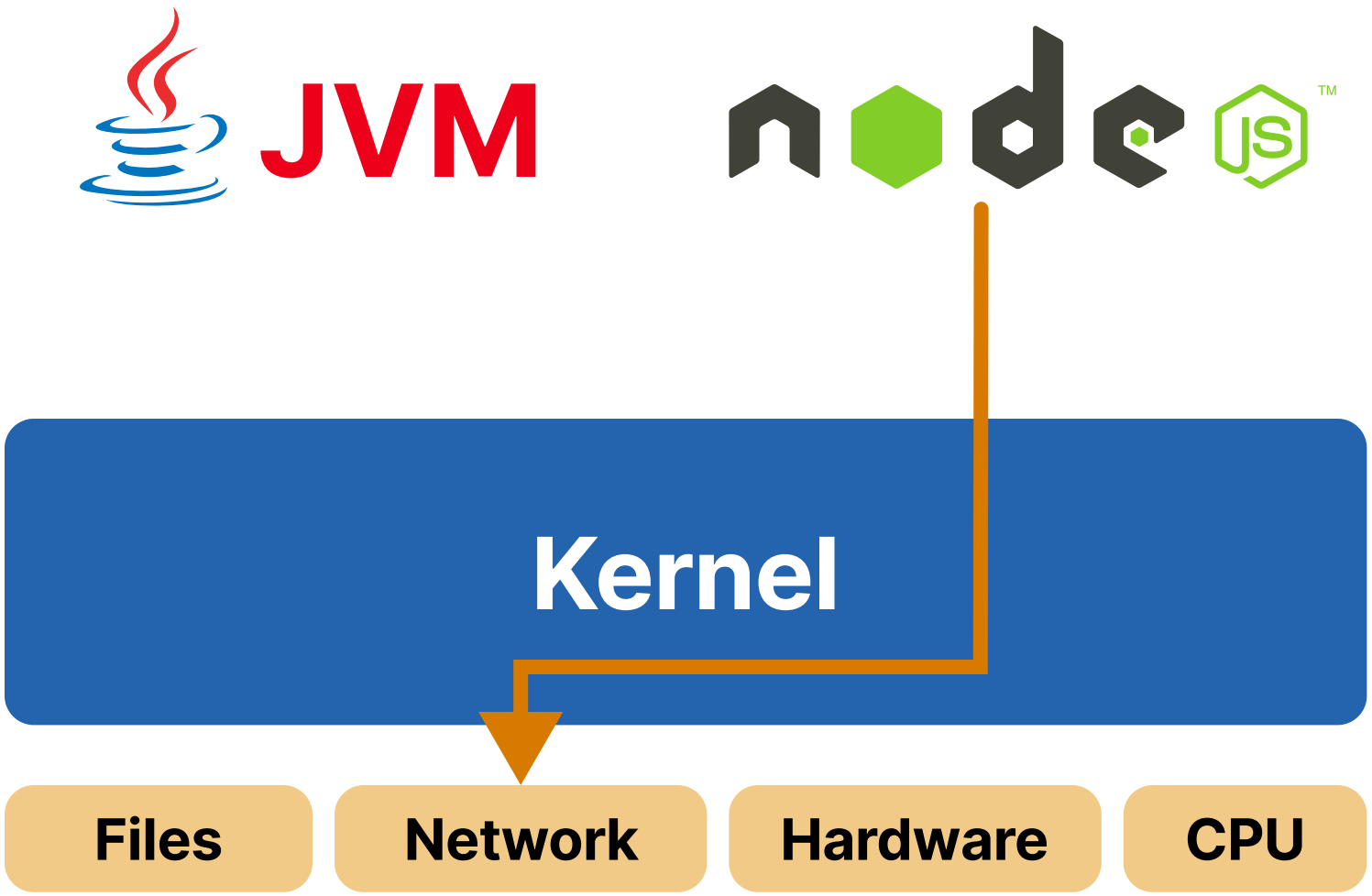
Limit resources utilization

Limit visibility

System calls


JVM
node
JS
System call
Kernel
Files
Network
Hardware
CPU


JVM
node
JS
Kernel
Files
Network
Hardware
CPU


JVM
node
JS
Kernel
Files
Network
Hardware
CPU

Control groups

Control groups

Limit the resources a process can use.

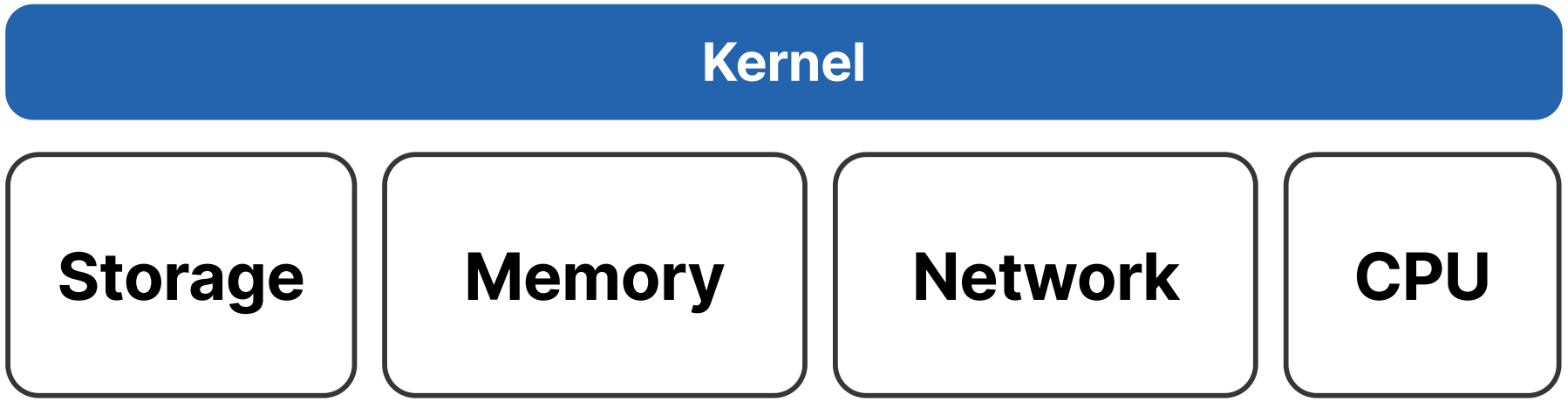
Control groups

"The process can use only 256MB of memory"



Control group #1

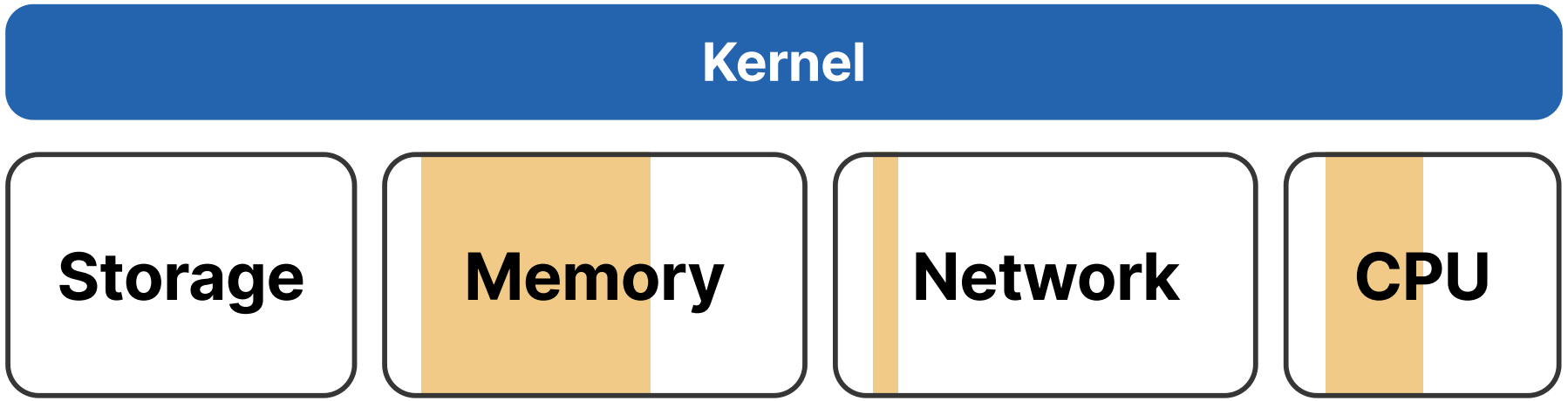






Control group #1



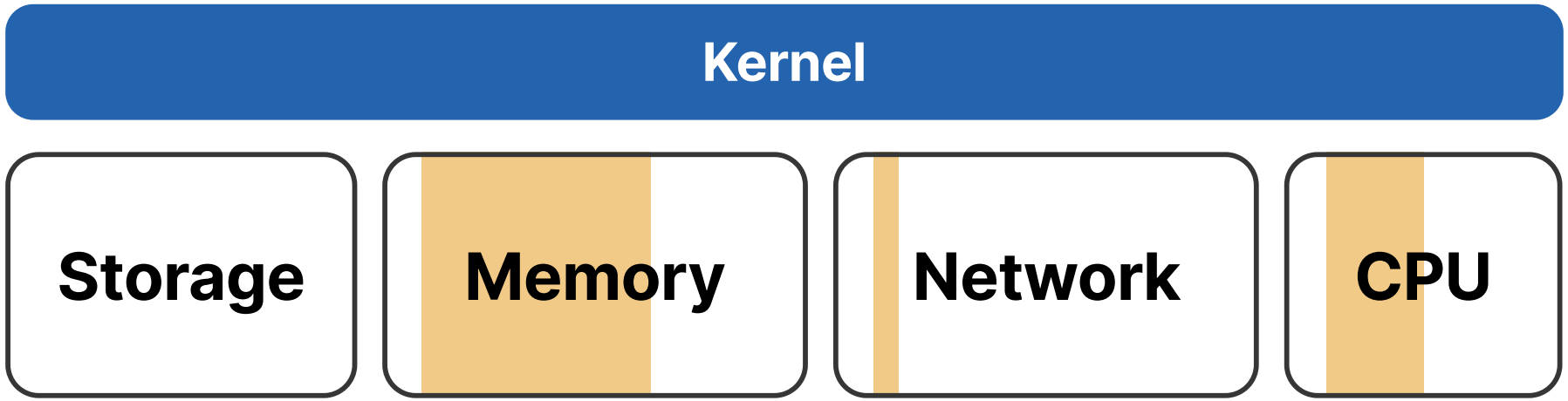




Control group #1



Control group #2

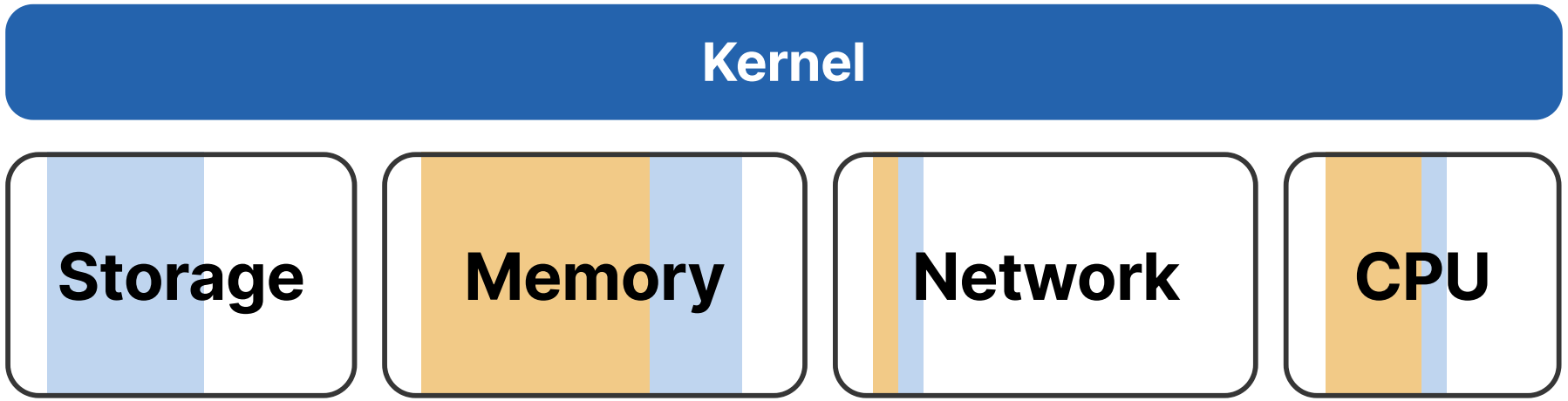




Control group #1



Control group #2

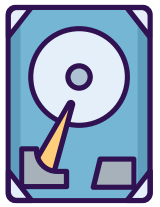


Namespaces

Limit the resources a process can see.

Namespaces

“The process can only see mounts and directories that belong to it”



Hard drive
(root namespace)

/mnt /bin /usr /proc

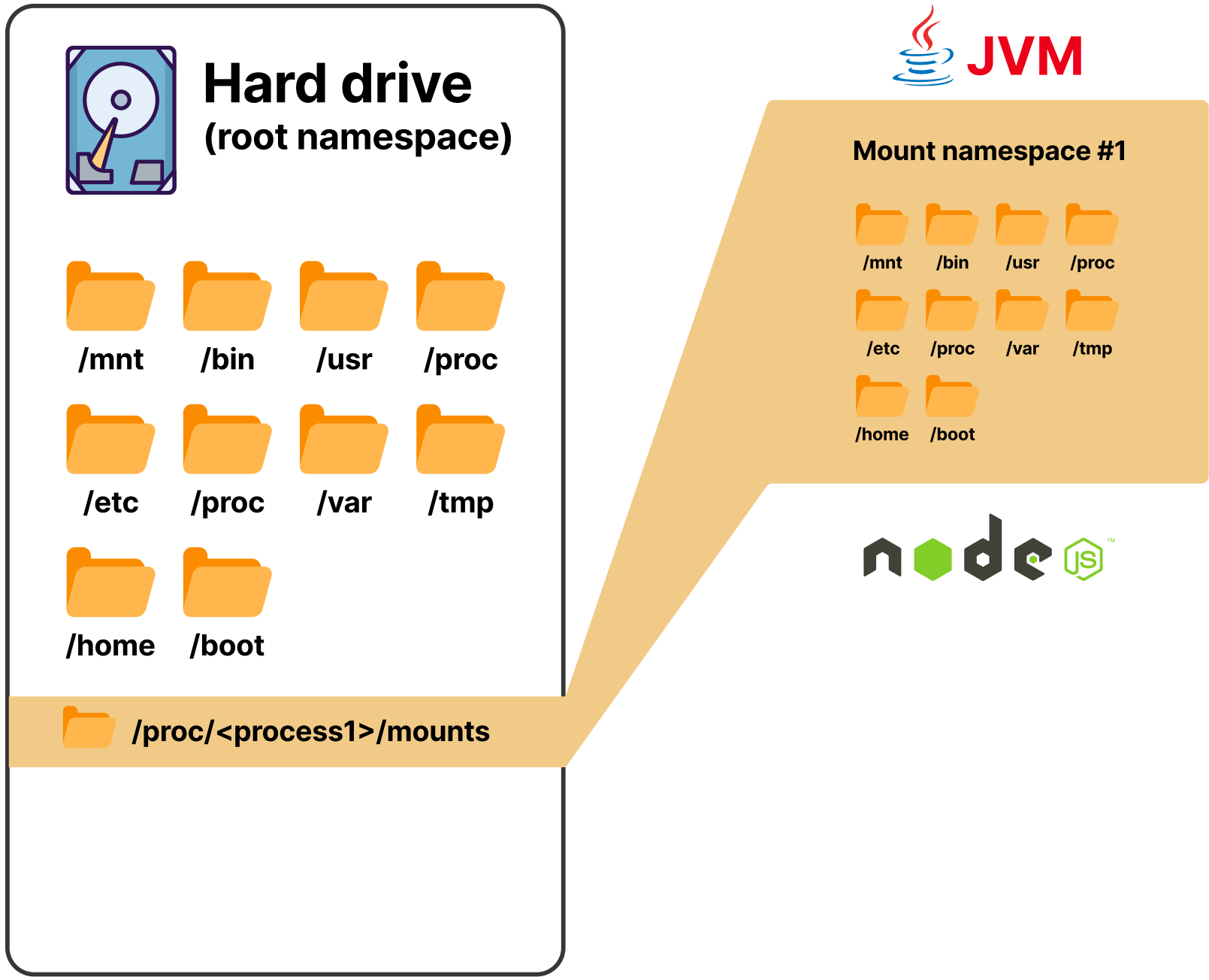
/etc /proc /var /tmp

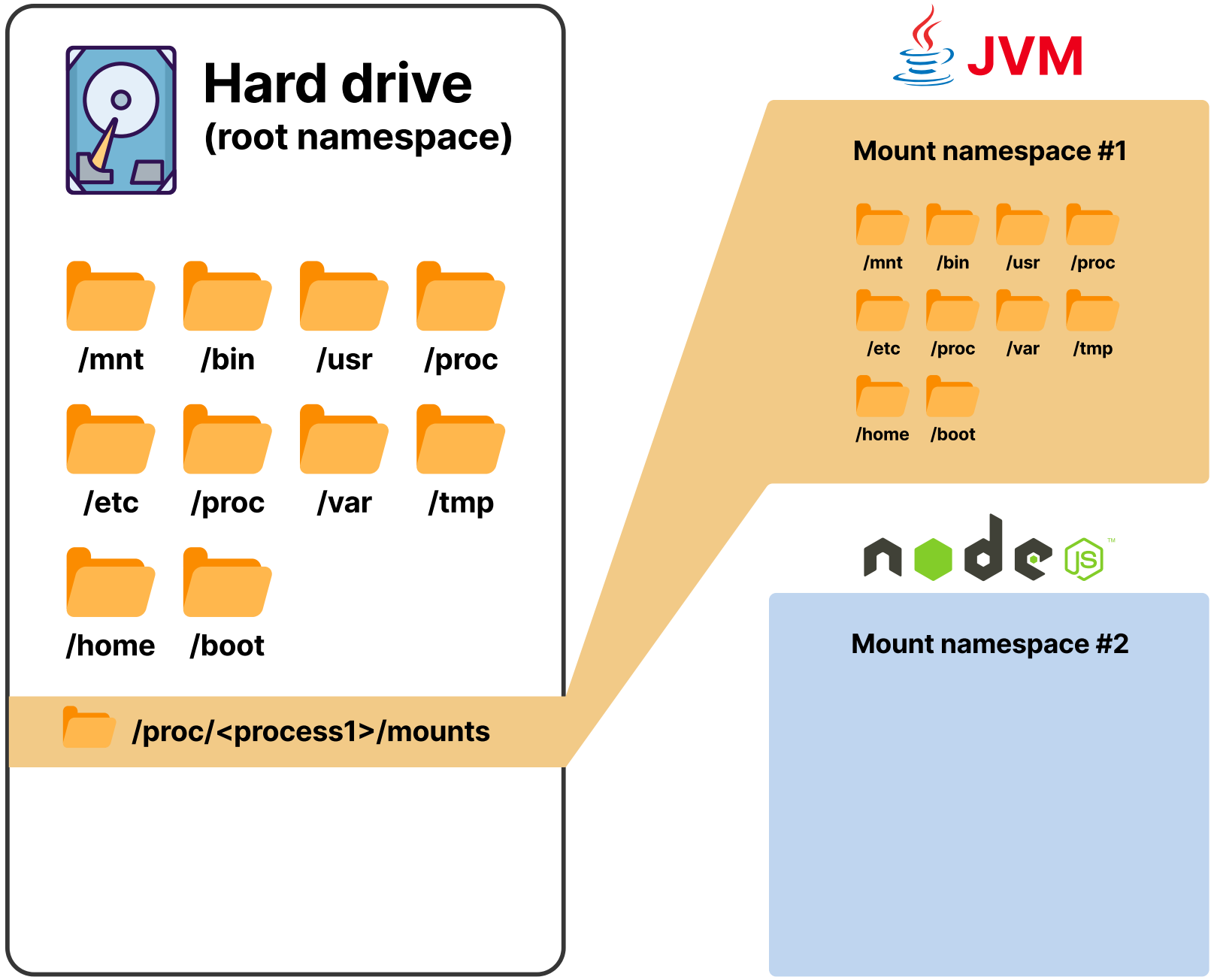
/home /boot

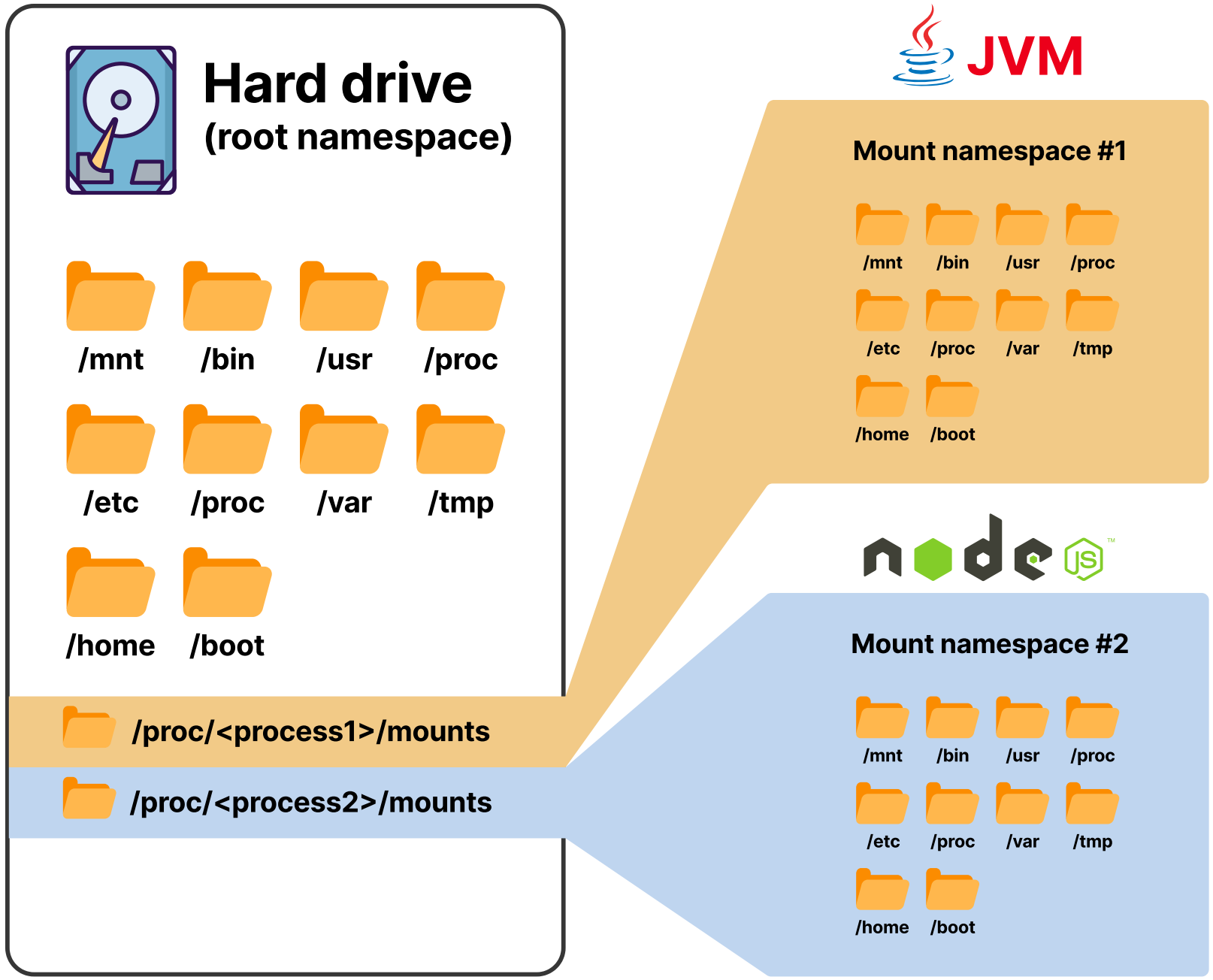
JVM

Mount namespace #1

node


Hard drive
(root namespace)
/mnt
/bin
/usr
/proc
/etc
/proc
/var
/tmp
/home
/boot
/proc/<process1>/mounts
JVM
Mount namespace #1
/mnt
/bin
/usr
/proc
/etc
/proc
/var
/tmp
/home
/boot
node


Hard drive
(root namespace)
/mnt
/bin
/usr
/proc
/etc
/proc
/var
/tmp
/home
/boot
/proc/<process1>/mounts
JVM
Mount namespace #1
/mnt
/bin
/usr
/proc
/etc
/proc
/var
/tmp
/home
/boot
Mount namespace #2


Hard drive
(root namespace)
/mnt
/bin
/usr
/proc
/etc
/proc
/var
/tmp
/home
/boot
/proc/<process1>/mounts
/proc/<process2>/mounts
JVM
Mount namespace #1
/mnt
/bin
/usr
/proc
/etc
/proc
/var
/tmp
/home
/boot
Mount namespace #2
/mnt
/bin
/usr
/proc
/etc
/proc
/var
/tmp
/home
/boot

From cgroups & namespaces to Docker

+ Control groups

+ Control groups
+ Namespaces

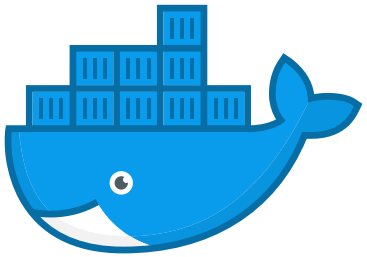
+ Control groups
+ Namespaces
+ (abstractions)

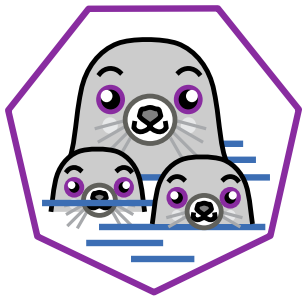
+ Control groups
+ Namespaces
+ (abstractions)

= 

+ Control groups
+ Namespaces
+ (abstractions)

=





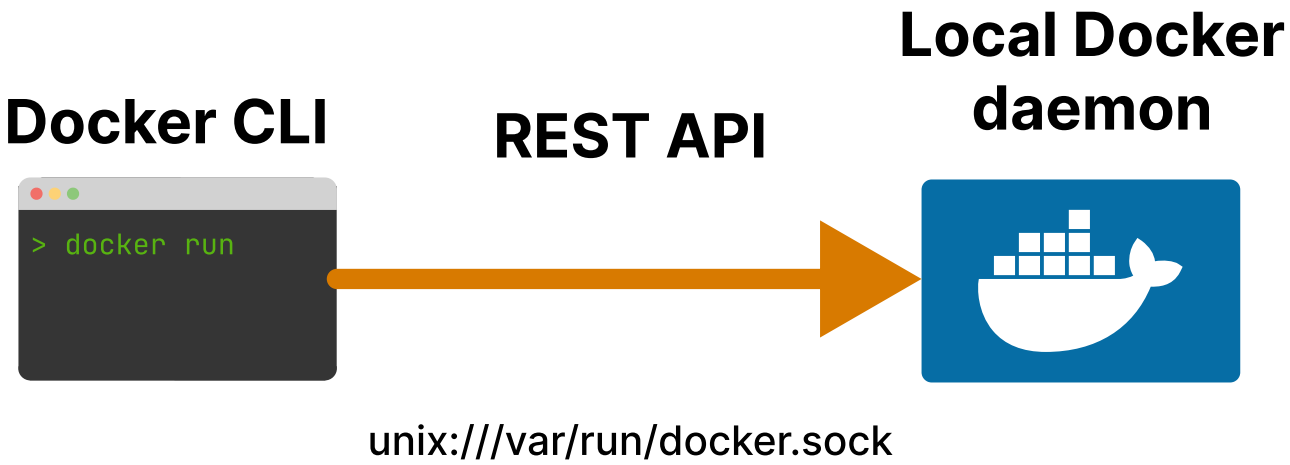


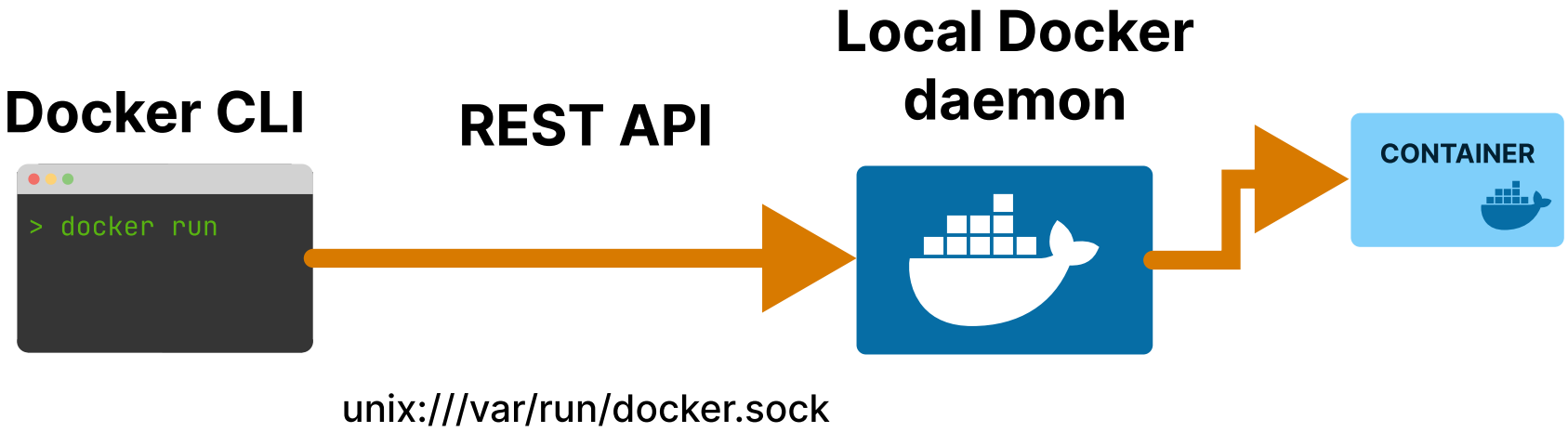

docker

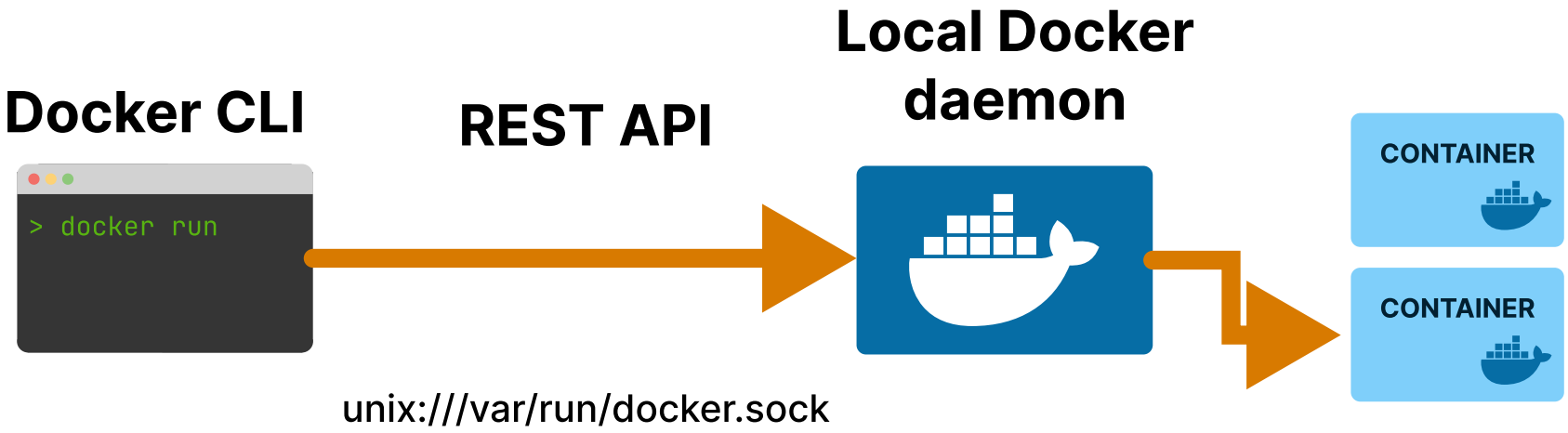
Architecture

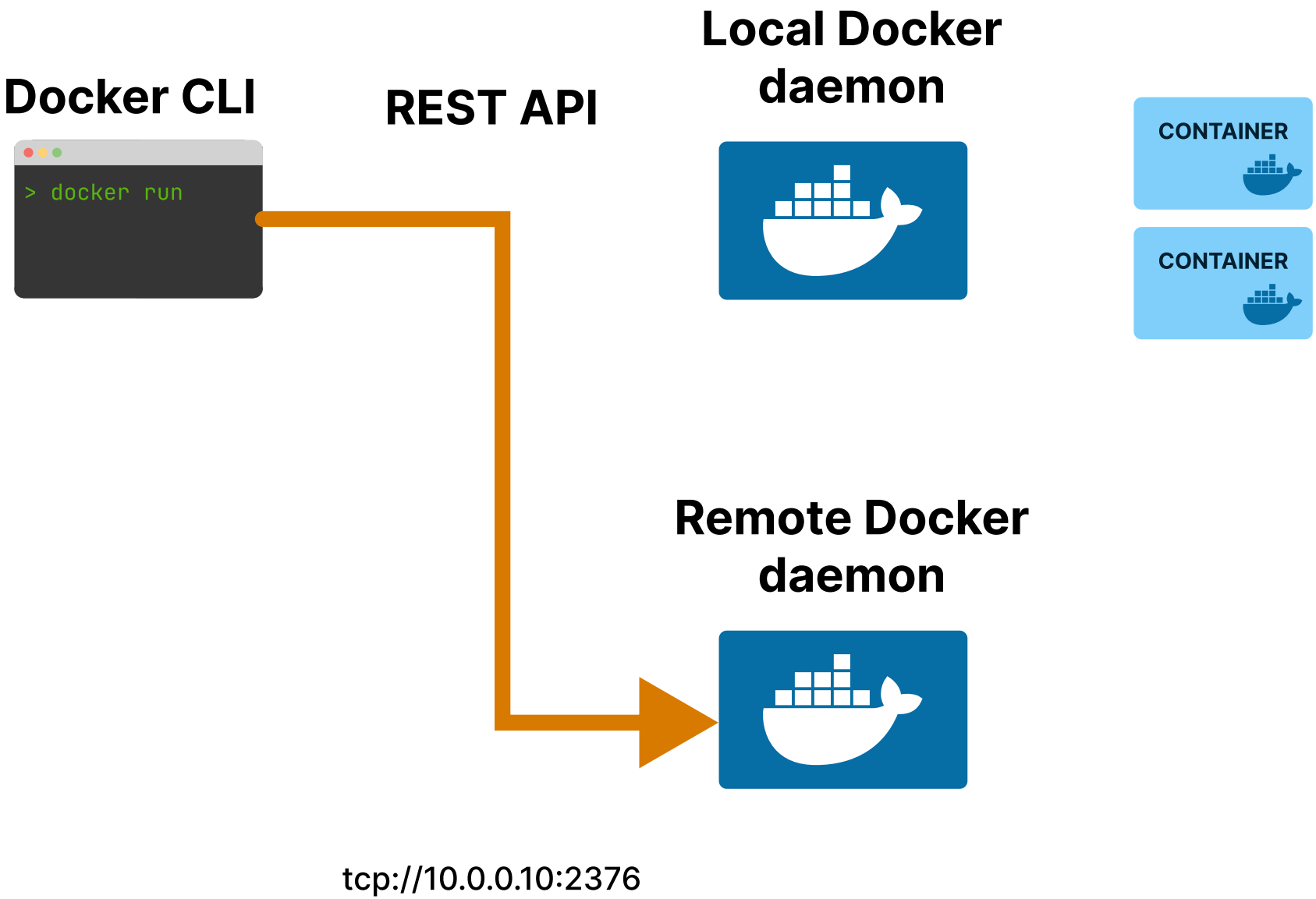
Docker

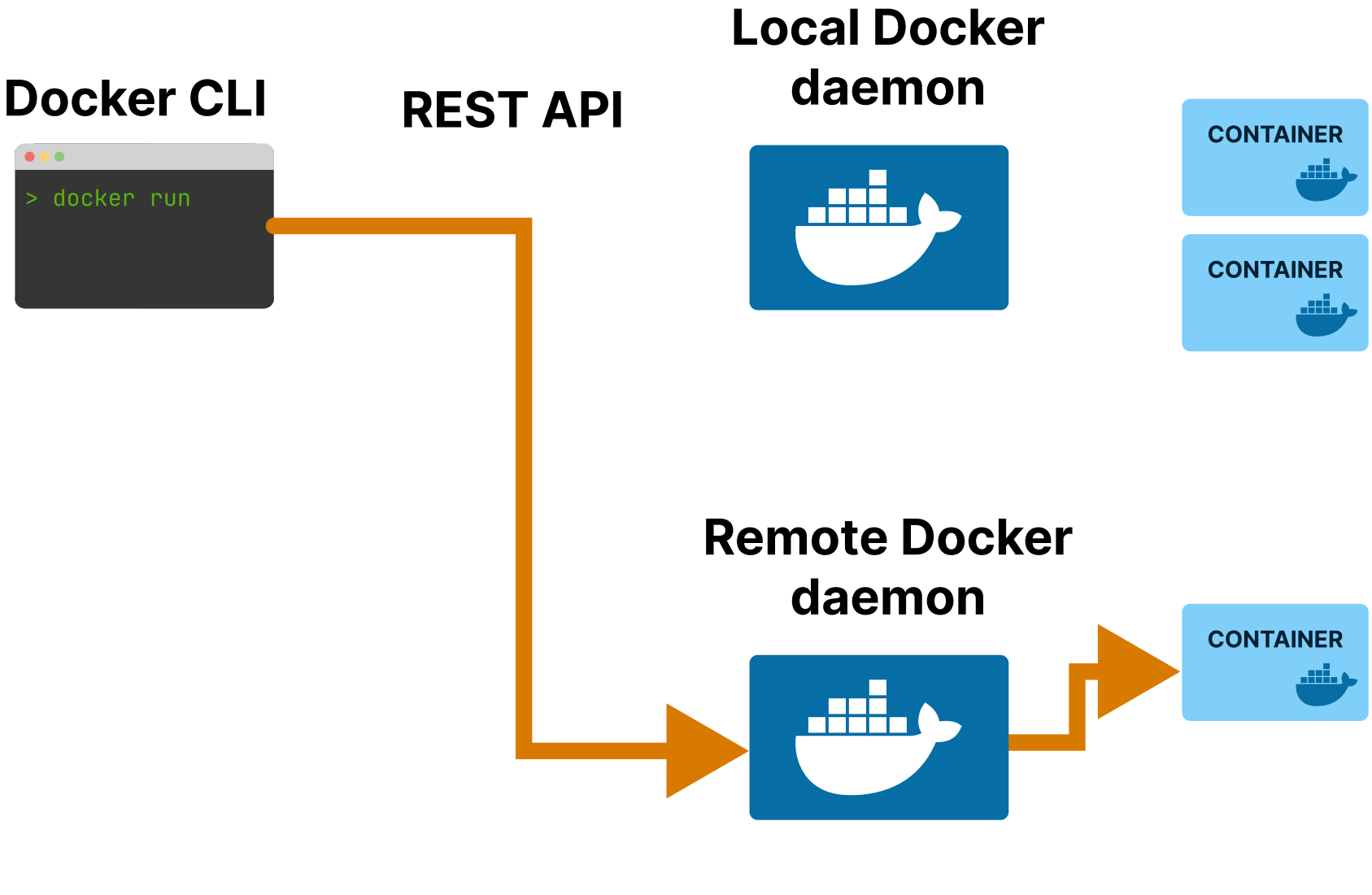

Docker CLI
> docker run
Local Docker daemon


Docker CLI
REST API
Local Docker daemon
> docker run
unix:///var/run/docker.sock


Docker CLI
REST API
Local Docker daemon
CONTAINER
> docker run
unix:///var/run/docker.sock


Docker CLI
REST API
Local Docker daemon
> docker run
CONTAINER
CONTAINER
unix:///var/run/docker.sock


Docker CLI
> docker run
REST API
Local Docker daemon
CONTAINER
CONTAINER
Remote Docker daemon
tcp://10.0.0.10:2376


Docker CLI
> docker run
REST API
Local Docker daemon
CONTAINER
CONTAINER
Remote Docker daemon
CONTAINER
tcp://10.0.0.10:2376

Running containers

Docker

$ docker run -ti ubuntu bash

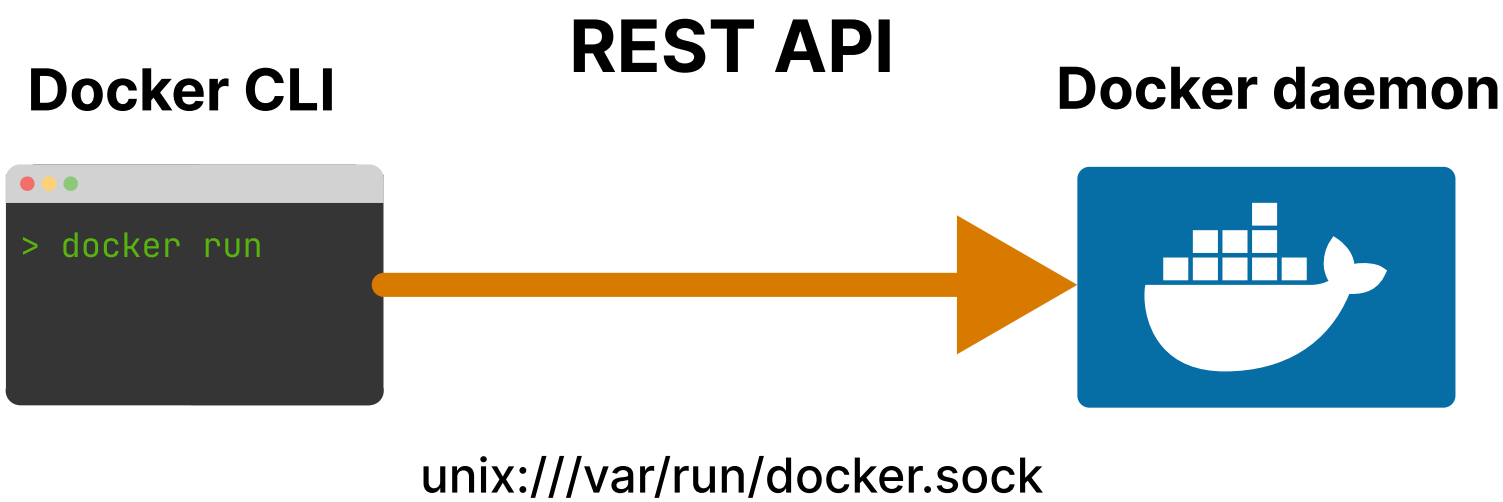
```
$ docker run -ti ubuntu bash
```

```
$ docker run -ti ubuntu bash
```

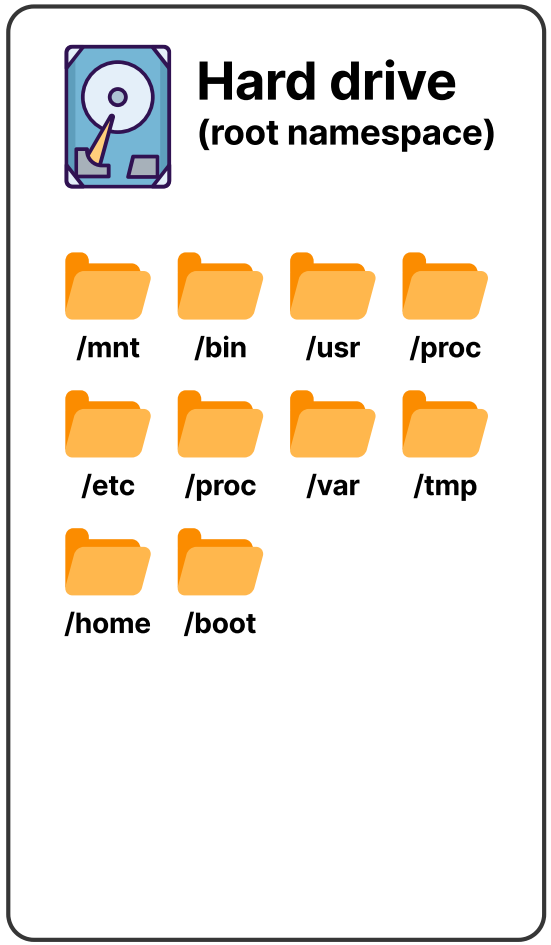
```
$ docker run -ti ubuntu bash
```

```
$ docker run -ti ubuntu bash
```

$ docker run -ti ubuntu bash



$ docker run -ti ubuntu bash

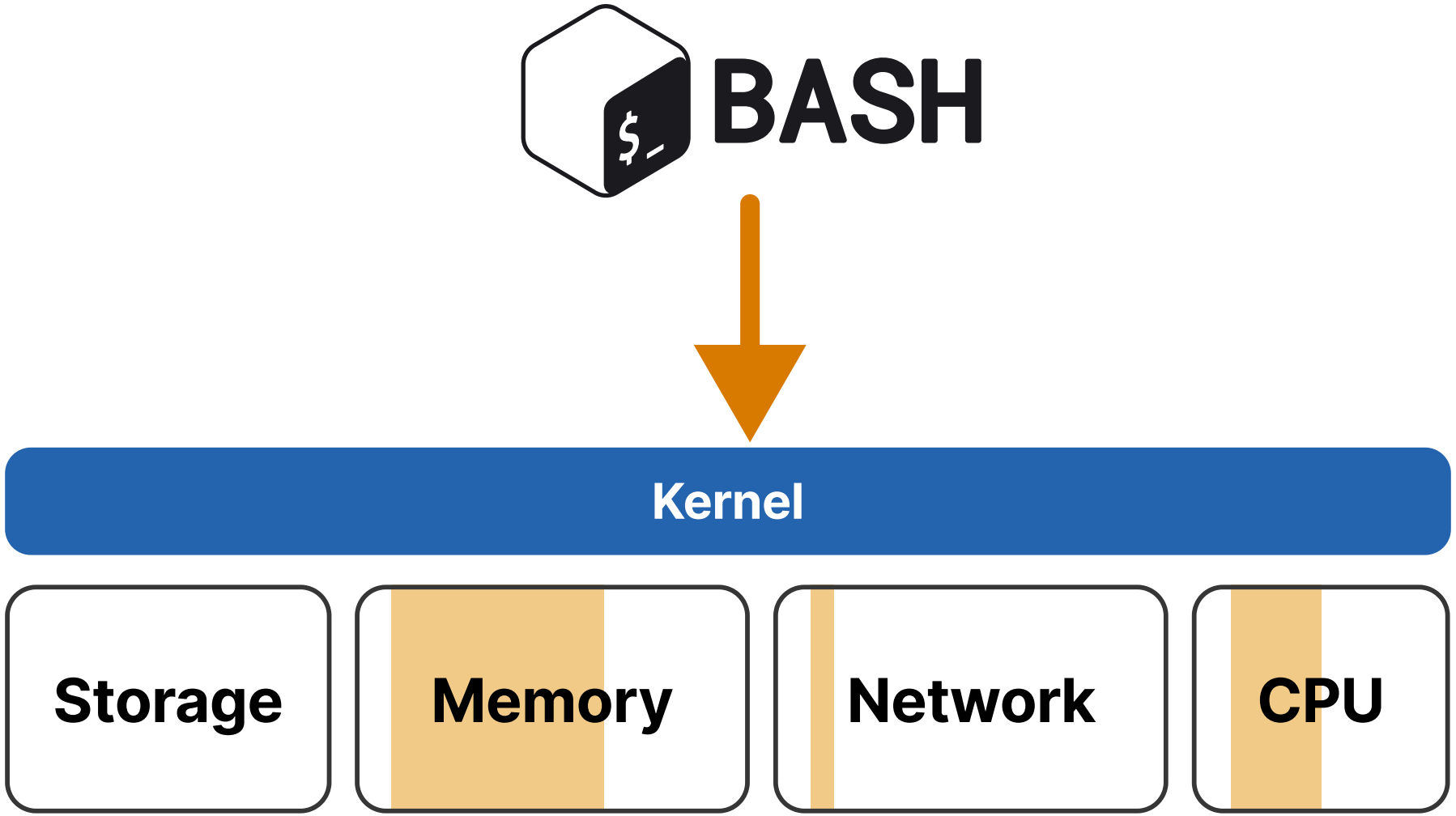


ubuntu

Mount namespace #1

FULLY ISOLATED

$ docker run -ti ubuntu bash

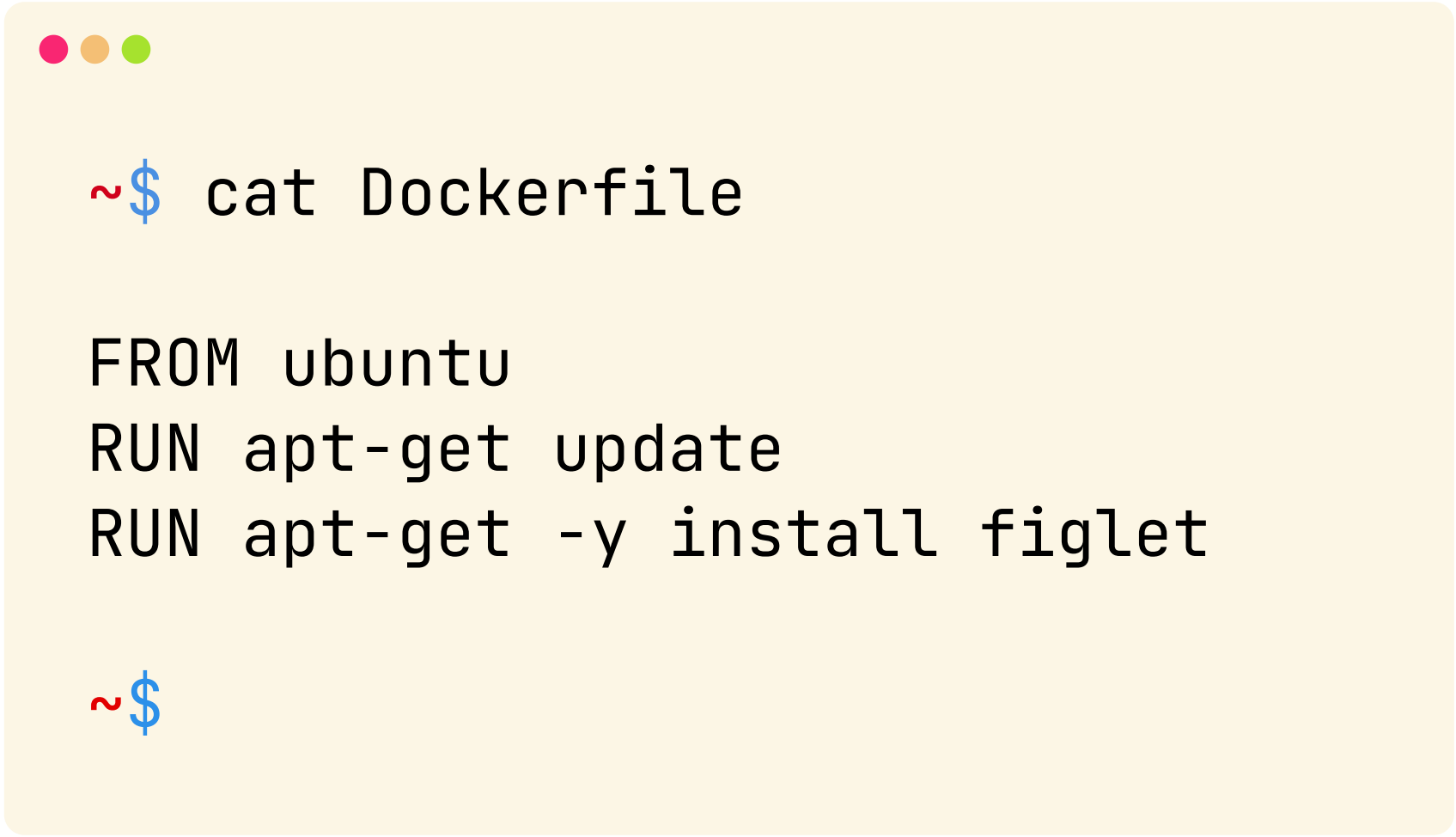


```
$ docker run -ti --rm ubuntu bash
```

Docker run tip

Building images

Docker

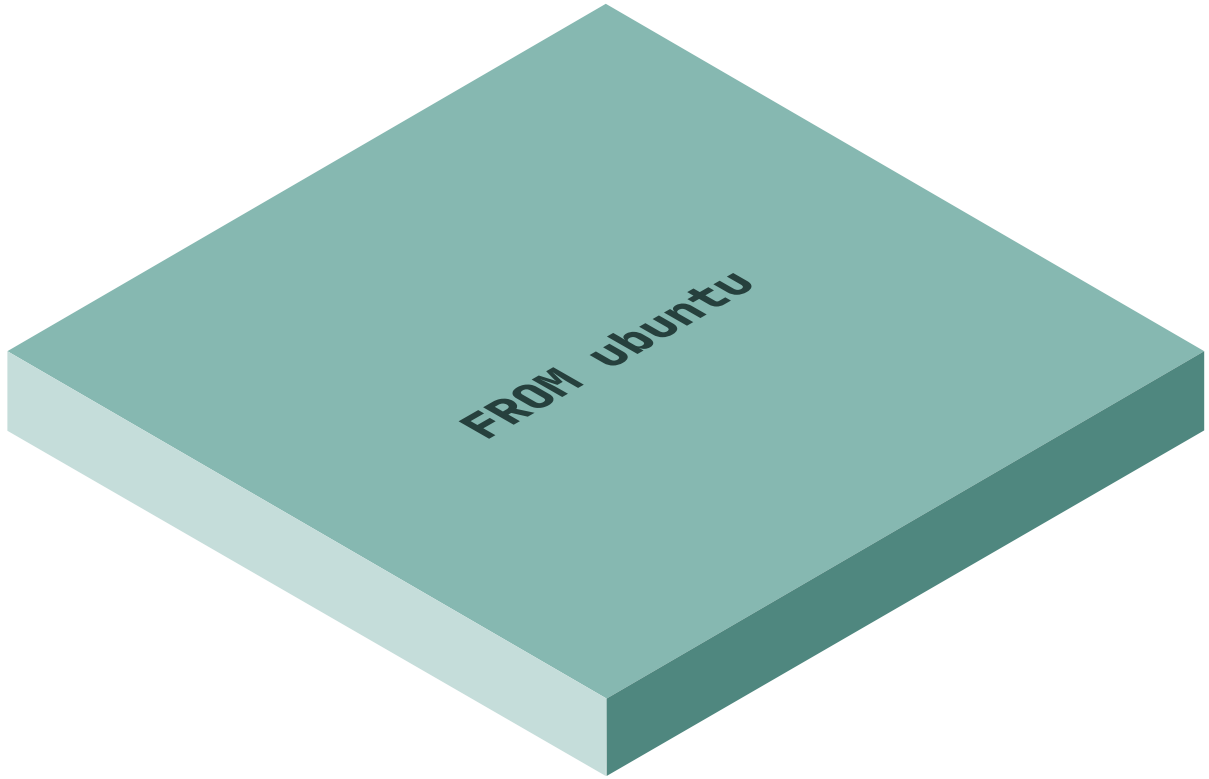

~$ cat Dockerfile
FROM ubuntu
RUN apt-get update
RUN apt-get -y install figlet
~$

```
$ docker build -t app .
```

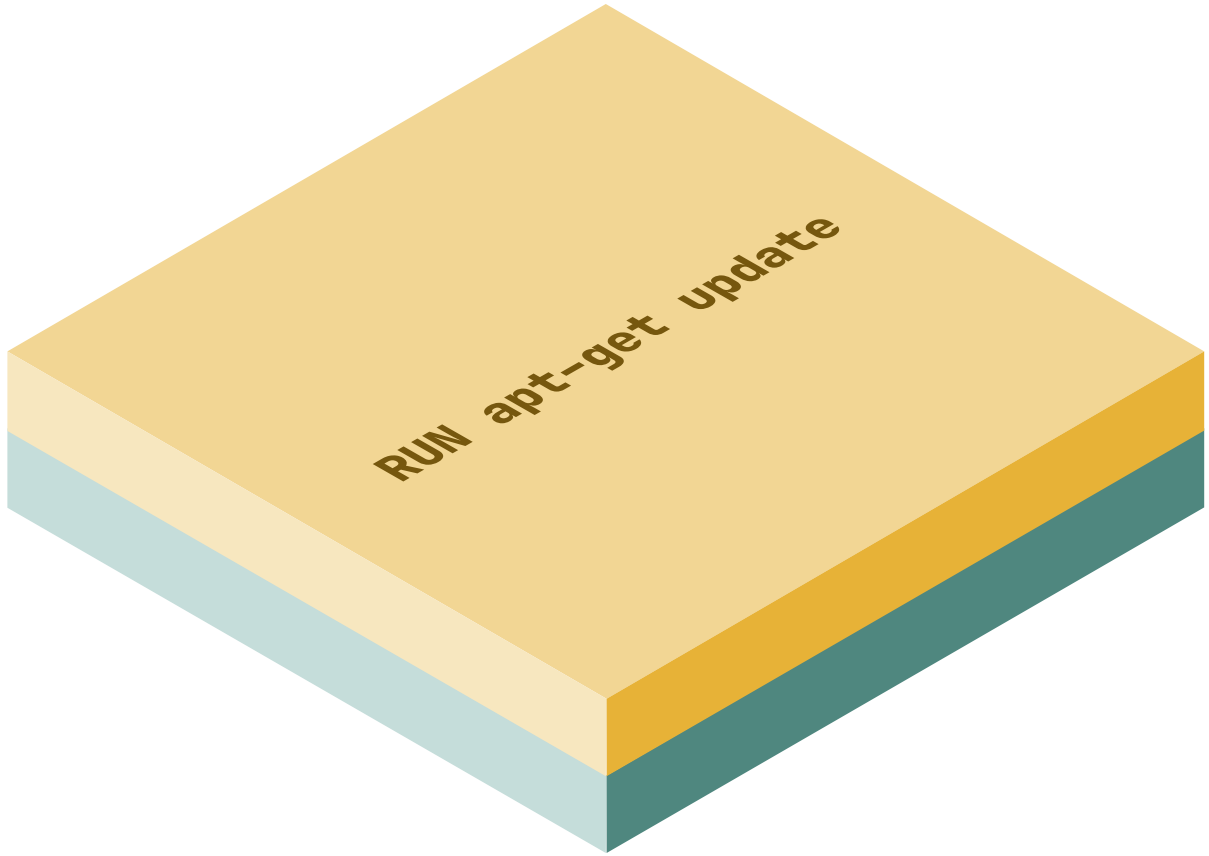
```
$ docker build -t app .
```

```
$ docker build -t app .
```

```
$ docker build -t app .
```



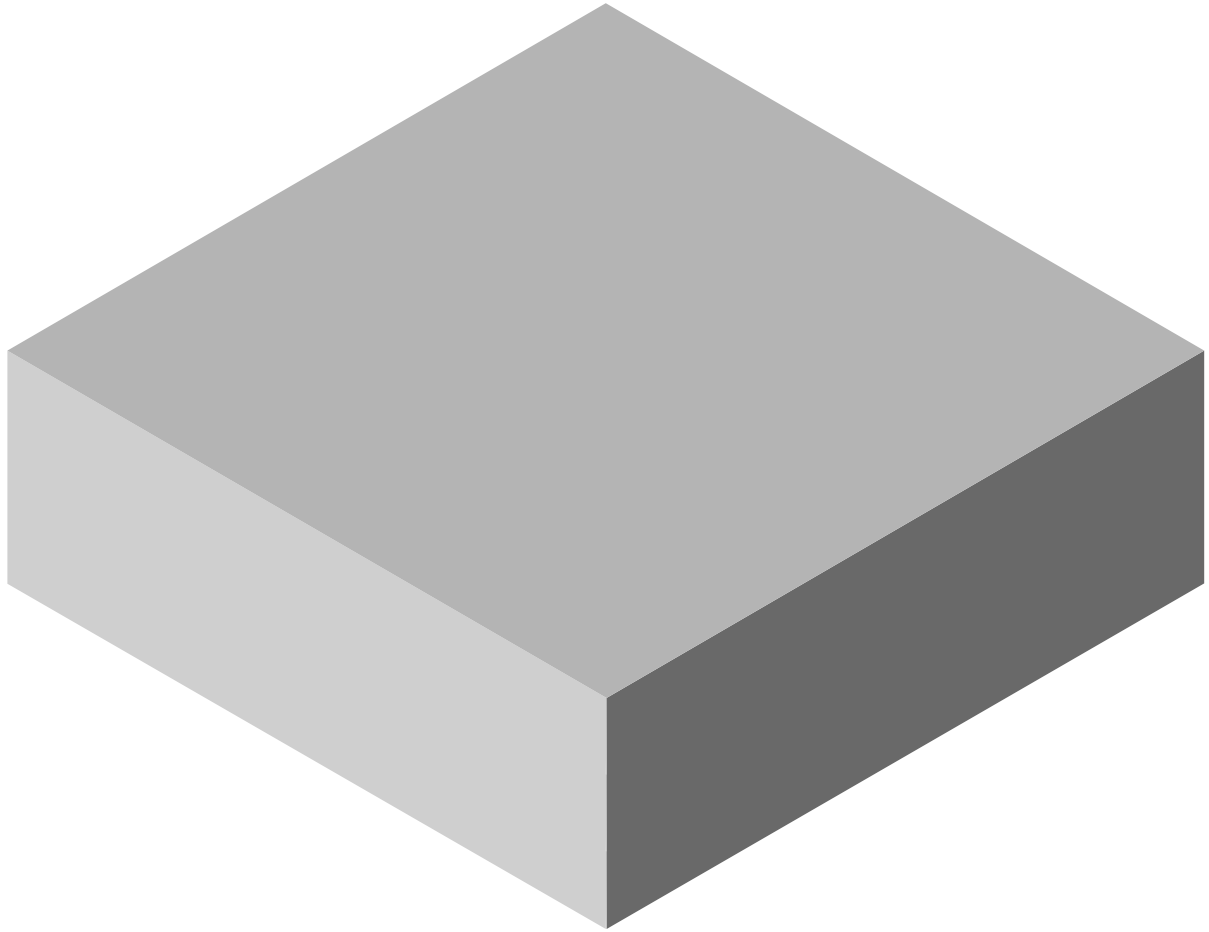
Layer 1



Layer 2



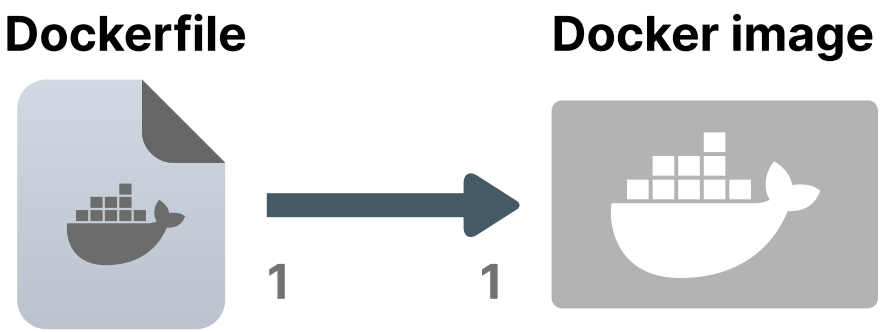
Layer 3

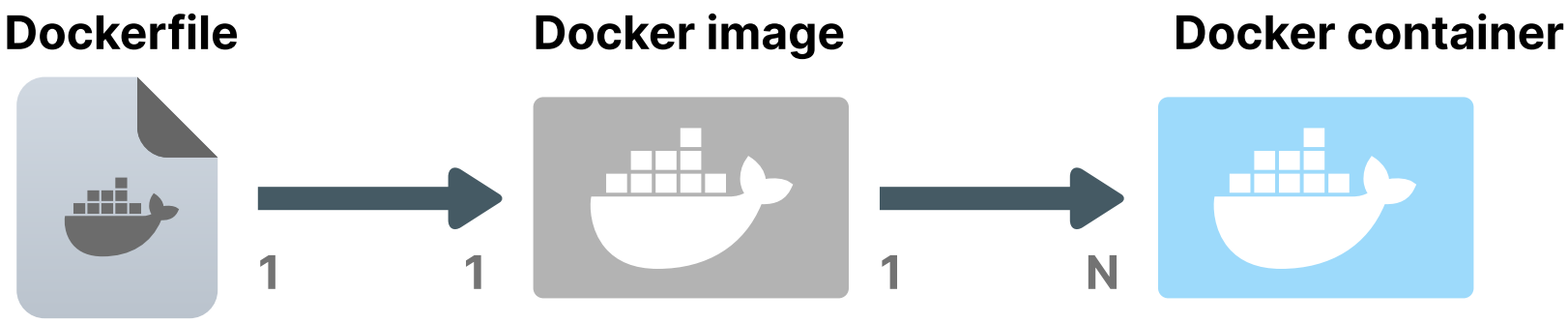


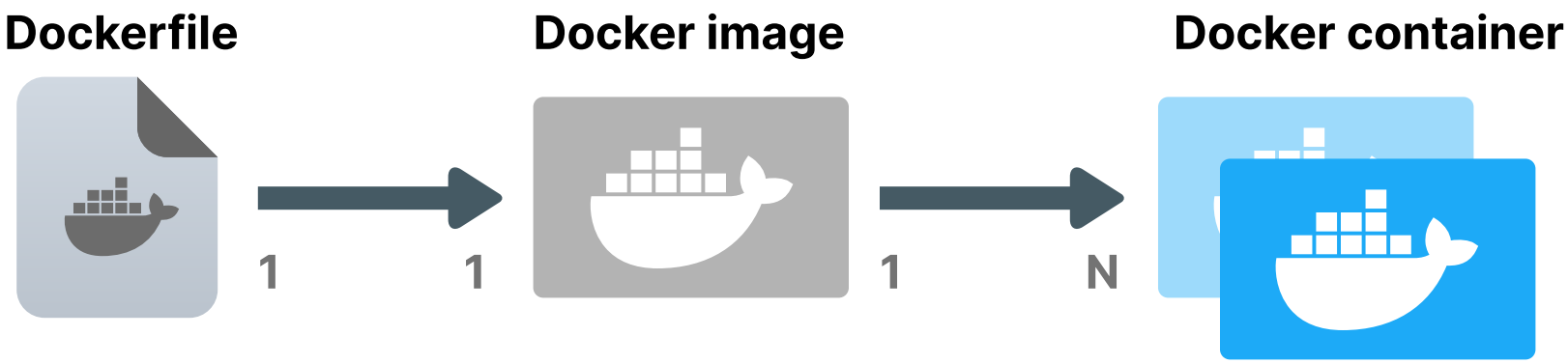
Container image

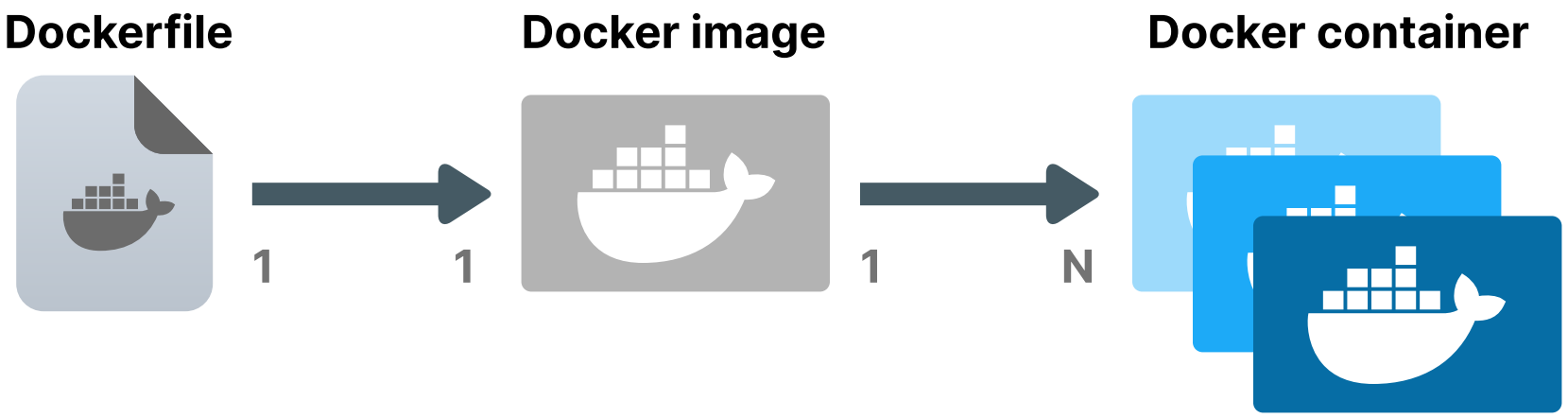
Dockerfile, images and containers

Docker


Dockerfile
Docker image
1
1

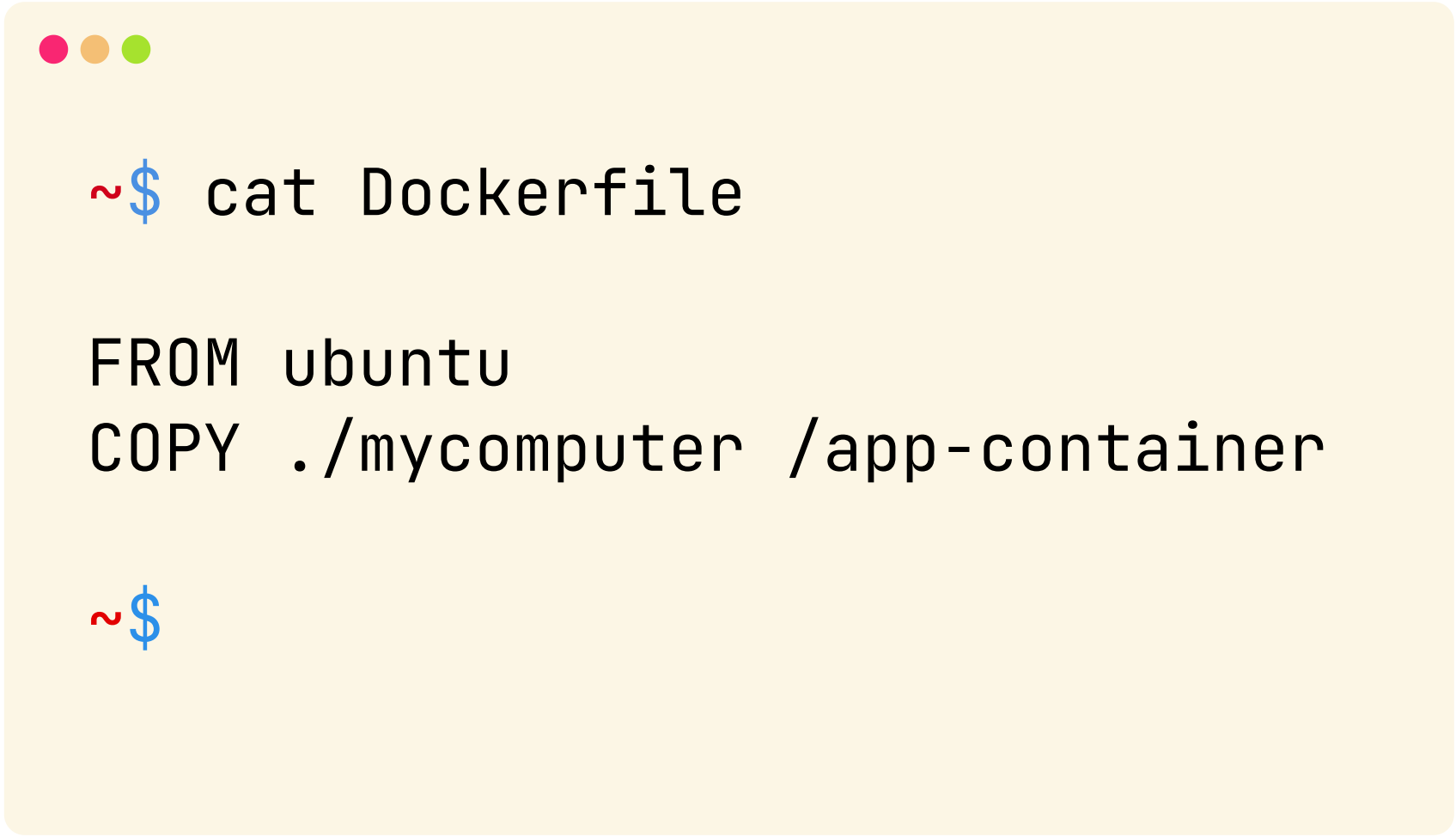

Dockerfile
Docker image
Docker container
1
1
1
N

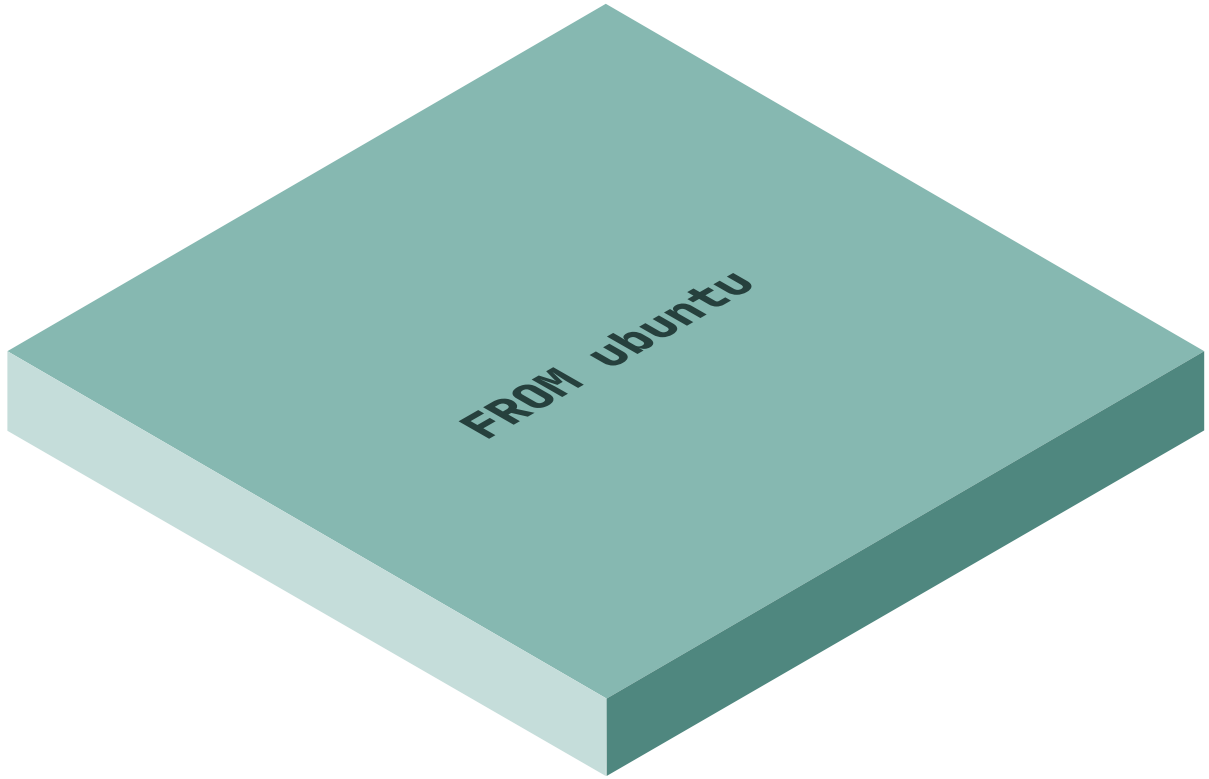

Dockerfile
Docker image
Docker container
1
1
1
N


Dockerfile
Docker image
Docker container
1
1
1
N

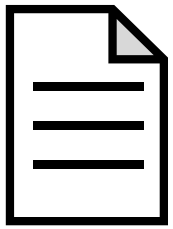
Copying files into a container

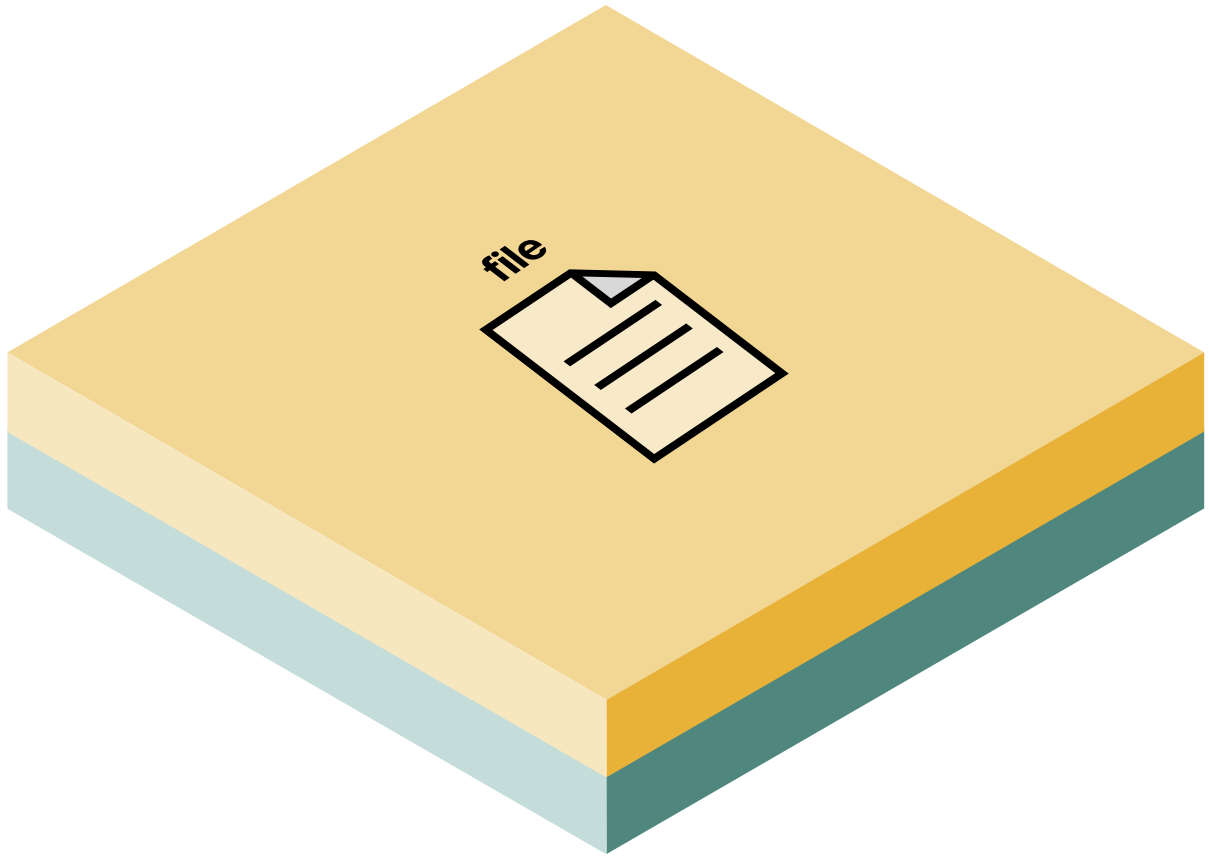
Docker

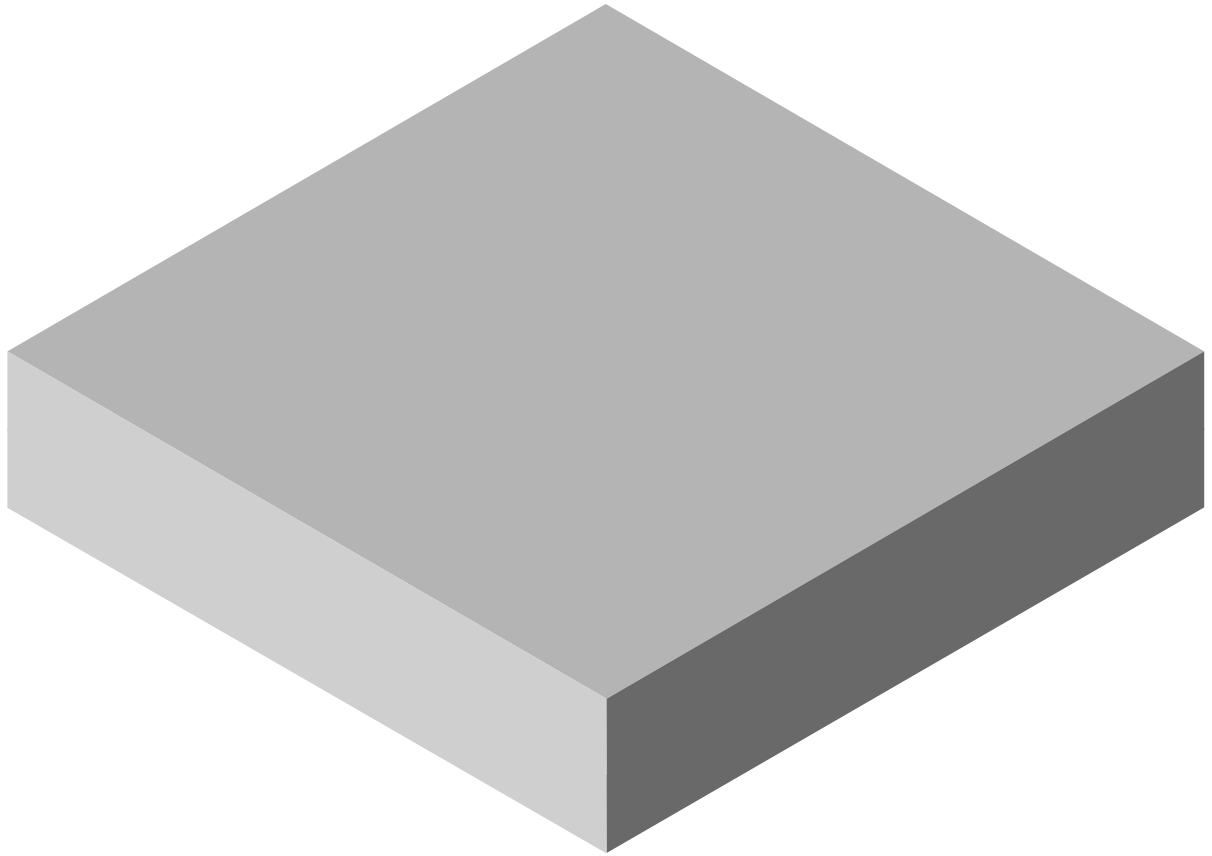

~$ cat Dockerfile
FROM ubuntu
COPY ./mycomputer /app-container
~$



file




file



Container image

Mounting a filesystem

Docker

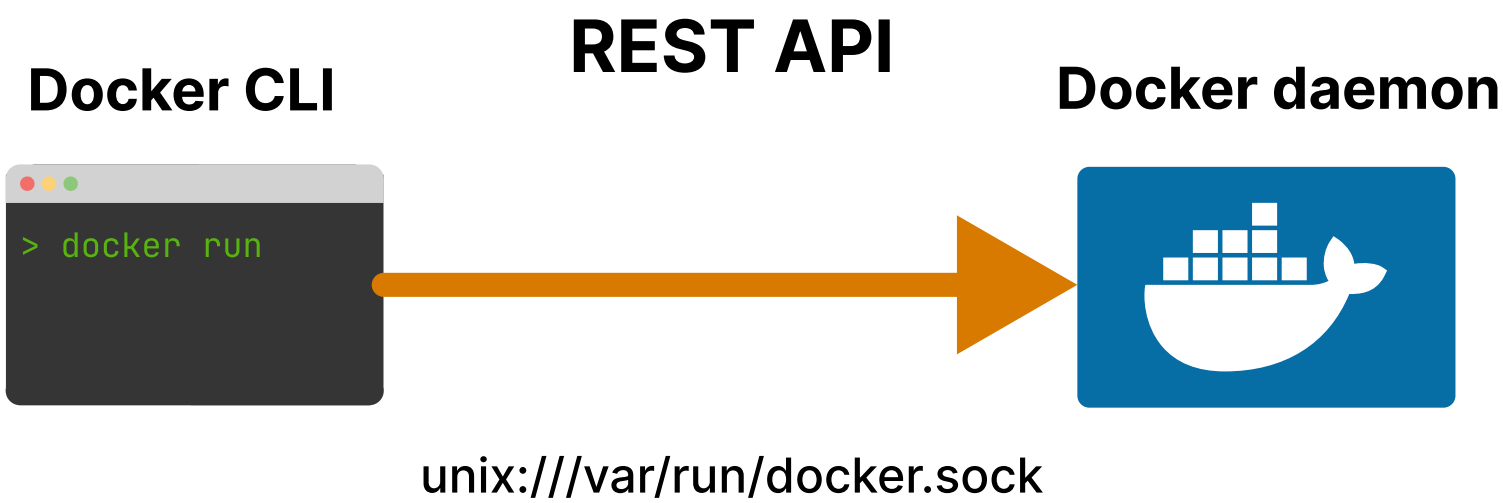
```
$ docker run -ti -v /tmp:/app ubuntu
```

```
$ docker run -ti -v /tmp:/app ubuntu
```

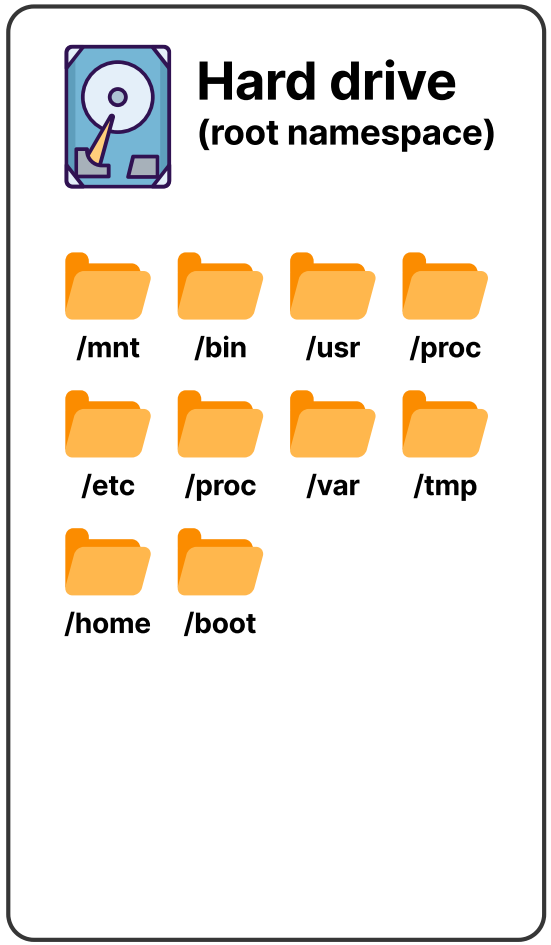
```
$ docker run -ti -v /tmp:/app ubuntu
```

```
$ docker run -ti -v /tmp:/app ubuntu
```

$ docker run -ti -v /tmp:/app ubuntu



$ docker run -ti -v /tmp:/app ubuntu

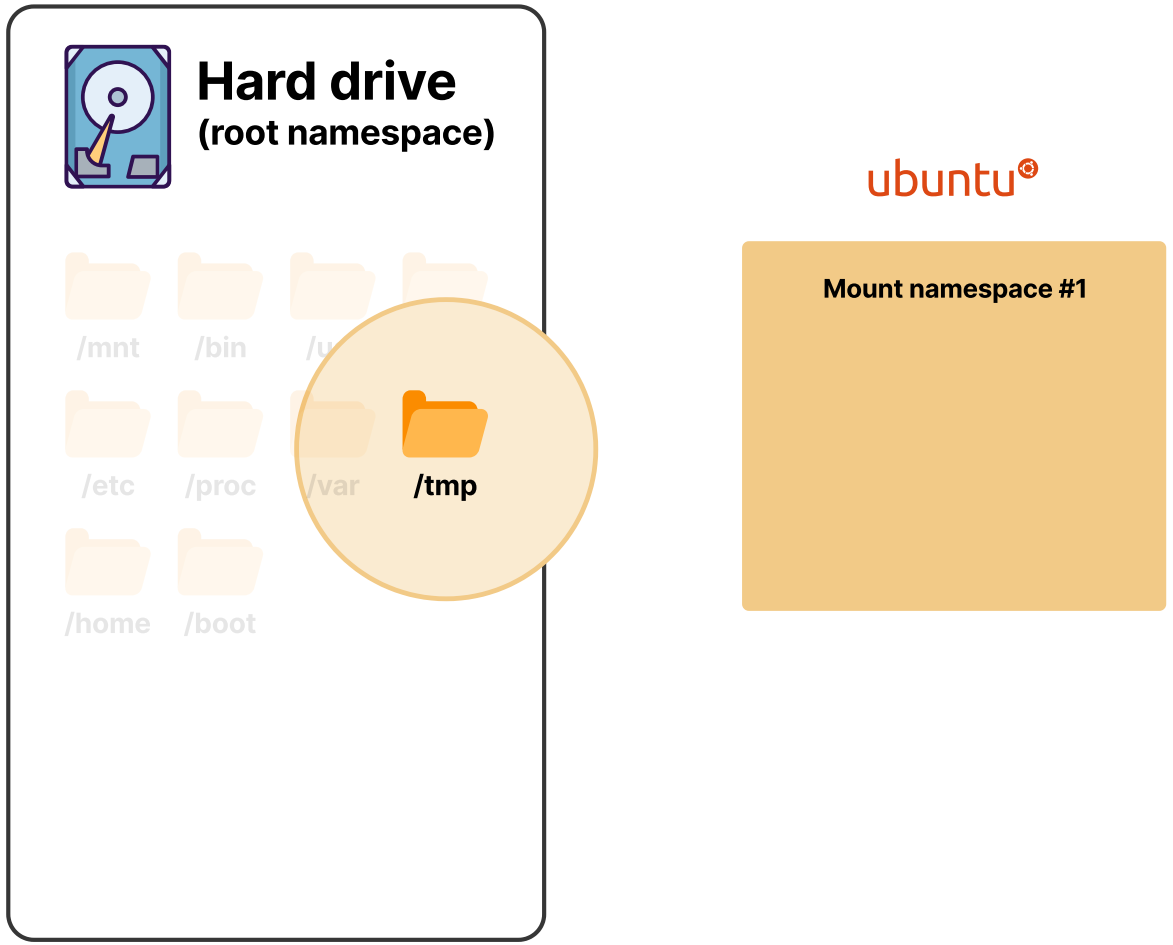


ubuntu

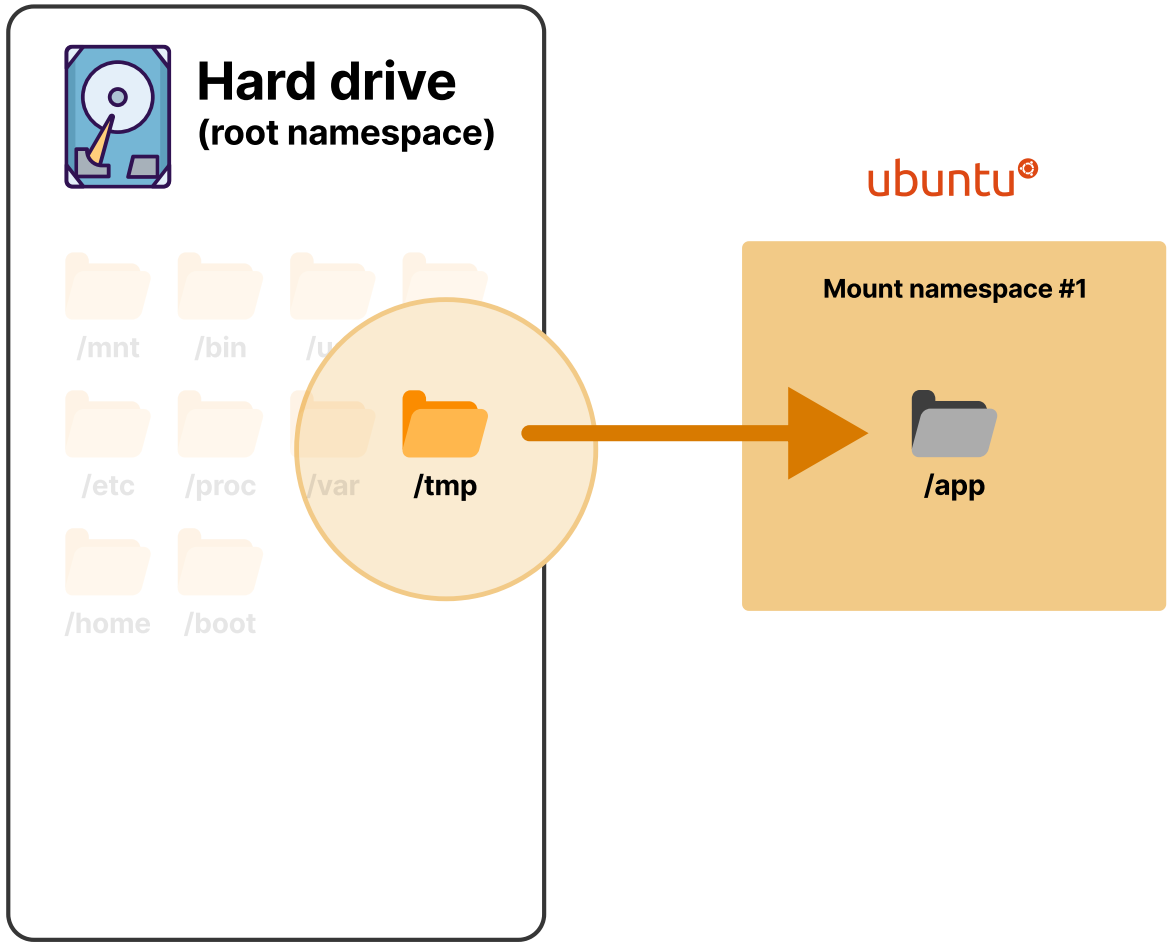
Mount namespace #1

FULLY ISOLATED

$ docker run -ti -v /tmp:/app ubuntu

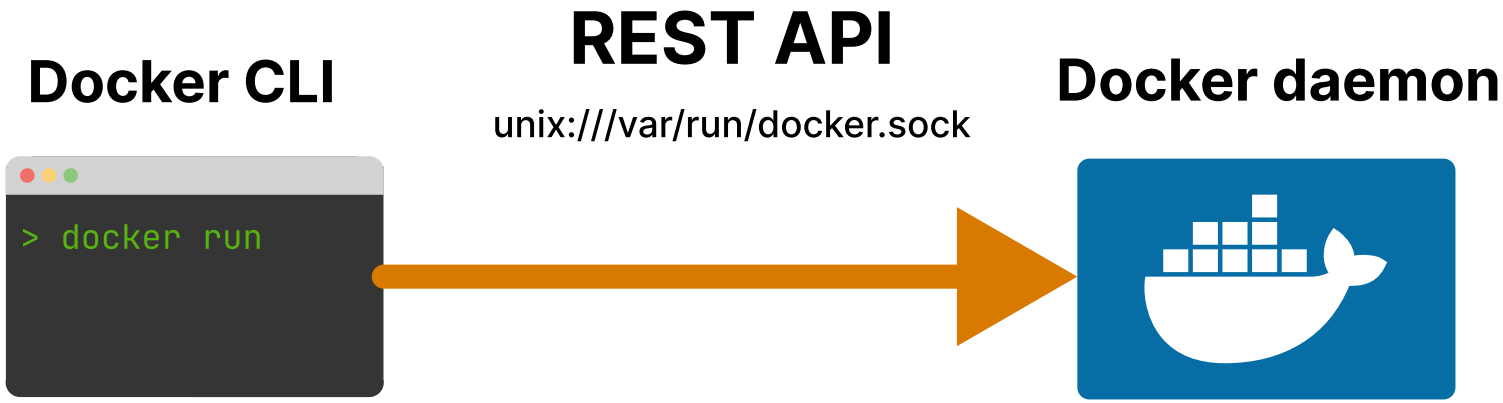


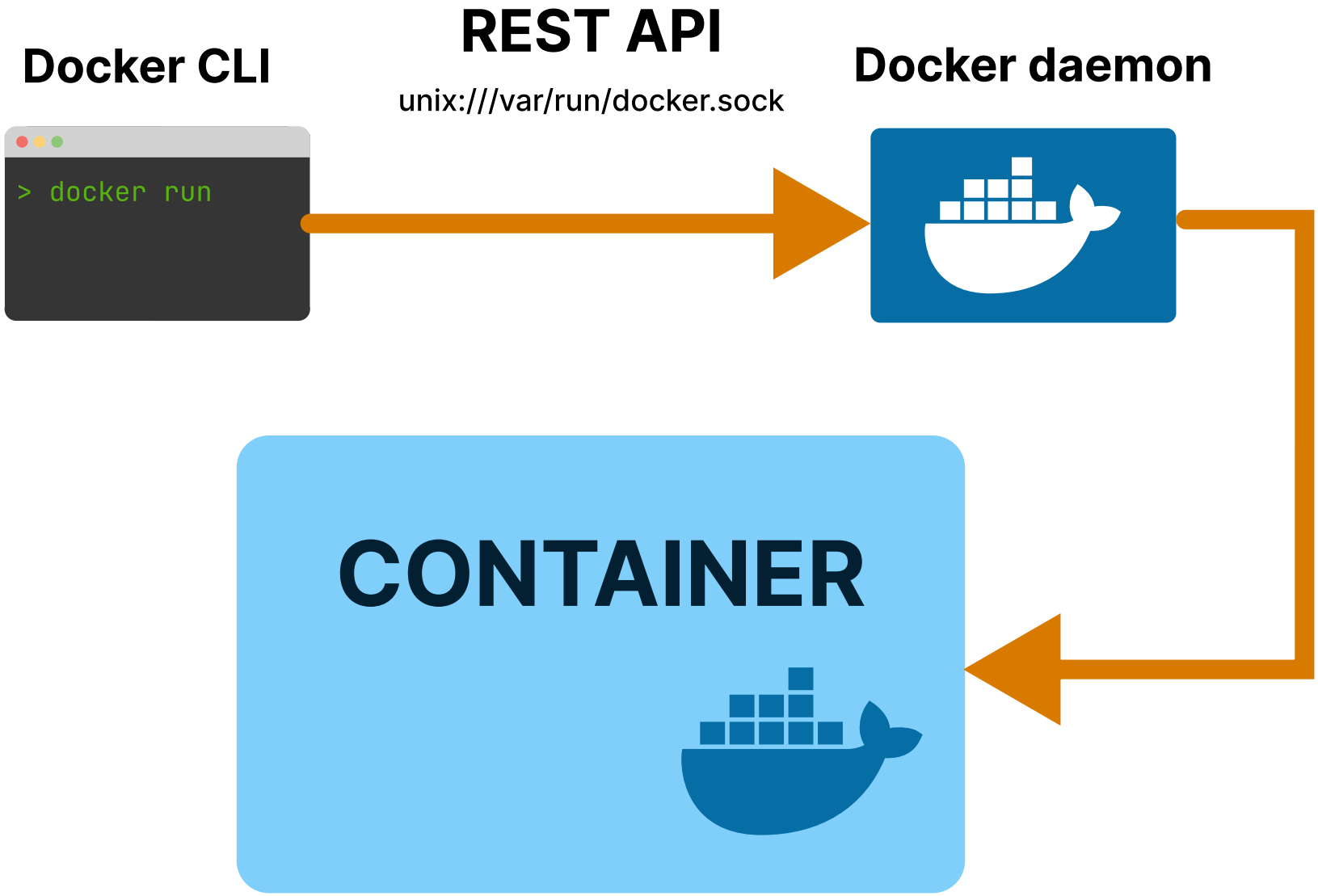
$ docker run -ti -v /tmp:/app ubuntu

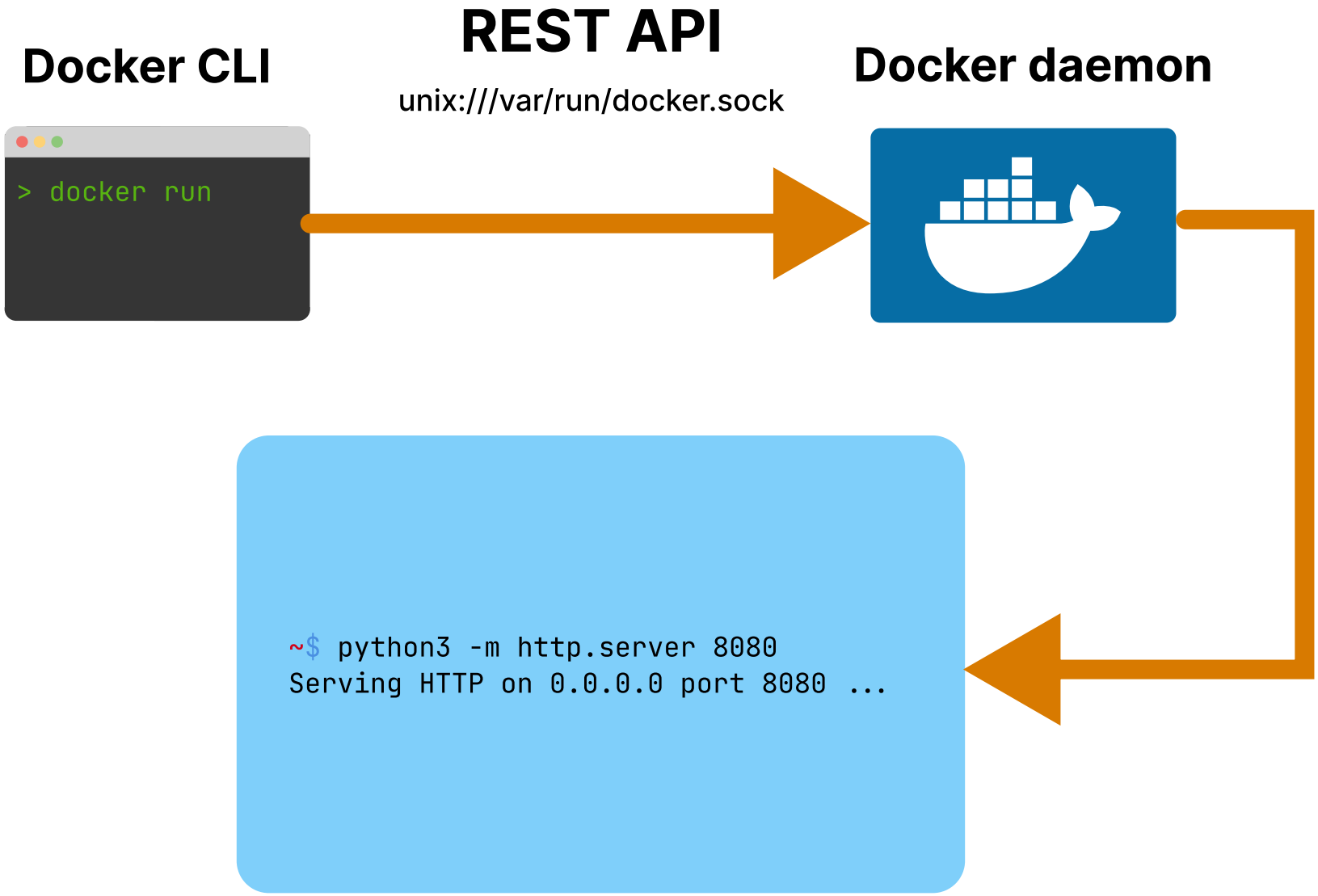


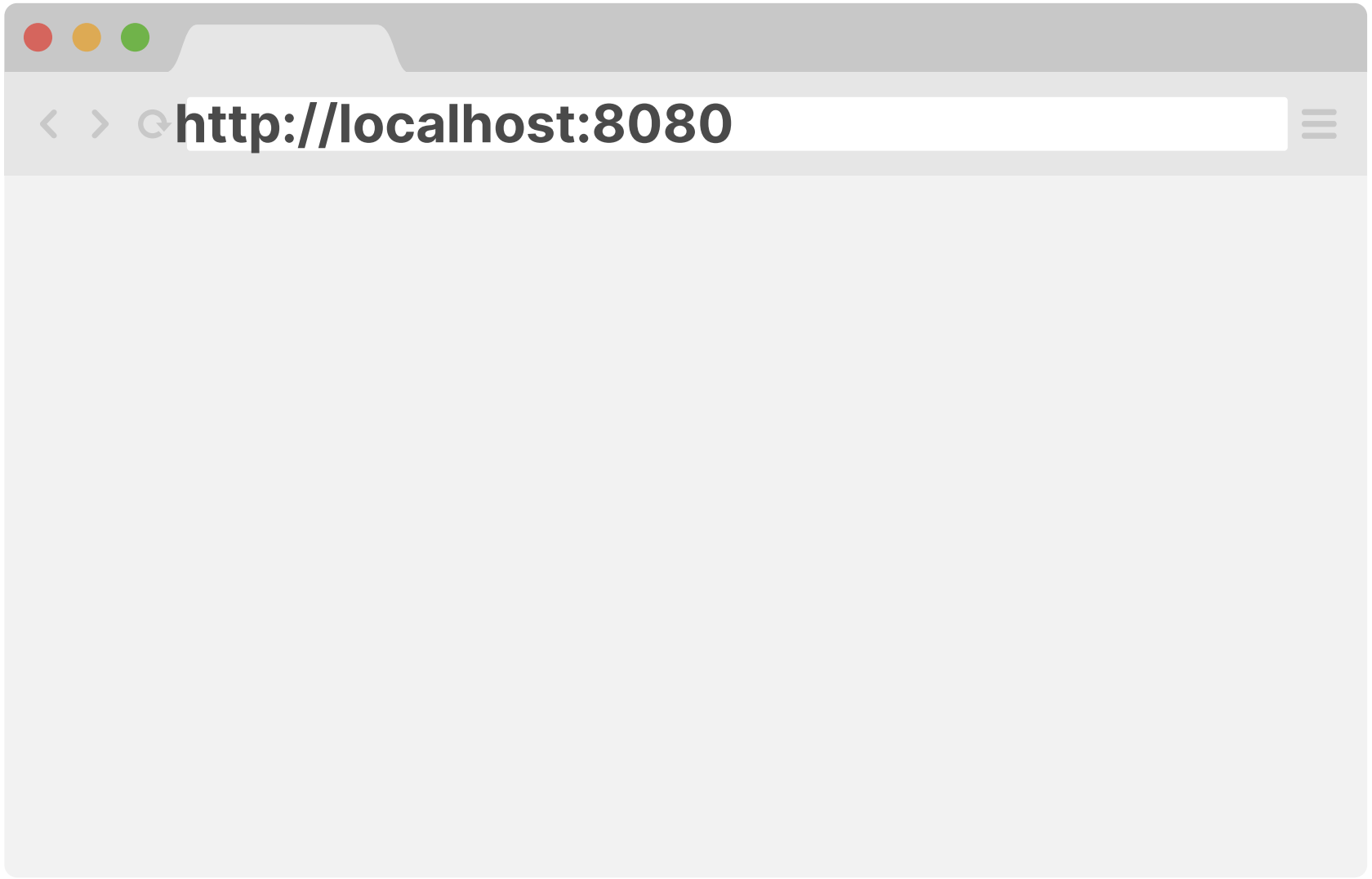
Forwarding ports

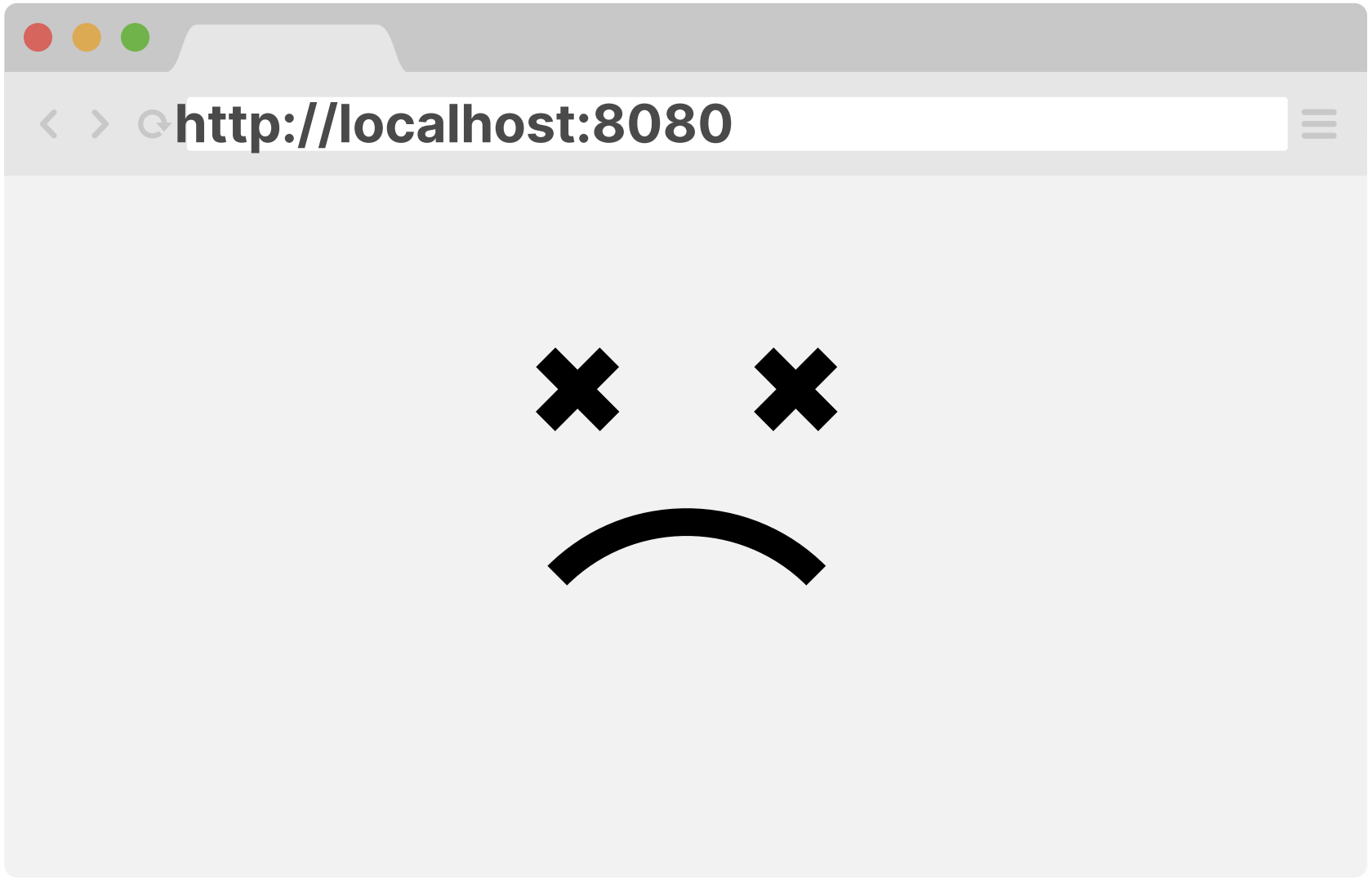
Docker

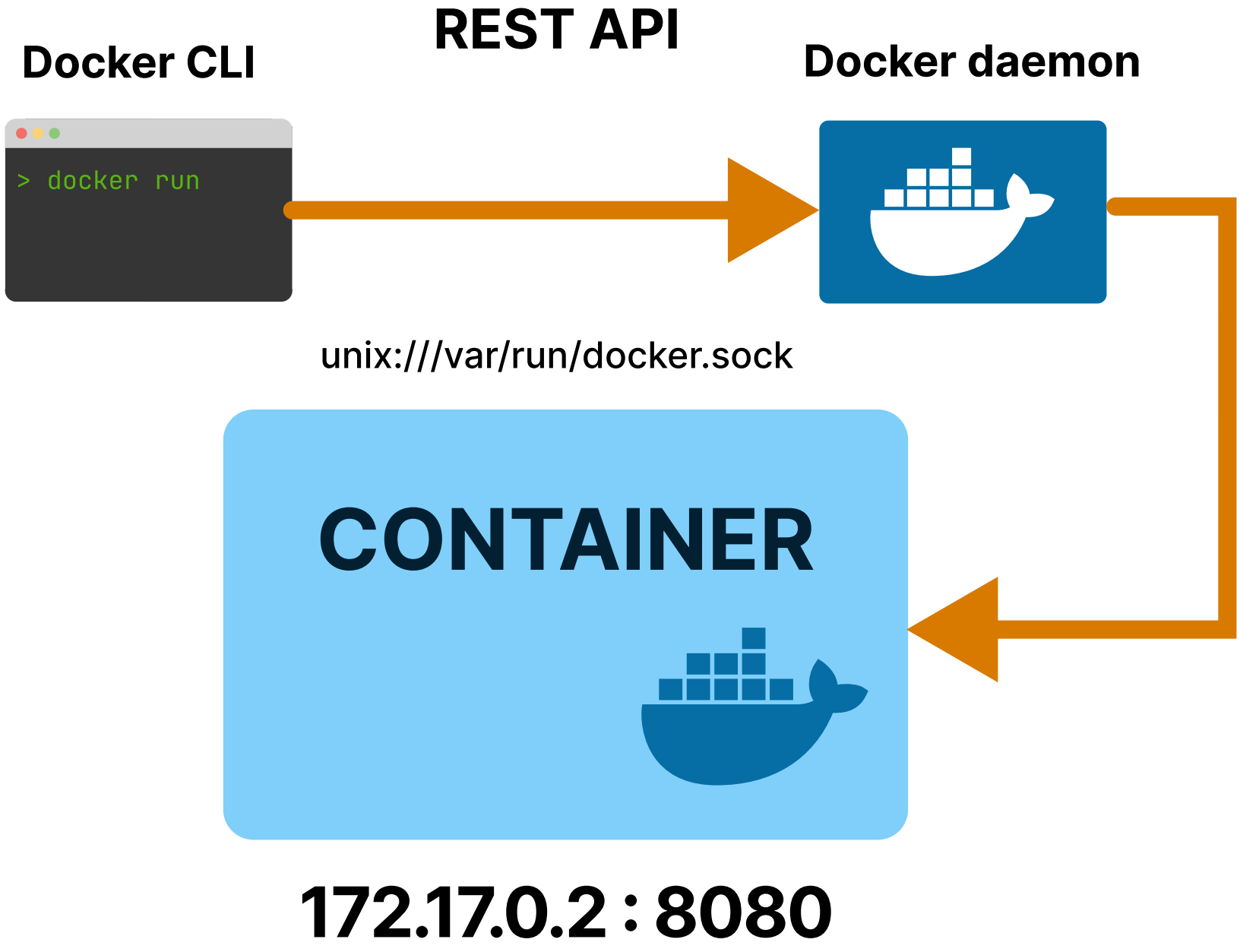

Docker CLI
REST API
unix:///var/run/docker.sock
Docker daemon
> docker run


REST API
Docker CLI
unix:///var/run/docker.sock
Docker daemon
> docker run
CONTAINER


REST API
Docker CLI
unix:///var/run/docker.sock
Docker daemon
> docker run
~$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 ...


http://localhost:8080


http://localhost:8080


REST API
Docker CLI
Docker daemon
> docker run
unix:///var/run/docker.sock
CONTAINER
172.17.0.2 : 8080

```
$ docker run -ti -p 3000:8080 ubuntu
```

```
$ docker run -ti -p 3000:8080 ubuntu
```

```
$ docker run -ti -p 3000:8080 ubuntu
```

```
$ docker run -ti -p 3000:8080 ubuntu
```

Injecting environment variables

Docker

```
$ docker run -ti -e ENV=test ubuntu
```

```
$ docker run -ti -e ENV=test ubuntu
```

Attaching to running containers

Docker

$ docker exec -ti abc123 bash

```
$ docker exec -ti abc123 bash
```

```
$ docker exec -ti abc123 bash
```

```
$ docker exec -ti abc123 bash
```

```
$ docker exec -ti abc123 bash
```

Managing containers

Docker

$ docker ps

List all running containers

$ docker stop [container id]

Wait for a container to stop

$ docker kill [container id]

Force a container to stop

Recap

Docker

Recap — Docker

1. Client-server architecture
2. Running containers
3. Dockerfile, images, containers
4. COPYing and mounting
5. Forwarding ports

Recap — Docker

1. Client-server architecture

2. Running containers

3. Dockerfile, images, containers

4. COPYing and mounting

5. Forwarding ports

Recap — Docker

1. Client-server architecture
2. Running containers
3. **Dockerfile, images, containers**
4. COPYing and mounting
5. Forwarding ports

Recap — Docker

1. Client-server architecture
2. Running containers
3. Dockerfile, images, containers
4. **COPYing and mounting**
5. Forwarding ports

Recap — Docker

1. Client-server architecture
2. Running containers
3. Dockerfile, images, containers
4. COPYing and mounting
5. **Forwarding ports**

Recap — Docker

6. Injecting environment variables

7. Attaching to running containers

3. Managing containers

Recap — Docker

6. Injecting environment variables

7. Attaching to running containers

3. Managing containers

Recap — Docker

6. Injecting environment variables

7. Attaching to running containers

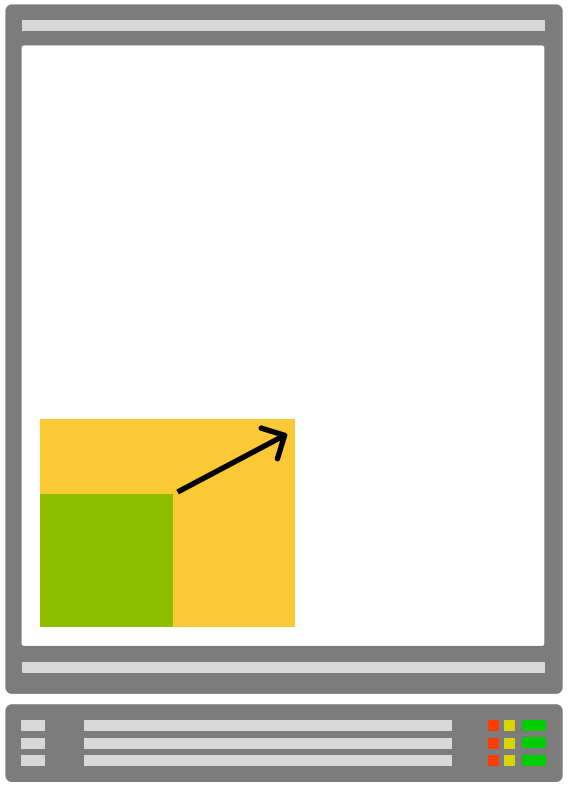
3. Managing containers

Containers vs virtual machines

Allocations

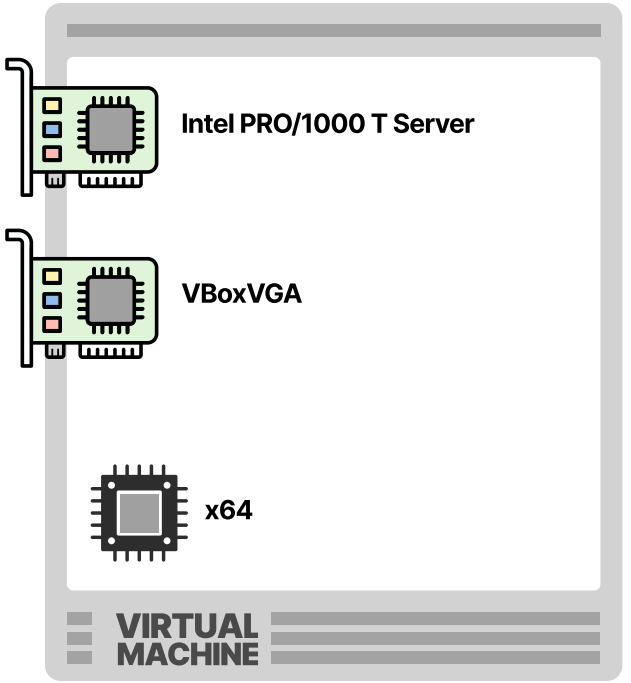
Containers vs virtual machines

Containers



Dynamic allocation

Virtual Machines

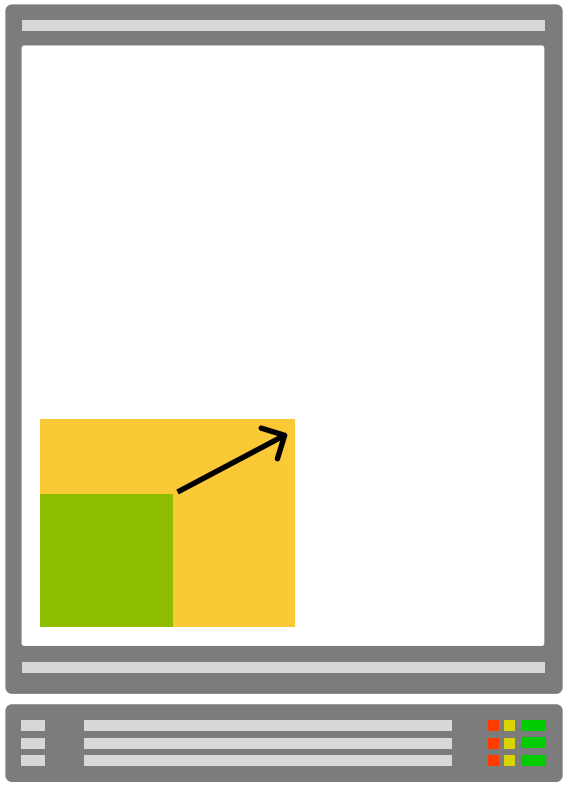


Fixed allocation

Performance

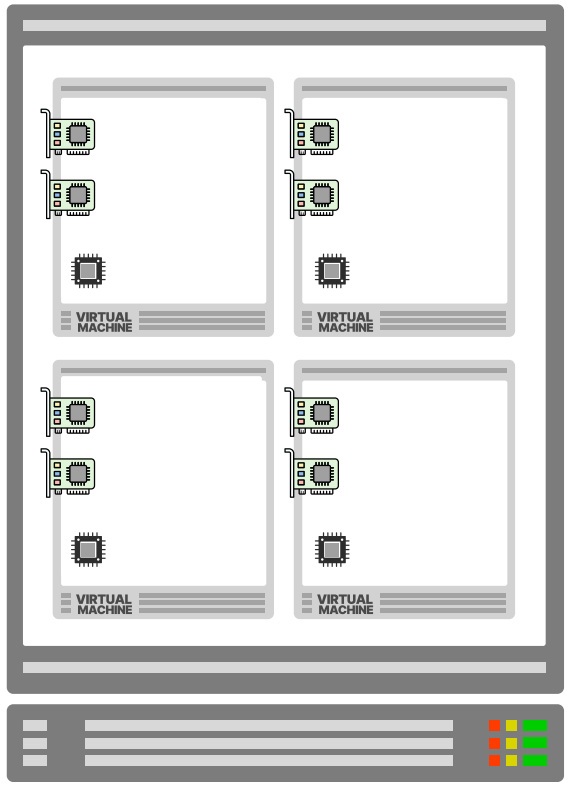
Containers vs virtual machines

Containers



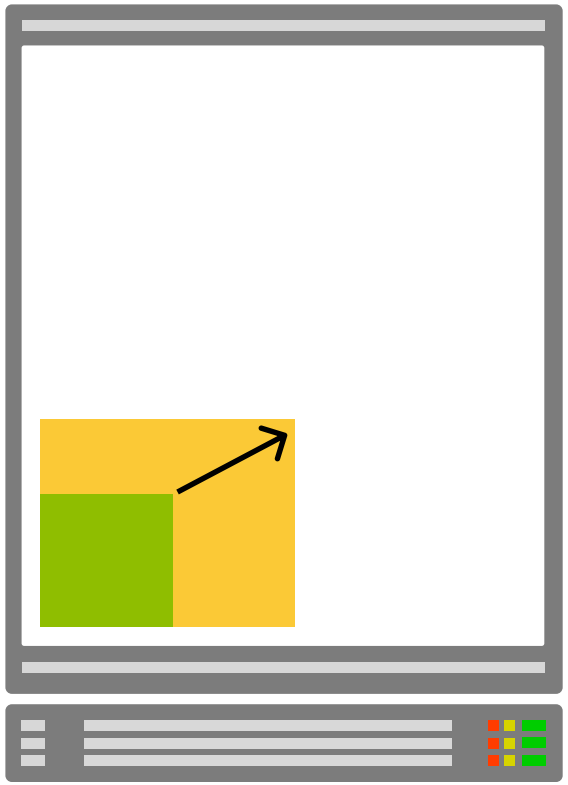
OS virtualization

Virtual Machines



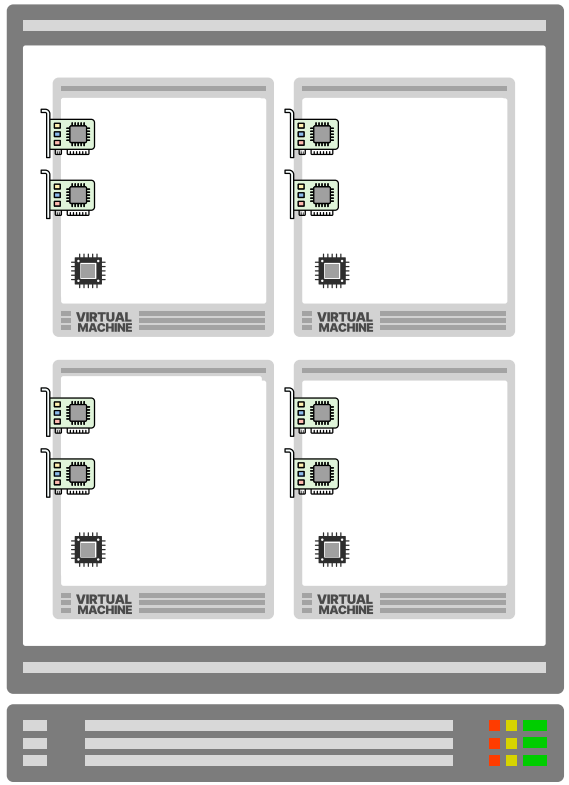
Hardware-level virtualisation

Containers



Startup time in milliseconds

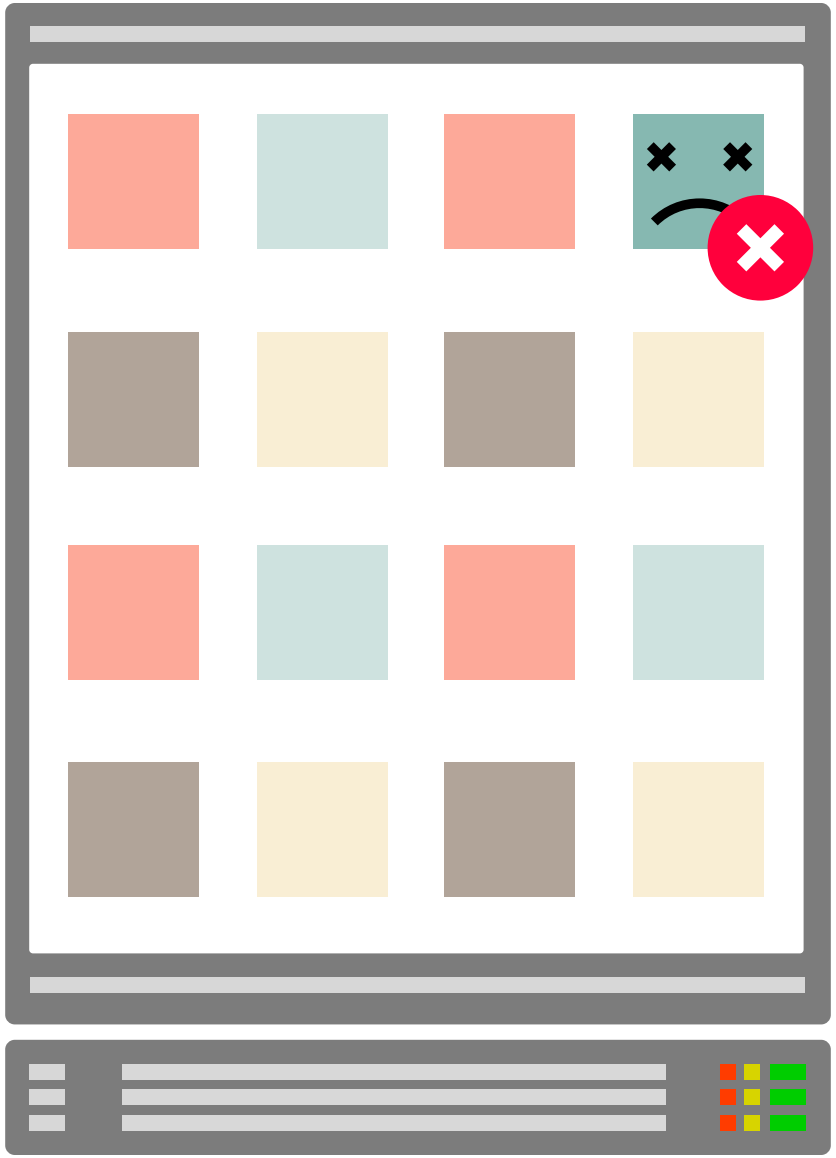
Virtual Machines

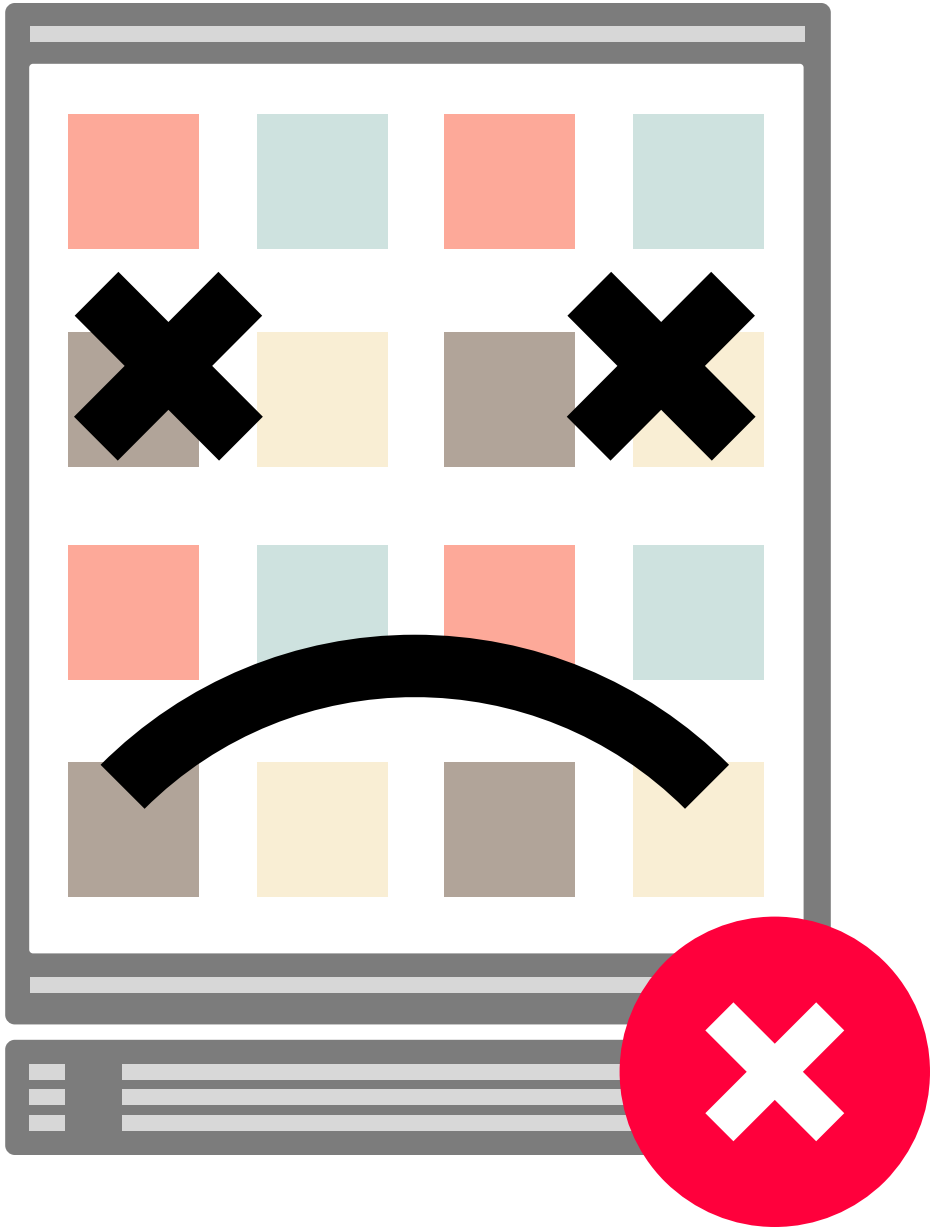


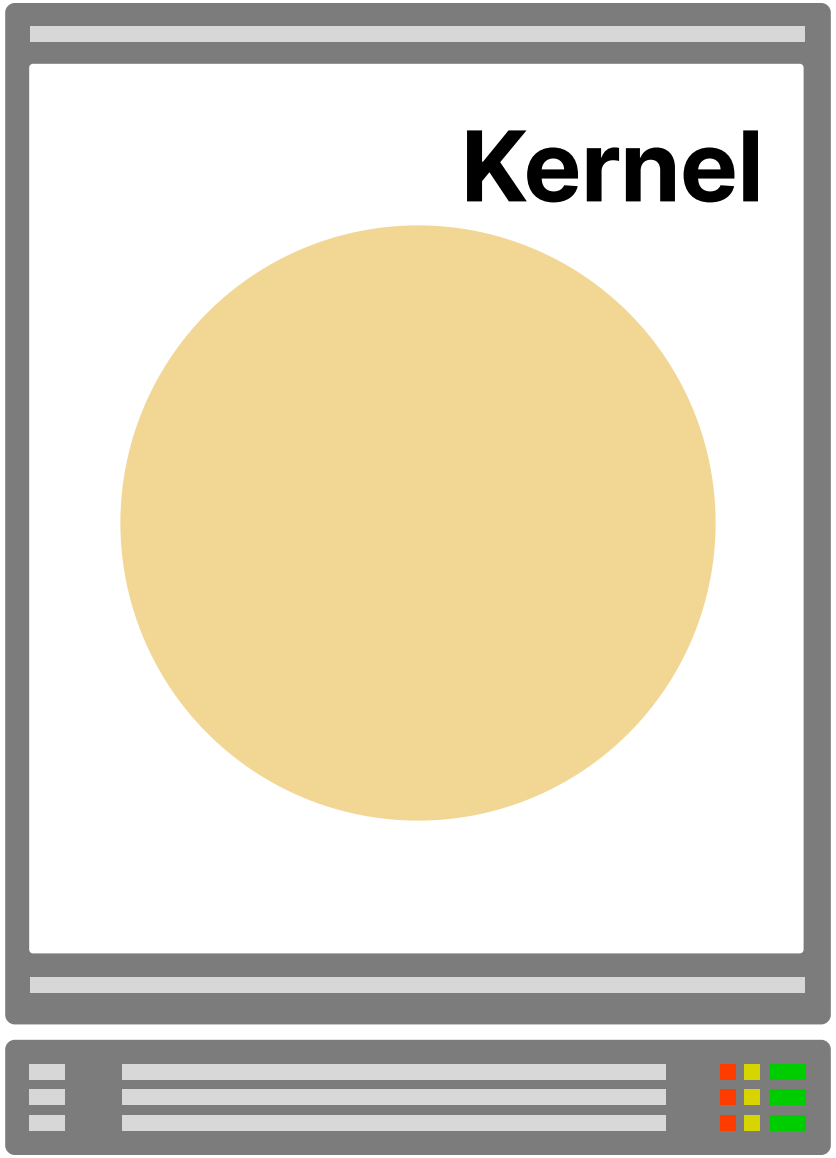
Startup time in minutes

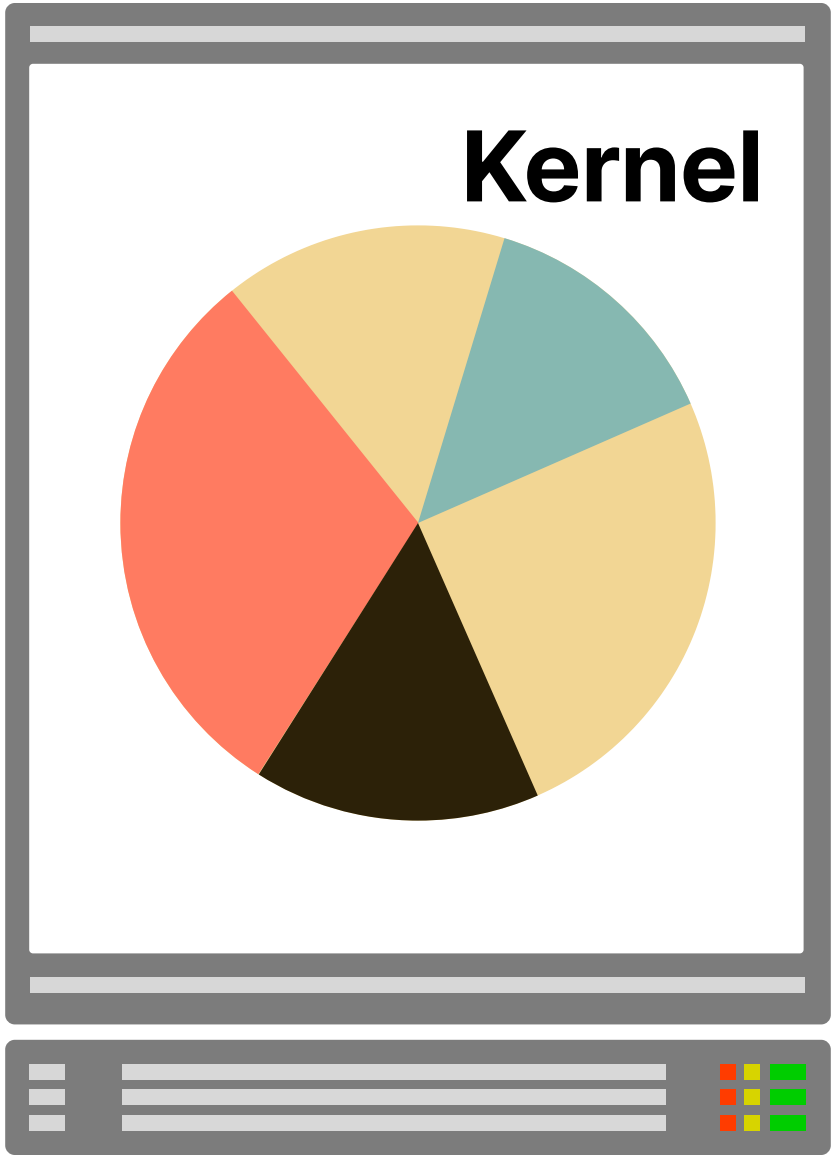
Isolation

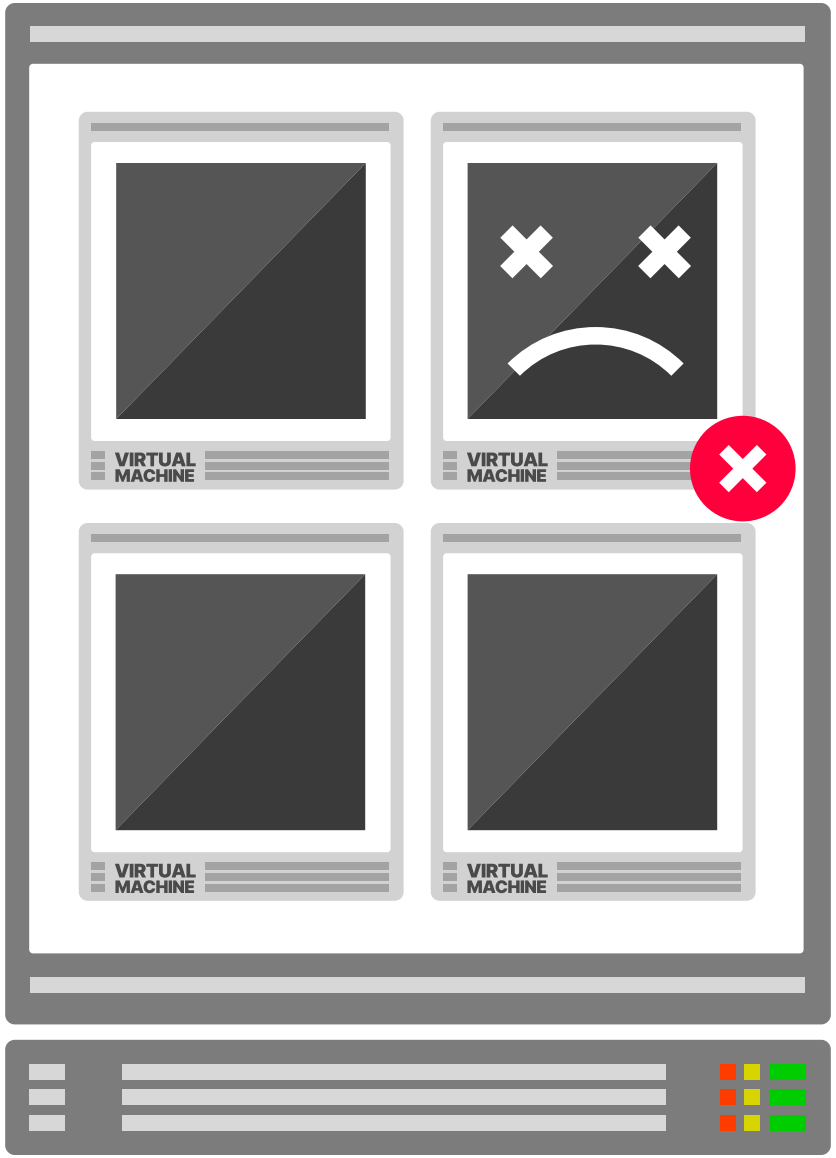
Containers vs virtual machines

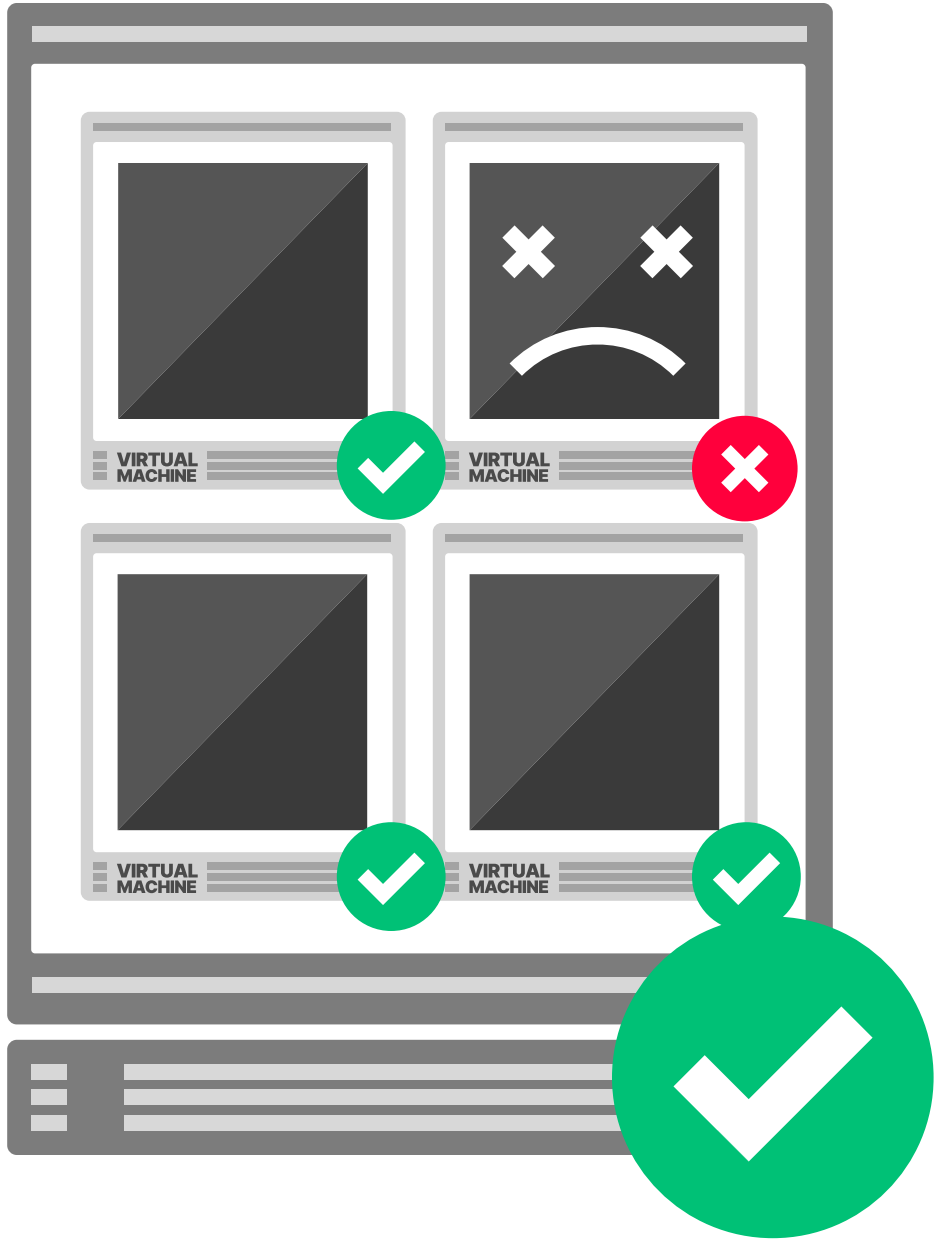


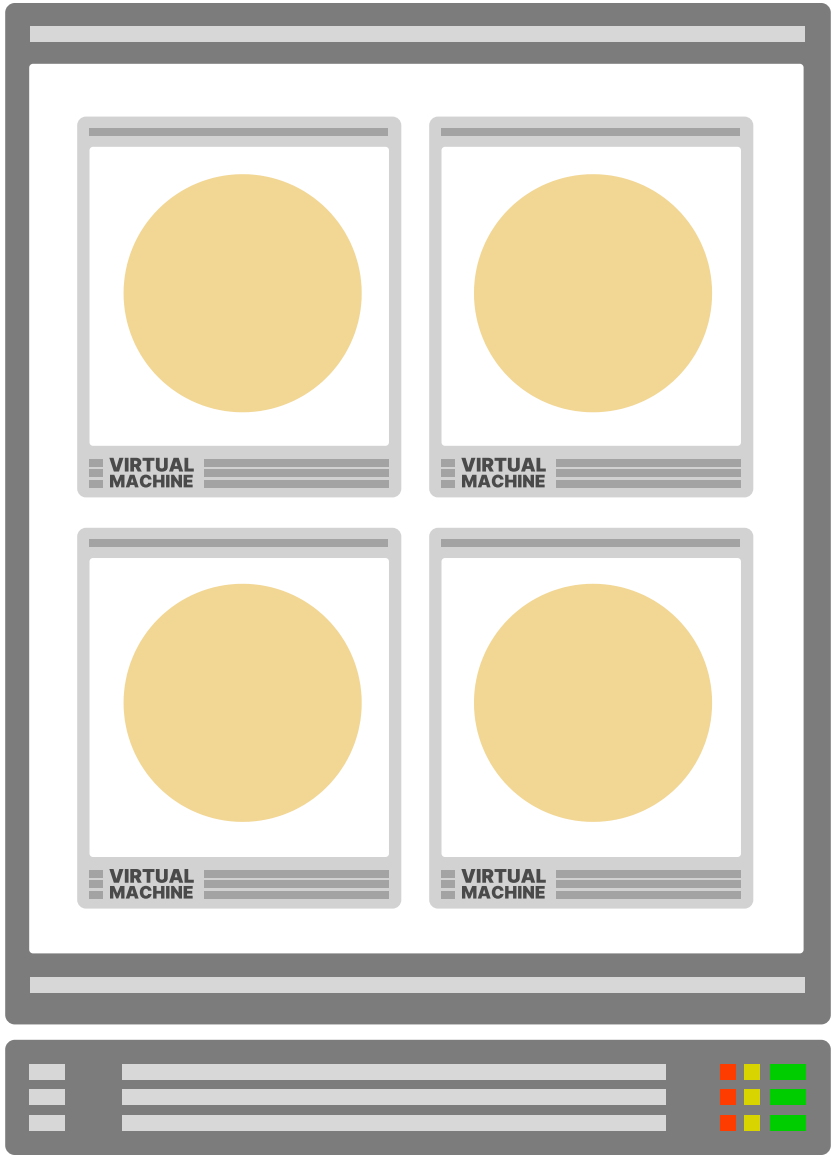



Kernel


Kernel


VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE


VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE


VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE

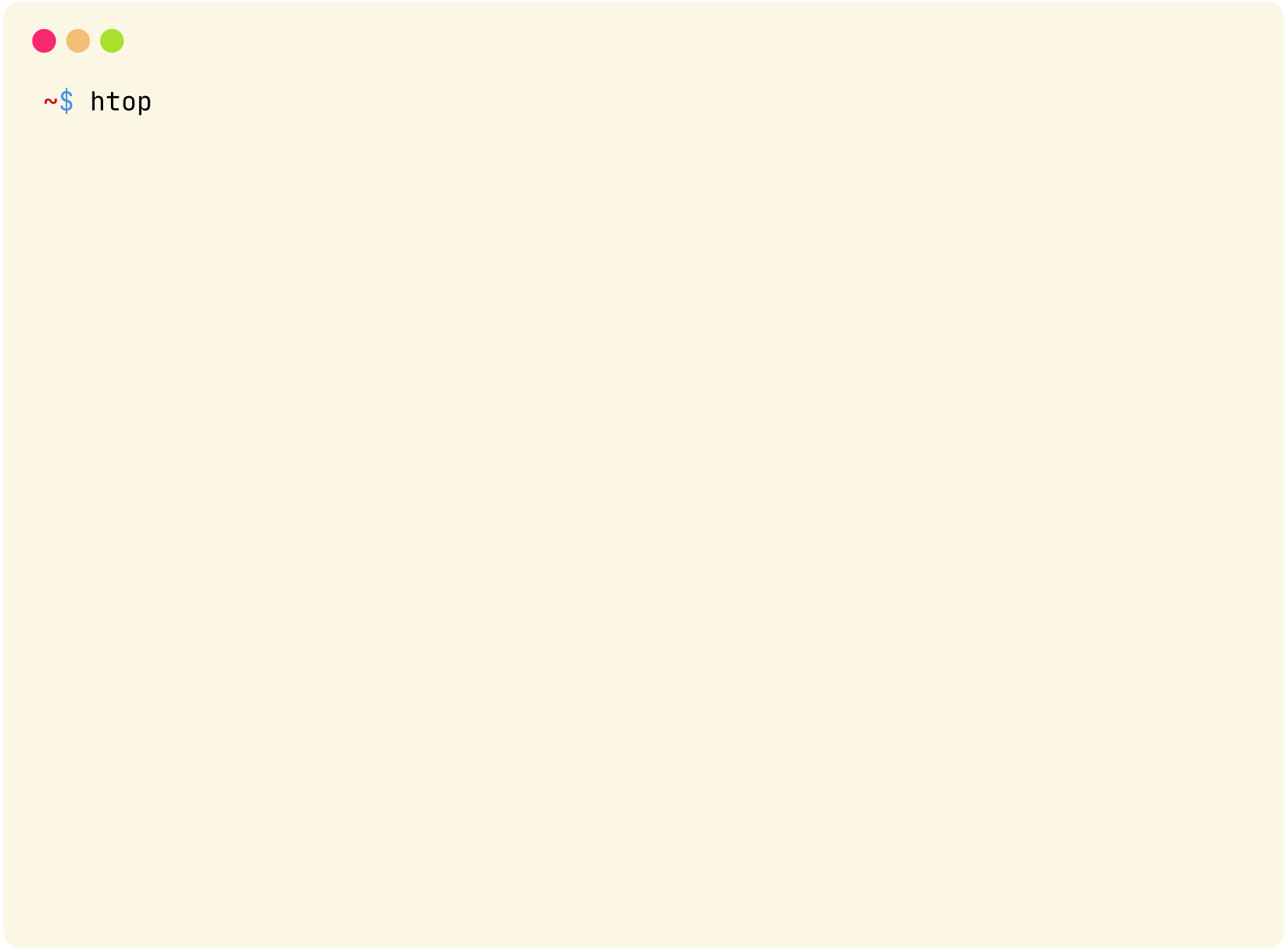

VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE
VIRTUAL MACHINE

Containers are processes

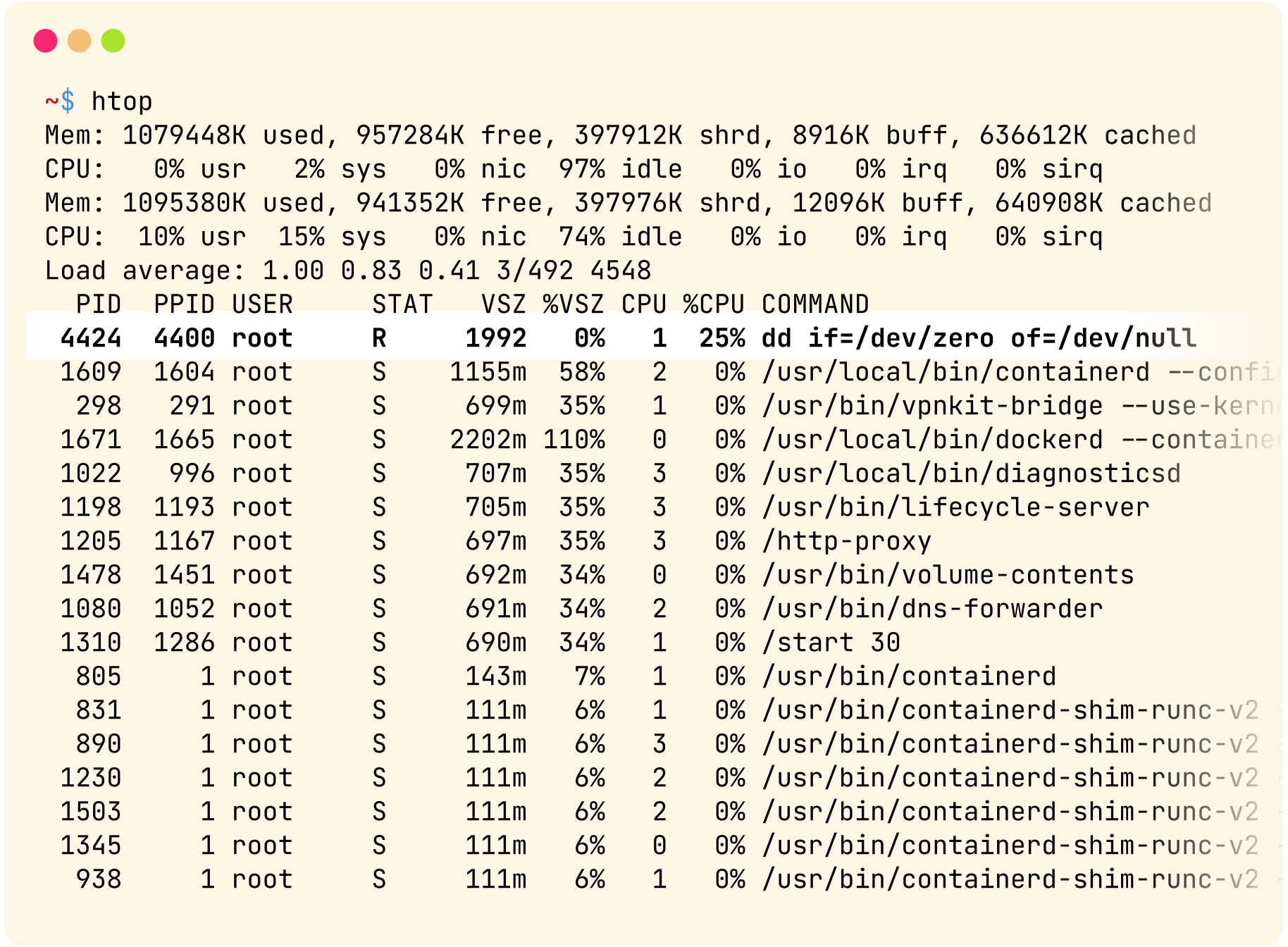
Containers vs virtual machines

```
$ docker run -ti ubuntu dd if=/dev/zero of=/dev/null
```

```
$ docker run -ti ubuntu dd if=/dev/zero of=/dev/null
```


~$ htop

```
~$ htop
Mem: 1079448K used, 957284K free, 397912K shrd, 8916K buff, 636612K cached
CPU:   0% usr   2% sys   0% nic  97% idle   0% io   0% irq   0% sirq
Mem: 1095380K used, 941352K free, 397976K shrd, 12096K buff, 640908K cached
CPU:  10% usr  15% sys   0% nic  74% idle   0% io   0% irq   0% sirq
Load average: 1.00 0.83 0.41 3/492 4548
  PID  PPID USER     STAT   VSZ %VSZ CPU %CPU COMMAND
 4424  4400 root     R     1992   0%   1  25% dd if=/dev/zero of=/dev/null
 1609  1604 root     S    1155m  58%   2   0% /usr/local/bin/containerd --confi
  298   291 root     S     699m  35%   1   0% /usr/bin/vpnkit-bridge --use-kern
 1671  1665 root     S    2202m 110%   0   0% /usr/local/bin/dockerd --containe
 1022   996 root     S     707m  35%   3   0% /usr/local/bin/diagnosticsd
 1198  1193 root     S     705m  35%   3   0% /usr/bin/lifecycle-server
 1205  1167 root     S     697m  35%   3   0% /http-proxy
 1478  1451 root     S     692m  34%   0   0% /usr/bin/volume-contents
 1080  1052 root     S     691m  34%   2   0% /usr/bin/dns-forwarder
 1310  1286 root     S     690m  34%   1   0% /start 30
  805     1 root     S     143m   7%   1   0% /usr/bin/containerd
  831     1 root     S     111m   6%   1   0% /usr/bin/containerd-shim-runc-v2 -
  890     1 root     S     111m   6%   3   0% /usr/bin/containerd-shim-runc-v2 -
 1230     1 root     S     111m   6%   2   0% /usr/bin/containerd-shim-runc-v2 -
 1503     1 root     S     111m   6%   2   0% /usr/bin/containerd-shim-runc-v2 -
 1345     1 root     S     111m   6%   0   0% /usr/bin/containerd-shim-runc-v2 -
  938     1 root     S     111m   6%   1   0% /usr/bin/containerd-shim-runc-v2 -
```


~$ htop
Mem: 1079448K used, 957284K free, 397912K shrd, 8916K buff, 636612K cached
CPU: 0% usr   2% sys   0% nic 97% idle   0% io   0% irq   0% sirq
Mem: 1095380K used, 941352K free, 397976K shrd, 12096K buff, 640908K cached
CPU: 10% usr 15% sys   0% nic 74% idle   0% io   0% irq   0% sirq
Load average: 1.00 0.83 0.41 3/492 4548
PID PPID USER STAT VSZ %VSZ CPU %CPU COMMAND
4424 4400 root R 1992 0% 1 25% dd if=/dev/zero of=/dev/null
1609 1604 root S 1155m 58% 2 0% /usr/local/bin/containerd --confi
298 291 root S 699m 35% 1 0% /usr/bin/vpnkit-bridge --use-kern
1671 1665 root S 2202m 110% 0 0% /usr/local/bin/dockerd --containe
1022 996 root S 707m 35% 3 0% /usr/local/bin/diagnosticsd
1198 1193 root S 705m 35% 3 0% /usr/bin/lifecycle-server
1205 1167 root S 697m 35% 3 0% /http-proxy
1478 1451 root S 692m 34% 0 0% /usr/bin/volume-contents
1080 1052 root S 691m 34% 2 0% /usr/bin/dns-forwarder
1310 1286 root S 690m 34% 1 0% /start 30
805 1 root S 143m 7% 1 0% /usr/bin/containerd
831 1 root S 111m 6% 1 0% /usr/bin/containerd-shim-runc-v2
890 1 root S 111m 6% 3 0% /usr/bin/containerd-shim-runc-v2
1230 1 root S 111m 6% 2 0% /usr/bin/containerd-shim-runc-v2
1503 1 root S 111m 6% 2 0% /usr/bin/containerd-shim-runc-v2
1345 1 root S 111m 6% 0 0% /usr/bin/containerd-shim-runc-v2
938 1 root S 111m 6% 1 0% /usr/bin/containerd-shim-runc-v2

Recap

Containers vs virtual machines

Recap — Containers vs virtual machines

1. Isolation
2. Startup time
3. Virtualization
4. Allocations
5. Containers are processes

Recap — Containers vs virtual machines

1. Isolation

2. Startup time

3. Virtualization

4. Allocations

5. Containers are processes

Recap — Containers vs virtual machines

1. Isolation

2. Startup time

3. Virtualization

4. Allocations

5. Containers are processes

Recap — Containers vs virtual machines

1. Isolation

2. Startup time

3. Virtualization

4. Allocations

5. Containers are processes

Recap — Containers vs virtual machines

1. Isolation
2. Startup time
3. Virtualization
4. Allocations
5. **Containers are processes**

Thanks.